

Software Development Tools & Practices

PUSL2020

Level 2

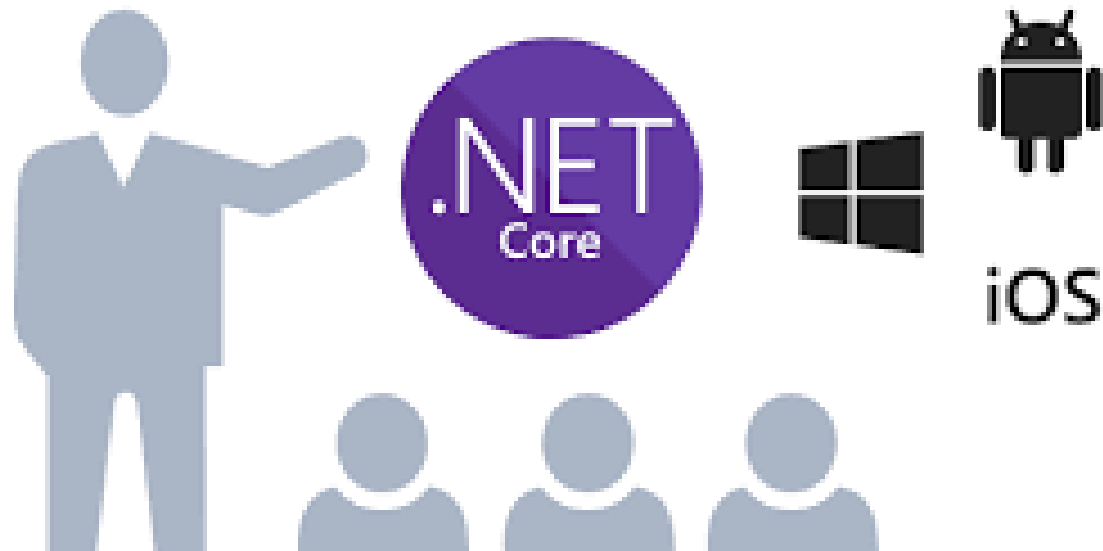
“APS.NET Development”

Dr. Rasika Ranaweera (ranaweera.r@nsbm.ac.lk | +94 11 544 6126)

Faculty of Computing | NSBM Green University

Understanding ASP.NET Core

- ASP.NET Core is built on .NET Core, which is a cross-platform version of the .NET Framework without the Windows-specific application programming interfaces (APIs).
- Microsoft has extended the reach of .NET, made it possible to deploy ASP.NET Core applications to a broader set of hosting environments, and, as a bonus, made it possible for developers to create ASP.NET Core web applications on Linux and macOS.



ASP.NET Core MVC

- ASP.NET Core MVC provides the functionality of the original ASP.NET MVC Framework built on the new ASP.NET Core platform.
- It integrates the features that were previously provided by Web API, it includes a more natural way of generating complex content, and it makes key development tasks, such as unit testing, simpler and more predictable.
- The ASP.NET Core MVC 2 release focuses on consolidation, working through some of the tooling and platform changes that were introduced in earlier versions.

Key Benefits of ASP.NET Core MVC

- 1. MVC Architecture

- User interaction with an application that adheres to the MVC pattern follows a natural cycle: the user takes an action, and in response, the application changes its data model and delivers an updated view to the user.
- Web applications necessitate combining several technologies (databases, HTML, and executable code, for example), usually split into a set of tiers or layers.

- 2. Extensibility

- We can easily replace key components with ones of your own implementation

- 3. Tight Control over HTML and HTTP

- 4. Testability

- 5. Powerful Routing System

- 6. Modern API and Cross-Platform

- 7. Open Source

ASP.NET Core in Visual Studio

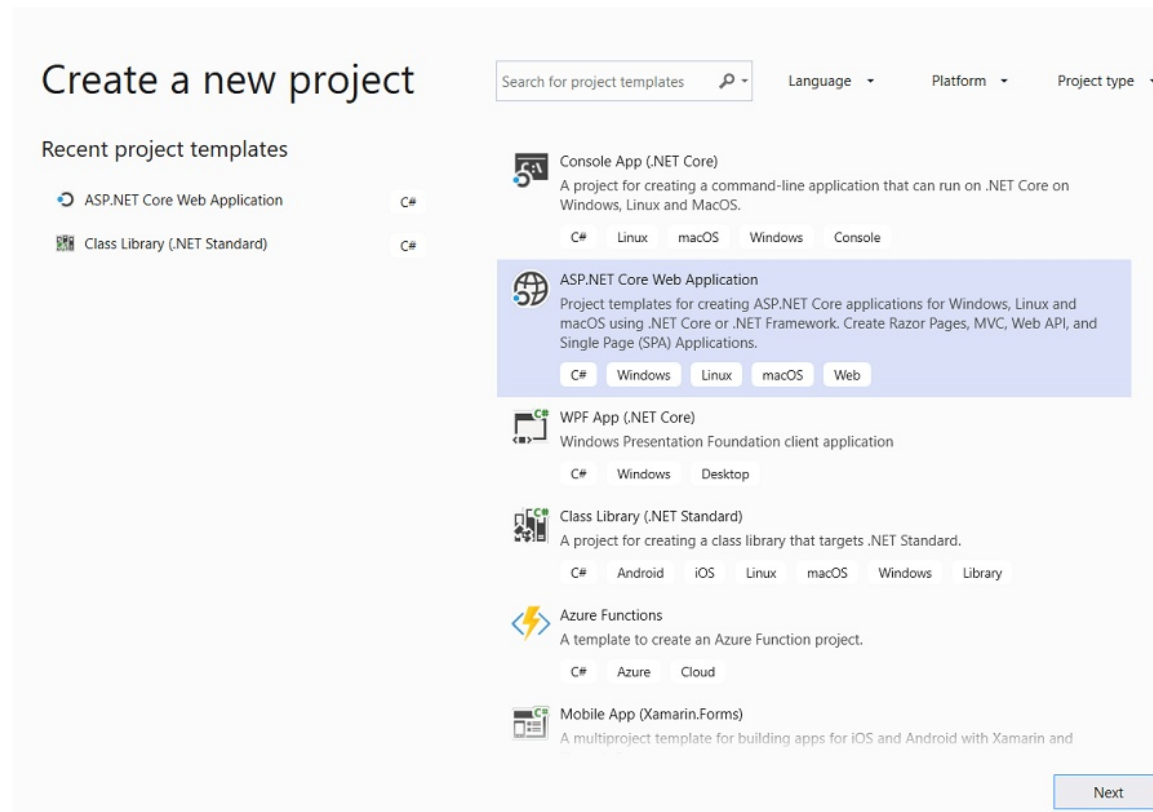
new ASP.NET Core Web Application - webApplication24

The image shows the Visual Studio interface for creating a new ASP.NET Core web application. On the left, the 'Create a new ASP.NET Core Web Application' dialog is open. The 'ASP.NET Core' dropdown is set to 'ASP.NET Core 2.1'. The 'Web Application (Model-View-Controller)' template is selected. The right pane shows the 'Create a new ASP.NET Core Web Application' dialog with the following options:

- Authentication:** No Authentication (Change)
- Advanced:**
 - ☒ Configure for HTTPS
 - ☐ Enable Docker Support (Requires Docker Desktop)
- OS:** Linux
- Author:** Microsoft
- Source:** SDK 3.0.100-preview5-011568

The 'Web Application (Model-View-Controller)' template is highlighted in blue. The 'Back' and 'Create' buttons are at the bottom right.

Practice I: Creating a New ASP.NET Core MVC Project



1. From the Visual Studio select **Create a new project**.

2. Select **ASP.NET Core Web Application** and then select **Next**.

3. Name the project **MvcMovie** and select Create. It's important to name the project **MvcMovie** so when you copy code, the namespace will match.

4. Select **Web Application(Model-View-Controller)**, and then select **Create**.

Create a new ASP.NET Core web application

.NET Core

ASP.NET Core 3.0

 **Empty**

An empty project template for creating an ASP.NET Core application. This template does not have any content in it.

 **API**

A project template for creating an ASP.NET Core application with an example Controller for a RESTful HTTP service. This template can also be used for ASP.NET Core MVC Views and Controllers.

 **Web Application**

A project template for creating an ASP.NET Core application with example ASP.NET Razor Pages content.

 **Web Application (Model-View-Controller)**

A project template for creating an ASP.NET Core application with example ASP.NET Core MVC Views and Controllers. This template can also be used for RESTful HTTP services.

 **Angular**

A project template for creating an ASP.NET Core application with Angular

 **React.js**

Authentication

No Authentication

[Change](#)

Advanced

☒ Configure for HTTPS

☐ Enable Docker Support

(Requires [Docker Desktop](#))

Linux

Author: Microsoft

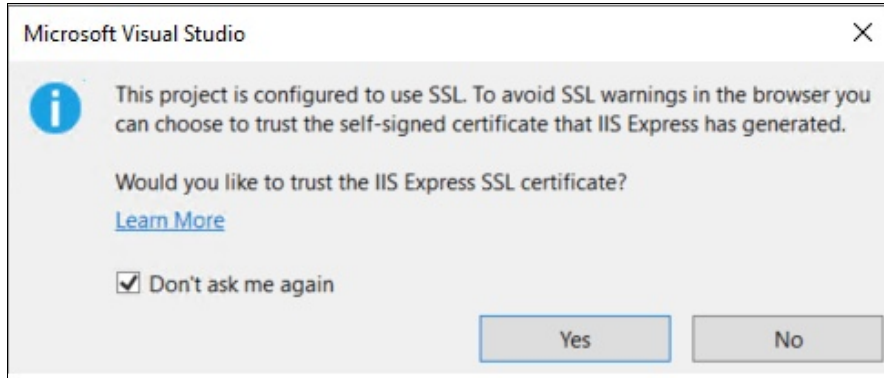
Source: .NET Core 3.0.0

Get additional project templates

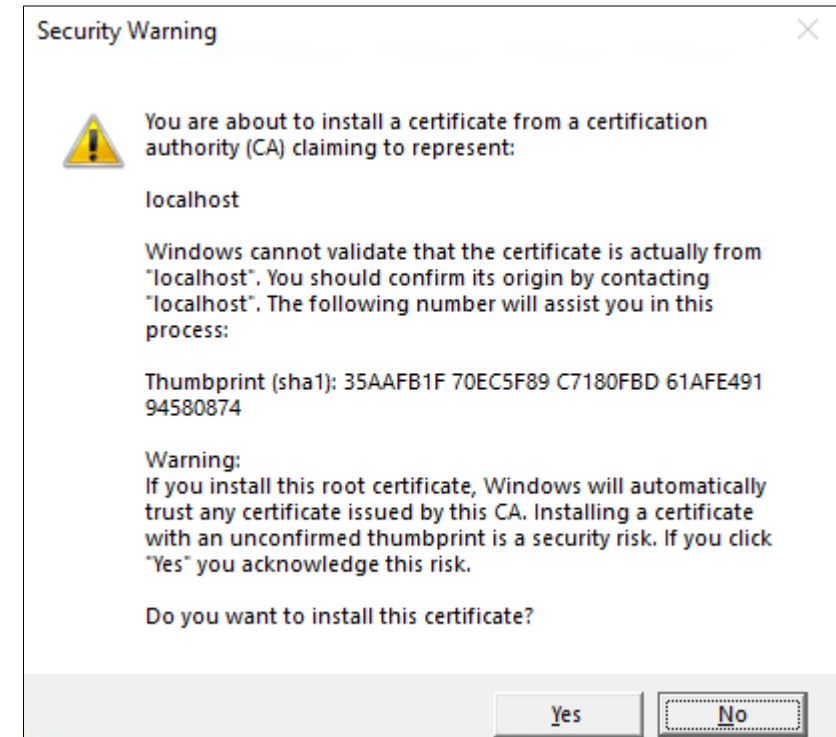
Back

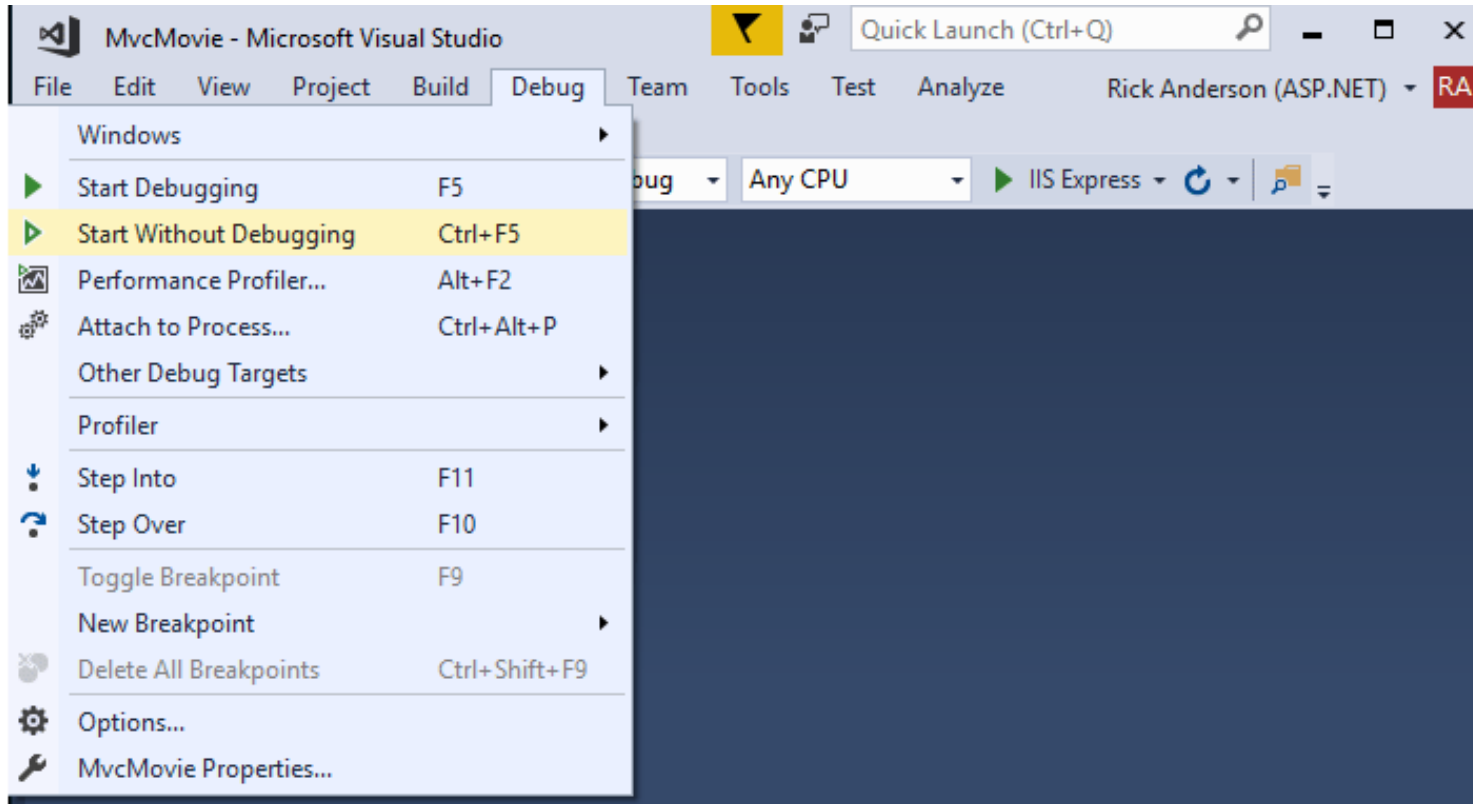
Create

Select **Ctrl-F5** to run the app in non-debug mode.
Visual Studio displays the following dialog:



Select **Yes** if you trust the IIS Express SSL
certificate.
The following dialog is displayed:

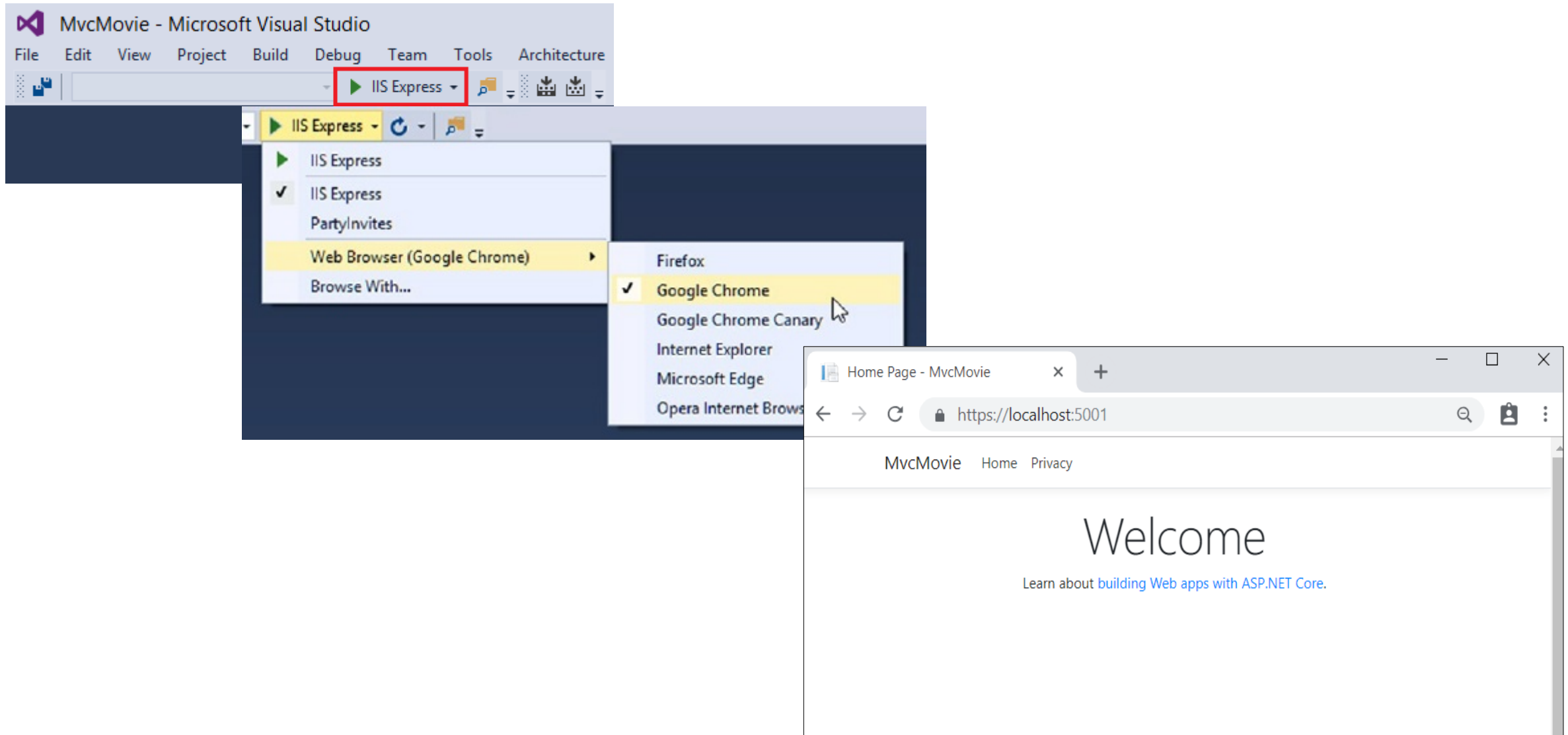




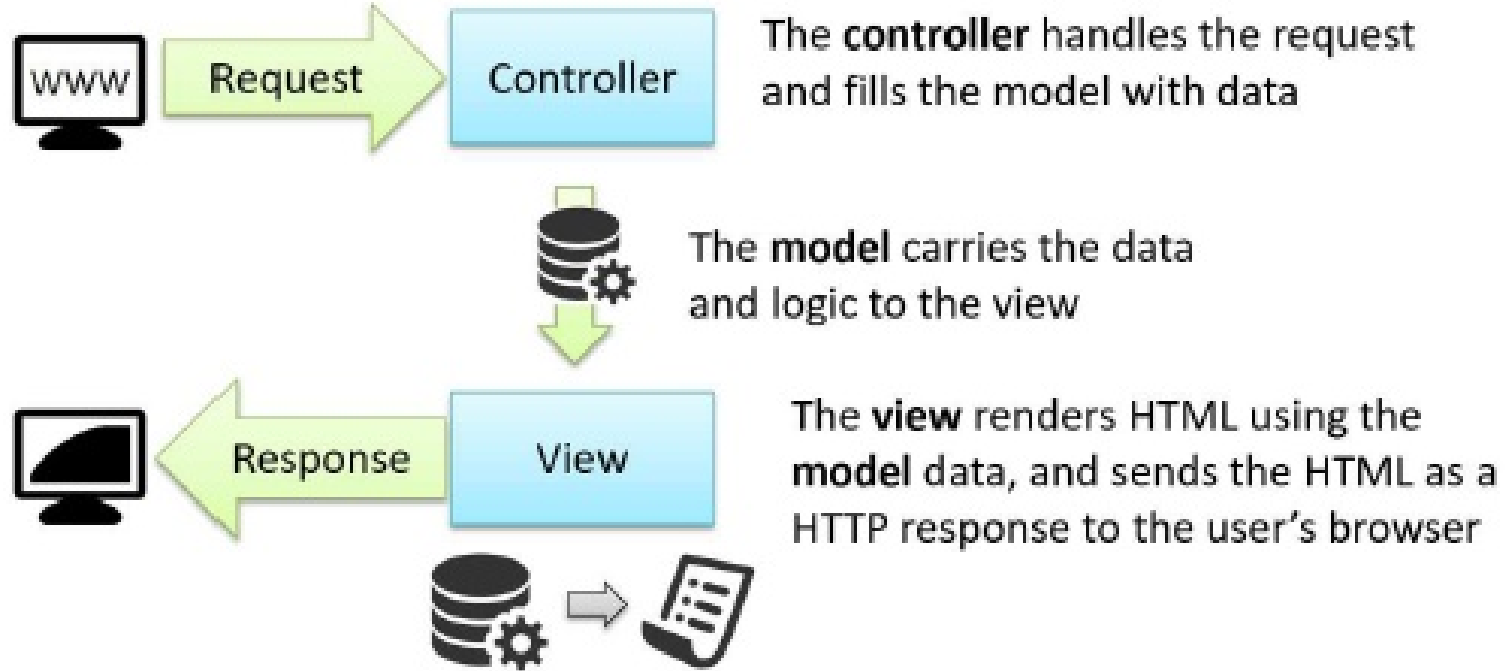
Select **Yes** if you agree to trust the development certificate.

- Visual Studio starts [IIS Express](#) and runs the app. Notice that the address bar shows localhost:port# and not something like example.com. That's because localhost is the standard hostname for your local computer. When Visual Studio creates a web project, a random port is used for the web server.
- Launching the app with Ctrl+F5 (non-debug mode) allows you to make code changes, save the file, refresh the browser, and see the code changes. Many developers prefer to use non-debug mode to quickly launch the app and view changes.
- You can launch the app in debug or non-debug mode from the **Debug** menu item:

You can debug the app by selecting the **IIS Express** button



Understanding the MVC Pattern



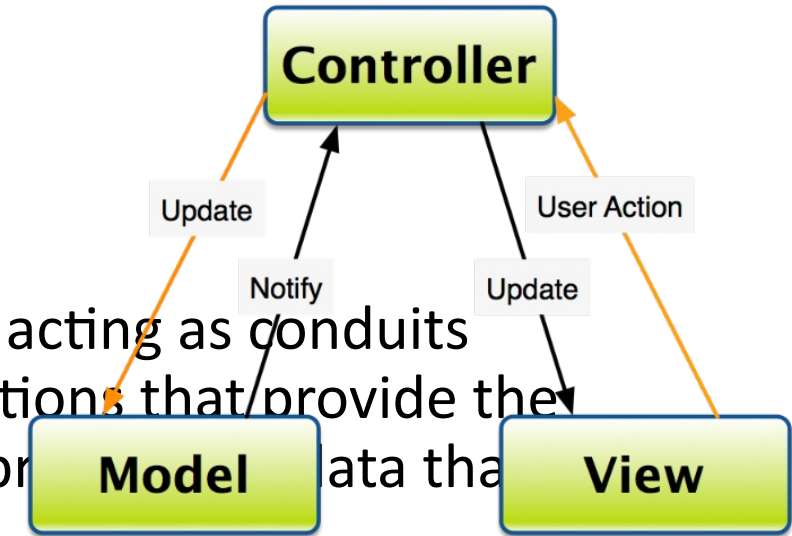
The term *model-view-controller* has been in use since the late 1970s and arose from the Smalltalk project at Xerox PARC, where it was conceived as a way to organize some early GUI applications. Some of the fine detail of the original MVC pattern was tied to Smalltalk-specific concepts, such as *screens* and *tools*, but the broader concepts are still applicable to applications, and they are especially well-suited to web applications.

MVC- Models

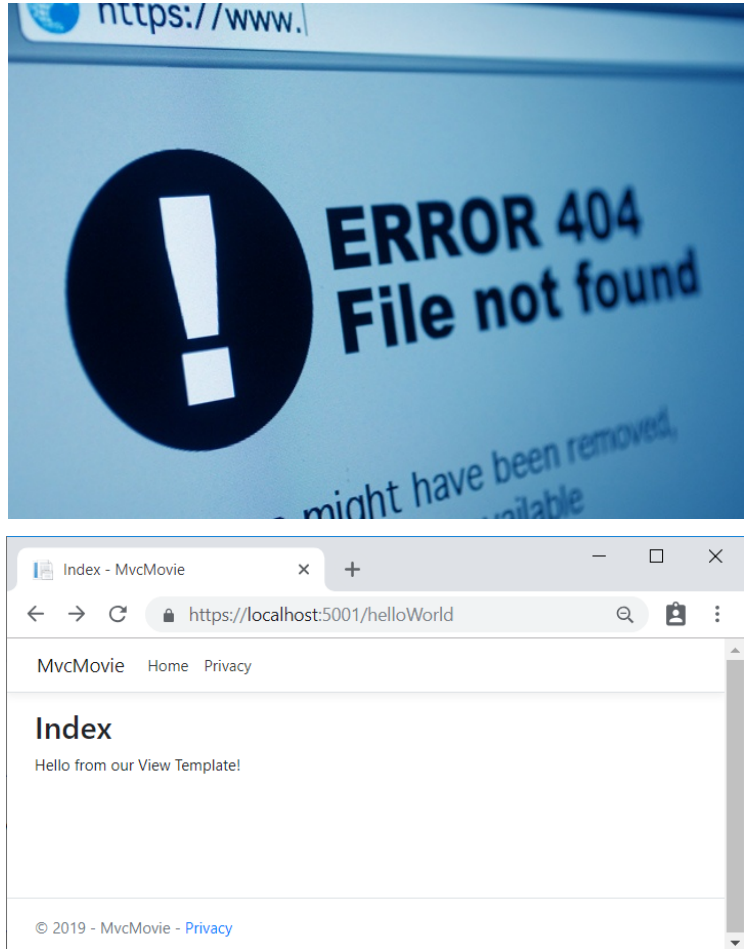
- For each of the components in the MVC pattern, I'll describe what should and should not be included.
- The model in an application built using the MVC pattern *should*
 - Contain the domain data
 - Contain the logic for creating, managing, and modifying the domain data
 - Provide a clean API that exposes the model data and operations on it
- The model *should not*
 - Expose details of how the model data is obtained or managed (in other words, details of the data storage mechanism should not be exposed to controllers and views)
 - Contain logic that transforms the model based on user interaction (because that is the controller's job)
 - Contain logic for displaying data to the user (that is the view's job)

MVC- Controllers

- Controllers are the connective tissue in the MVC pattern, acting as conduits between the data model and views. Controllers define actions that provide the business logic that operates on the data model and that provide data that the views display to the user.
- A controller built using the MVC pattern *should*
 - Contain the actions required to update the model based on user interaction
- The controller *should not*
 - Contain logic that manages the appearance of data (that is the job of the view)
 - Contain logic that manages the persistence of data (that is the job of the model)



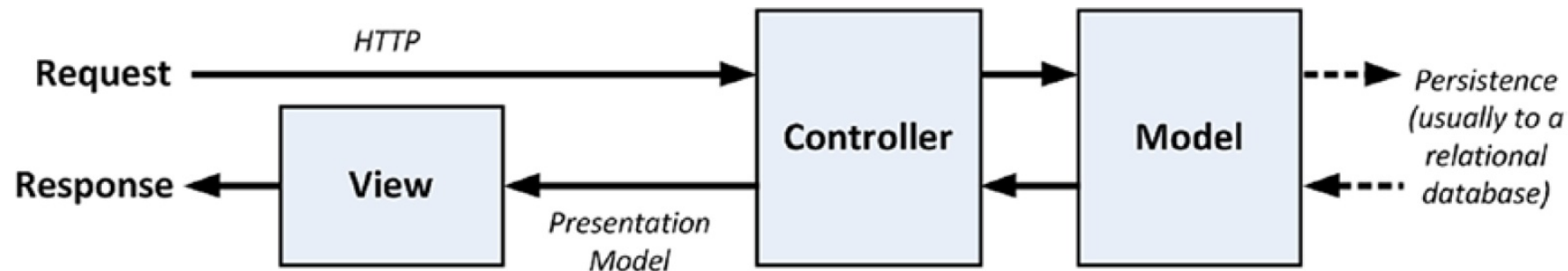
MVC- Views



- Views contain the logic required to display data to the user or to capture data from the user so that it can be processed by a controller action. Views *should*
 - Contain the logic and markup required to present data to the user
- Views *should not*
 - Contain complex logic (this is better placed in a controller)
 - Contain logic that creates, stores, or manipulates the domain model
- Views *can* contain logic, but it should be simple and used sparingly. Putting anything but the simplest method calls or expressions in a view makes the overall application harder to test and maintain.

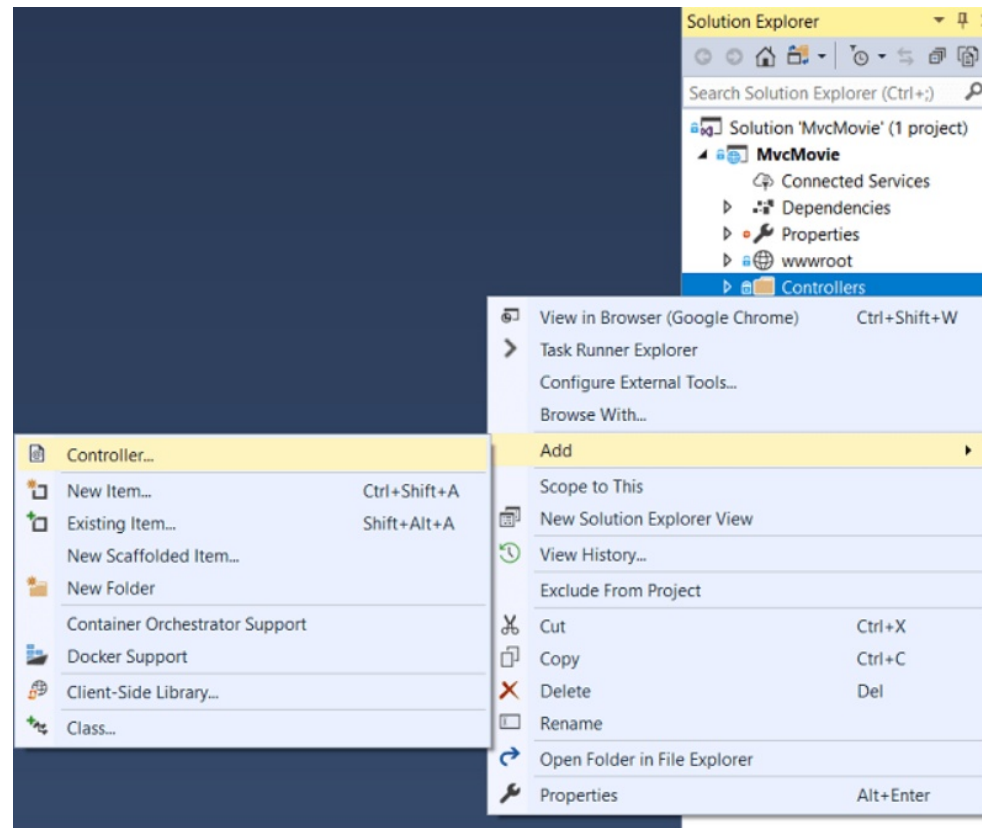
The ASP.NET Implementation of MVC

- In ASP.NET Core MVC, controllers are C# classes, usually derived from the `Microsoft.AspNetCore.Mvc.Controller` class.
- ASP.NET Core MVC uses a view engine, known as Razor, which is the component responsible for processing a view in order to generate a response for the browser.
- ASP.NET Core MVC doesn't apply any constraints on the implementation of your domain model. You can create a model using regular C# objects and implement persistence using any of the databases, object-relational mapping frameworks, or other data tools supported by .NET.

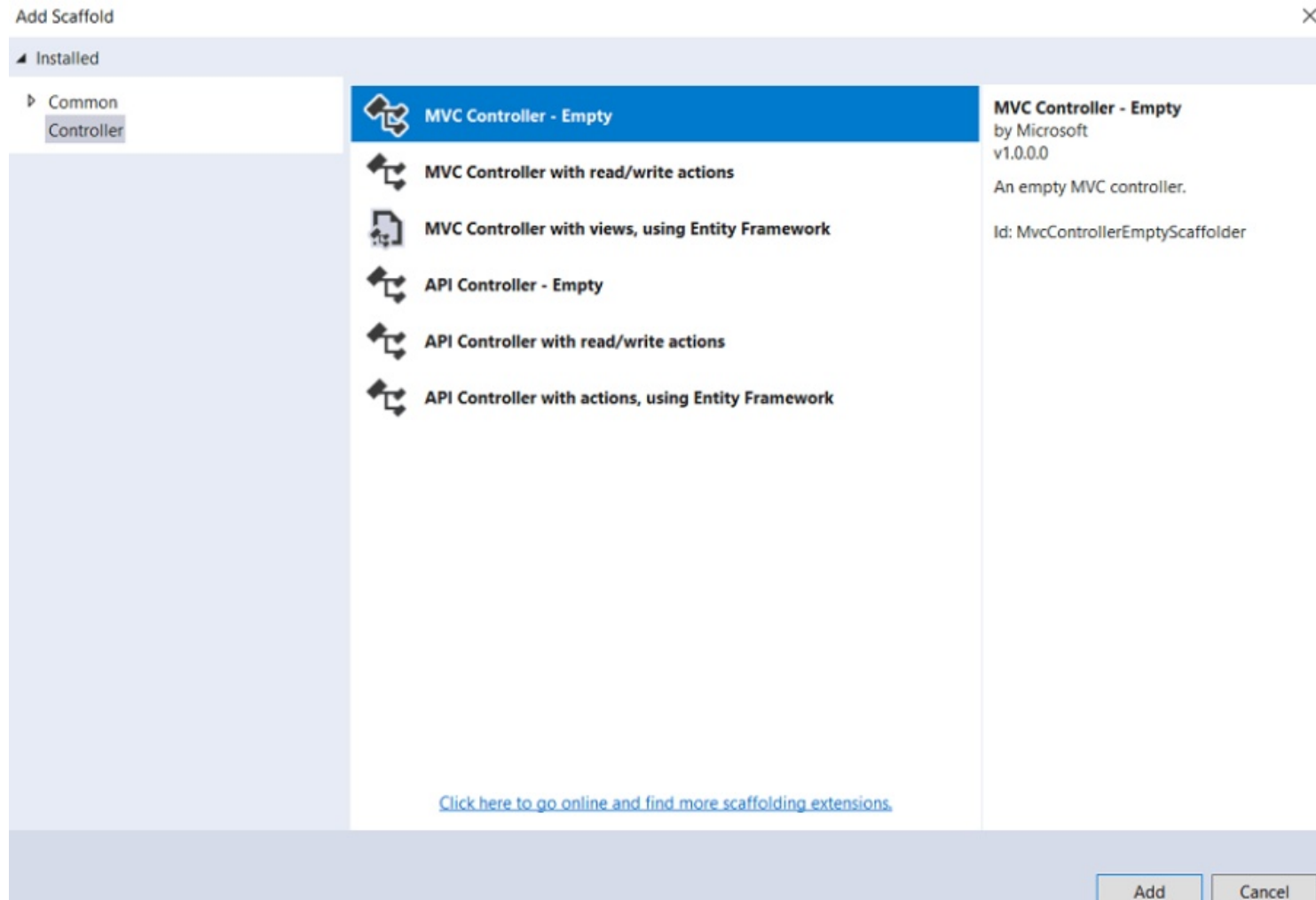


Practice II: Add a controller to an ASP.NET Core MVC app

- Following Practice I: In **Solution Explorer**, right-click **Controllers** > **Add** > **Controller**



- In the **Add Scaffold** dialog box, select **MVC Controller – Empty**
- In the **Add Empty MVC Controller** dialog, enter **HelloWorldController** and select **ADD**



- Replace the contents of *Controllers/HelloWorldController.cs* with the following:

```
using Microsoft.AspNetCore.Mvc;
using System.Text.Encodings.Web;

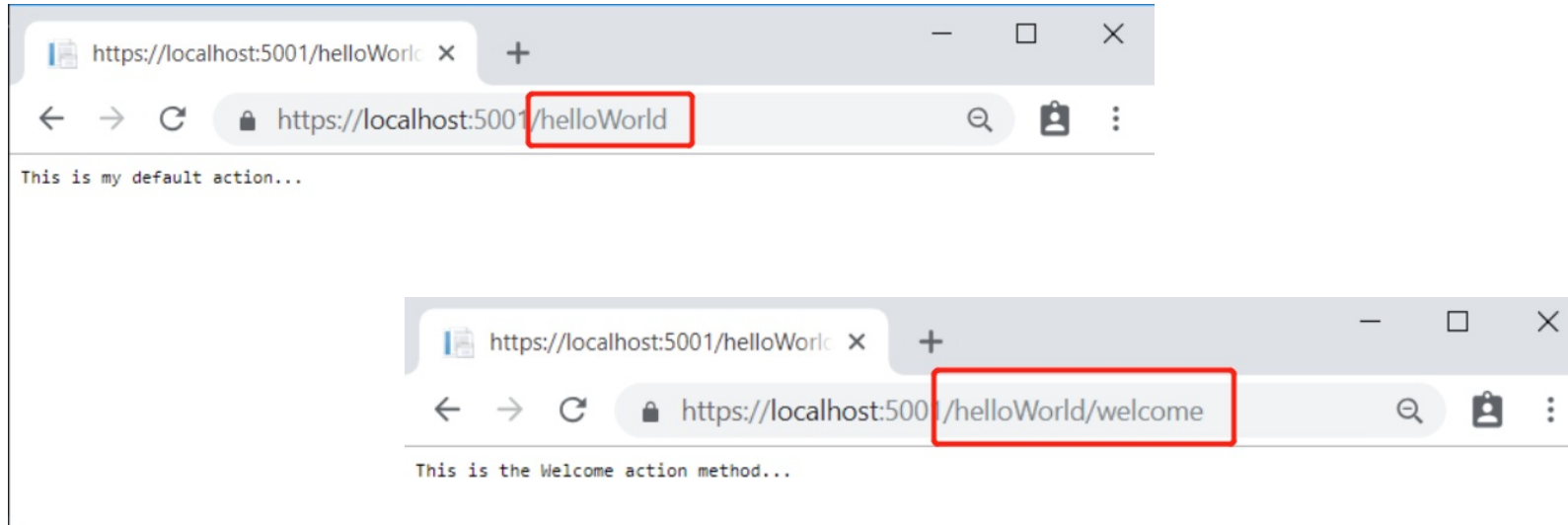
namespace MvcMovie.Controllers
{
    public class HelloWorldController : Controller
    {
        //
        // GET: /HelloWorld/

        public string Index()
        {
            return "This is my default action...";
        }

        //
        // GET: /HelloWorld/Welcome/

        public string Welcome()
        {
            return "This is the Welcome action method...";
        }
    }
}
```

Run the app in non-debug mode and append "HelloWorld" to the path in the address bar. The Index method returns a string.



The routing format is set in the `Configure` method in `Startup.cs` file.

```
C# Copy  
  
app.UseEndpoints(endpoints =>  
{  
    endpoints.MapControllerRoute(  
        name: "default",  
        pattern: "{controller=Home}/{action=Index}/{id?}");  
});
```

When you browse to the app and don't supply any URL segments, it defaults to the "Home" controller and the "Index" method specified in the template line highlighted above.

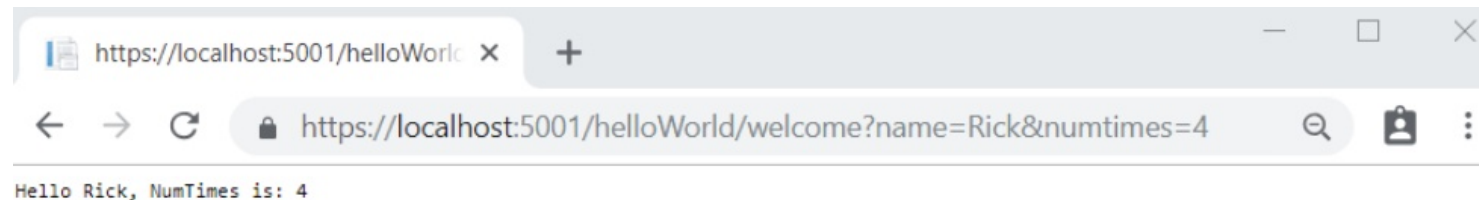
Modify the code to pass some parameter information from the URL to the controller. For example, */HelloWorld/Welcome?name=Rick&numtimes=4*. Change the Welcome method to include two parameters as shown in the following code.

```
// GET: /HelloWorld/Welcome/  
// Requires using System.Text.Encodings.Web;  
public string Welcome(string name, int numTimes = 1)  
{  
    return HtmlEncoder.Default.Encode($"Hello {name}, NumTimes is: {numTimes}");  
}
```

Run the app and browse to:

<https://localhost:{PORT}/HelloWorld/Welcome?name=Rick&numtimes=4>

** Replace {PORT} with your port number.*



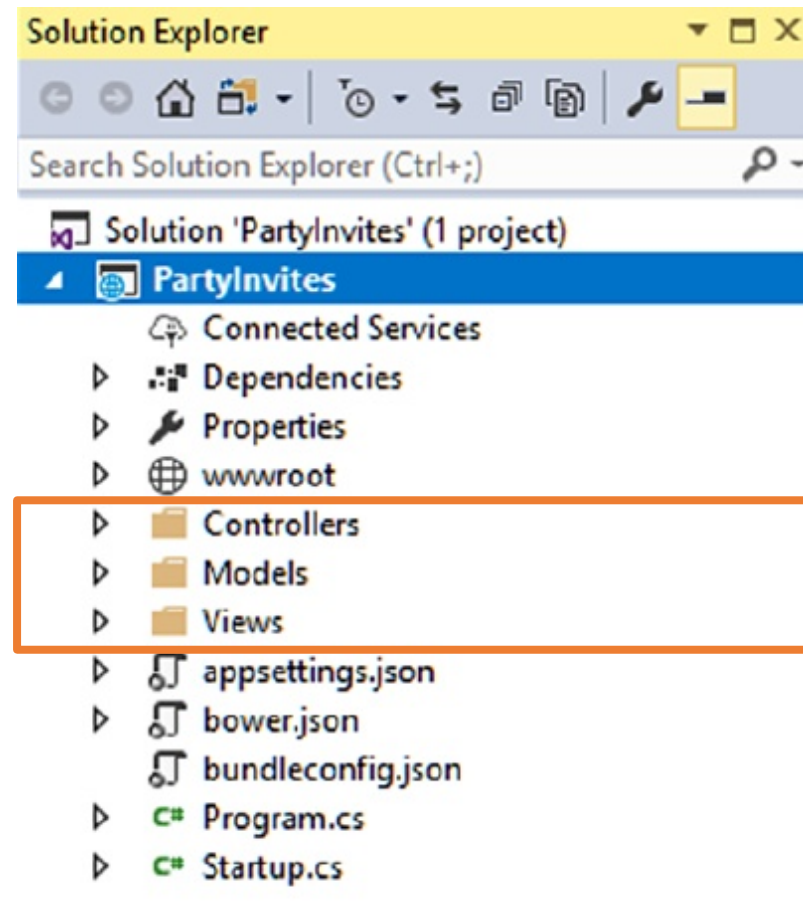
- Try to Replace 'Welcome' with the following code

```
public string Welcome(string name, int ID = 1)
{
    return HtmlEncoder.Default.Encode($"Hello {name}, ID: {ID}");
}
```

Run the app and enter the following URL: <https://localhost:{PORT}/HelloWorld/Welcome/3?name=Rick>

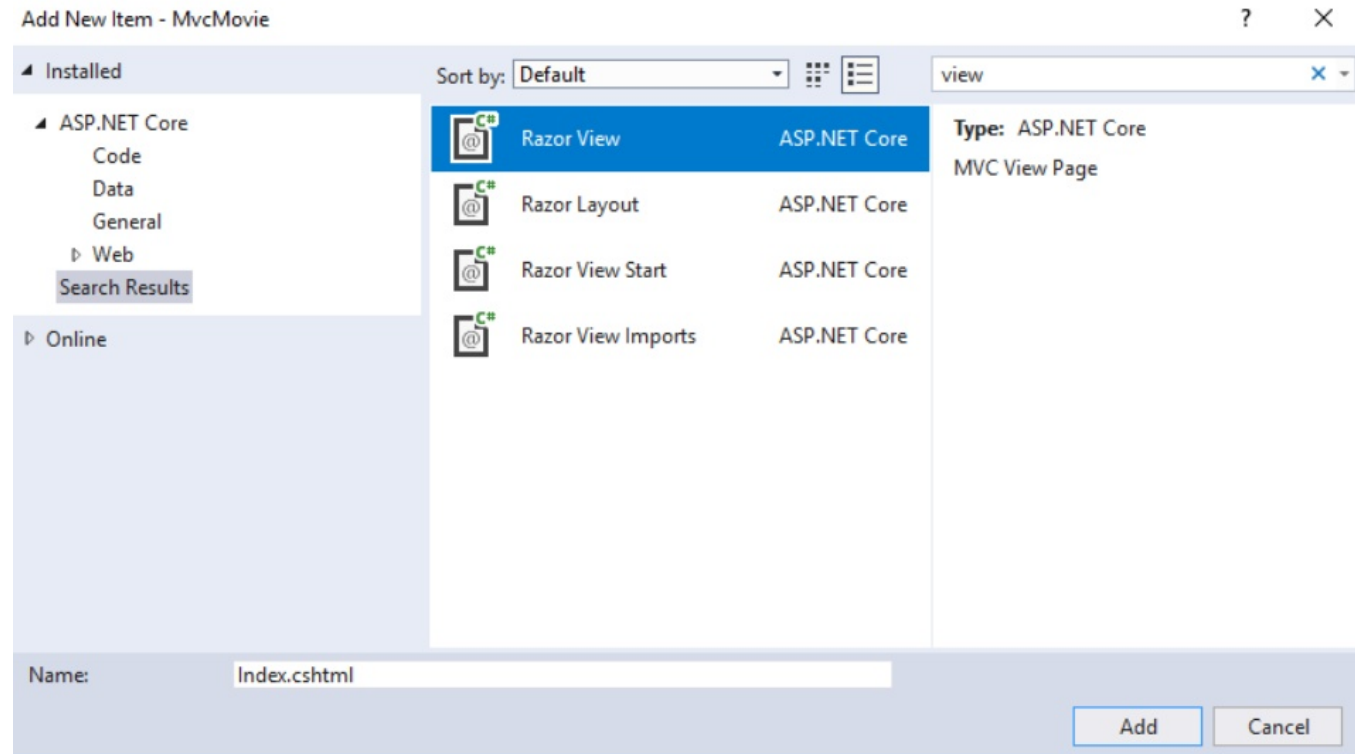
This time the third URL segment matched the route parameter id. The Welcome method contains a parameter id that matched the URL template in the MapControllerRoute method. The trailing ? (in id?) indicates the id parameter is optional.

Structure of a typical MVC Project



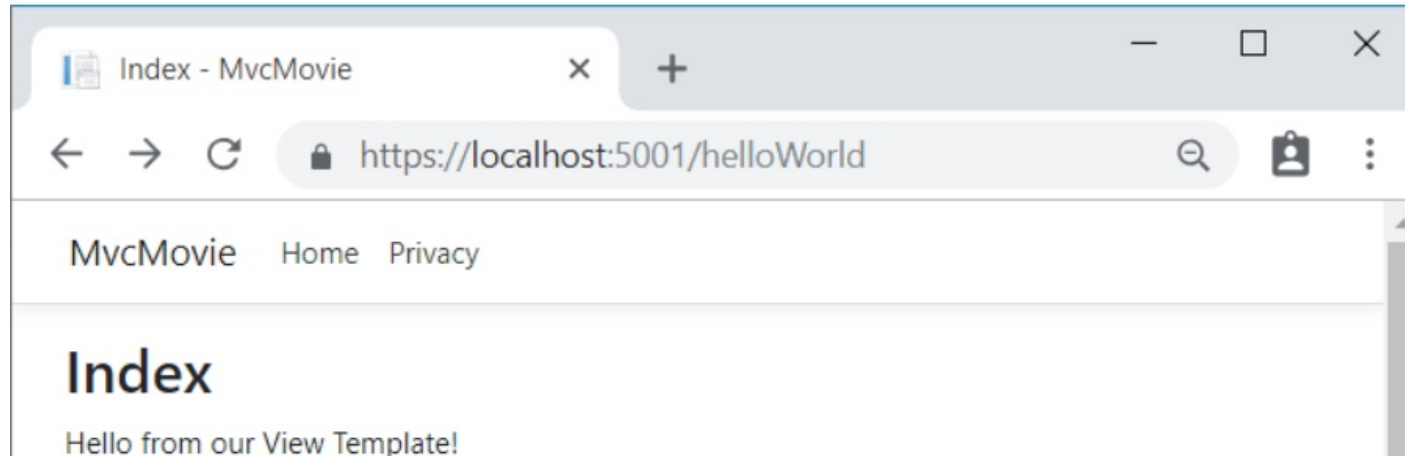
Rendering Web Pages

- The output from the previous example wasn't HTML—it was just the string Hello World. To produce an HTML response to a browser request, we need a *view*, which tells MVC how to generate a response to a request from a browser.
- Right click on the *Views* folder, and then **Add > New Folder** and name the folder *HelloWorld*.
- Right click on the *Views/HelloWorld* folder, and then **Add > New Item**.
- In the **Add New Item - MvcMovie** dialog
 - In the search box in the upper-right, enter *view*
 - Select **Razor View**
 - Keep the **Name** box value, *Index.cshtml*.
 - Select **Add**



Replace the contents of the *Views/HelloWorld/Index.cshtml* Razor view file with the following:

```
@{  
    ViewData["Title"] = "Index";  
}  
<h2>Index</h2>  
<p>Hello from our View Template!</p>
```



- Navigate to `https://localhost:{PORT}/HelloWorld`. The Index method in the HelloWorldController didn't do much; it ran the statement `return View();`, which specified that the method should use a view template file to render a response to the browser. Because a view template file name wasn't specified, MVC defaulted to using the default view file.
- The default view file has the same name as the method (Index), so in the `/Views/HelloWorld/Index.cshtml` is used. The image above shows the string "Hello from our View Template!" hard-coded in the view.

Change the title, footer, and menu link in the layout file

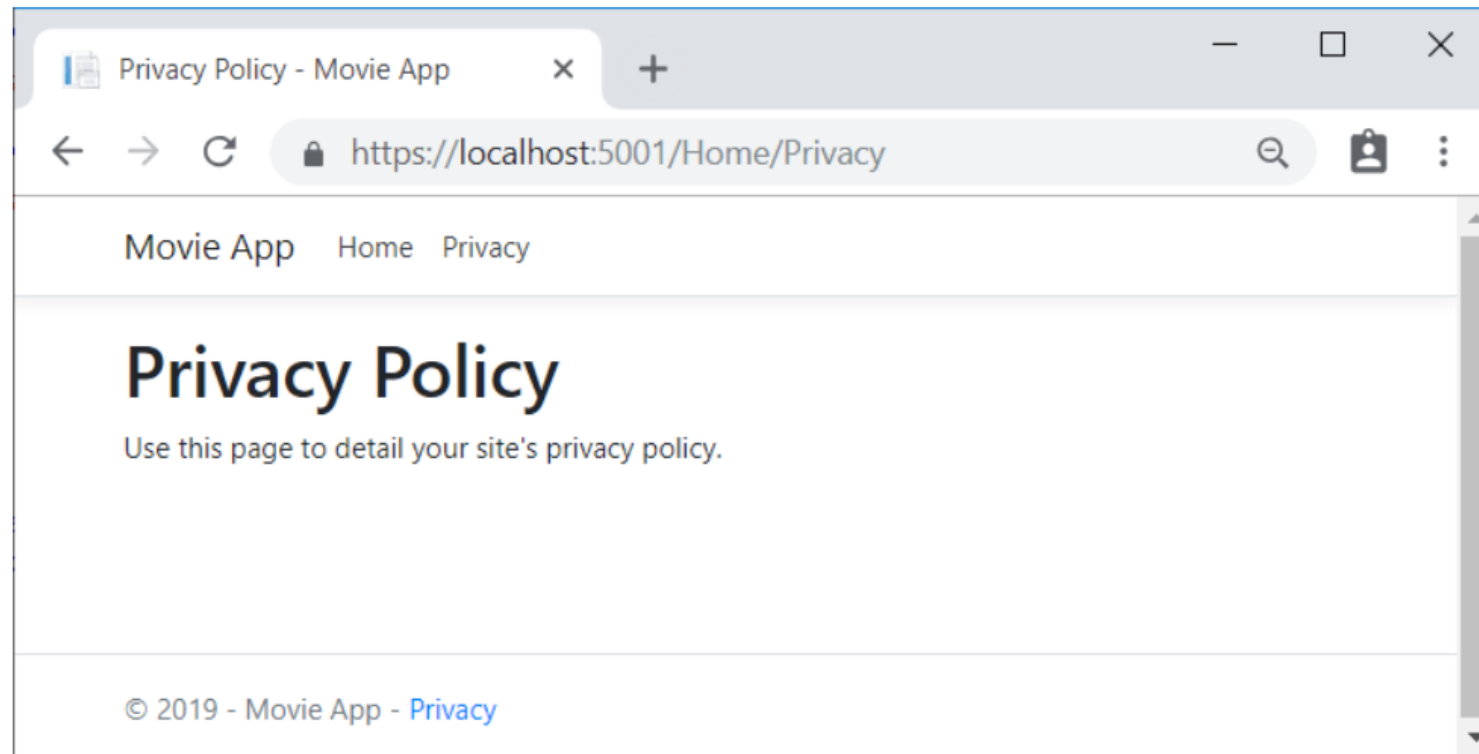
```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="utf-8" />
  <meta name="viewport" content="width=device-width, initial-
scale=1.0" />
  <title>@ViewData["Title"] - Movie App</title>
  <link rel="stylesheet"
href="~/lib/bootstrap/dist/css/bootstrap.css" />
  <link rel="stylesheet" href="~/css/site.css" />
</head>
<body>
  <header>
    <nav class="navbar navbar-expand-sm navbar-toggleable-sm
navbar-light bg-white border-bottom box-shadow mb-3">
      <div class="container">
        <a class="navbar-brand" asp-controller="Movies" asp-
action="Index">Movie App</a>
        <button class="navbar-toggler" type="button" data-
toggle="collapse" data-target=".navbar-collapse" aria-
controls="navbarSupportedContent"
          aria-expanded="false" aria-label="Toggle navigation">
          <span class="navbar-toggler-icon"></span>
        </button>
        <div class="navbar-collapse collapse d-sm-inline-flex flex-sm-
row-reverse">
          <ul class="navbar-nav flex-grow-1">
```

```
<li class="nav-item">
  <a class="nav-link text-dark" asp-area="" asp-
controller="Home" asp-action="Index">Home</a>
</li>
<li class="nav-item">
  <a class="nav-link text-dark" asp-area="" asp-
controller="Home" asp-action="Privacy">Privacy</a>
</li>
</ul>
</div>
</nav>
</header>
<div class="container">
  <main role="main" class="pb-3">
    @RenderBody()
  </main>
</div>

<footer class="border-top footer text-muted">
  <div class="container">
    &copy; 2019 - Movie App - <a asp-area="" asp-
controller="Home" asp-action="Privacy">Privacy</a>
  </div>
</footer>
<script src="~/lib/jquery/dist/jquery.js"></script>
<script src="~/lib/bootstrap/dist/js/bootstrap.bundle.js"></script>
<script src="~/js/site.js" asp-append-version="true"></script>
@RenderSection("Scripts", required: false)
</body>
</html>
```

Replace the content of the *Views/Shared/_Layout.cshtml* file with the following markup. The changes are highlighted:

Save your changes and select the **Privacy** link. Notice how the title on the browser tab displays **Privacy Policy - Movie App** instead of **Privacy Policy - Mvc Movie**:



Practice: Getting Started with EF Core

<https://learn.microsoft.com/en-us/aspnet/core/data/ef-mvc/intro?view=aspnetcore-9.0>

Create a new ASP.NET Core web application

.NET Core

ASP.NET Core 5.0

**ASP.NET Core Empty**
An empty project template for creating an ASP.NET Core application. This template does not have any content in it.

**ASP.NET Core Web API**
A project template for creating an ASP.NET Core application with an example Controller for a RESTful HTTP service. This template can also be used for ASP.NET Core MVC Views and Controllers.

**ASP.NET Core Web App**
A project template for creating an ASP.NET Core application with example ASP.NET Razor Pages content.

**ASP.NET Core Web App (Model-View-Controller)**
A project template for creating an ASP.NET Core application with example ASP.NET Core MVC Views and Controllers. This template can also be used for RESTful HTTP services.

**ASP.NET Core with Angular**
A project template for creating an ASP.NET Core application with Angular

**ASP.NET Core with React.js**

[Get additional project templates](#)

Authentication
No Authentication
[Change](#)

Advanced

☒ Configure for HTTPS

☐ Enable Docker Support
(Requires [Docker Desktop](#))

Windows

☐ Enable Razor runtime compilation

Author: Microsoft
Source: Templates 5.0.0

Back

Create

34

Software Quality Assurance



Pavithra Subashini

Senior lecturer

Department of Software
Engineering

Faculty of Computing



Why you must study Quality Assurance?



Ground rules

- Interactivity is must
- Proactivity is expected
- No mobile phones during lecture
- Allow access to digital devices during activities
- Activity based learning



Module set up

- Two-hour lectures per week – Activity based learning
- One-hour tutorial
- At the end of the module you will be able to ensure the quality of the software project.
- Encourage self study
 - Research on SQA field
 - Industrial changes/ adoption to the changes



Reference book

<https://www.istqb.org/downloads.html>

Refer latest books from NSBM Library





What is Software Quality Assurance?

“Set of activities to ensure the quality in software engineering processes.”

- It ultimately result in quality software products.
- The activities establish and evaluate the processes that produce products.
- It involves process-focused action.



What is testing?

“Method to check whether the actual software product matches expected requirements and to ensure that software product is Defect free.”

- It involves execution of software/system components by manual or automated tools
- The purpose of software testing is to identify errors, gaps or missing requirements in contrast to actual requirements



Typical Objectives of Testing

- To prevent defects by evaluate work products such as requirements, user stories, design, and code
- To verify whether all specified requirements have been fulfilled
- To check whether the test object is complete and validate if it works as the users and other stakeholders expect
- To build confidence in the level of quality of the test object



Typical Objectives of Testing

- To find defects and failures thus reduce the level of risk of inadequate software quality
- To provide sufficient information to stakeholders to allow them to make informed decisions, especially regarding the level of quality of the test object
- To comply with contractual, legal, or regulatory requirements or standards, and/or to verify the test object's compliance with such requirements or standards



What is a “bug”?

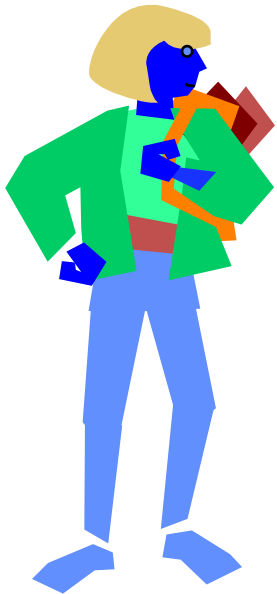
- **Error:** a human action that produces an incorrect result
- **Fault:** a manifestation of an error in software
 - also known as a defect or bug
 - if executed, a fault may cause a failure
- **Failure:** deviation of the software from its expected delivery or service
 - (found defect)

Failure is an event; fault is a state of the software, caused by an error

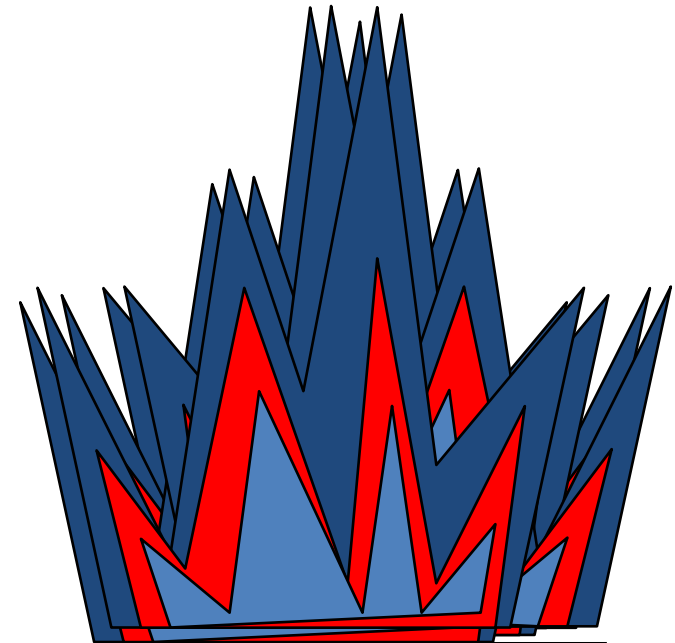
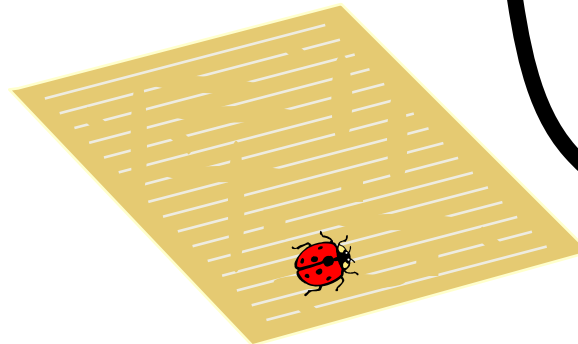


Error - Fault - Failure

A person makes
an error ...



... that creates a
fault in the
software ...



... that can cause
a failure
in operation



In class activity- Individual

Identify global top 10 software defetscs due to lack of testing.

Submit answers to nlern.



Reliability versus faults

- Reliability: the probability that software will not cause the failure of the system for a specified time under specified conditions
 - Can a system be fault-free? (zero faults, right first time) ✗
 - Can a software system be reliable but still have faults? ✓
 - Is a “fault-free” software application always reliable?





Why do faults occur in software?

- software is written by human beings
 - who know something, but not everything
 - who have skills, but aren't perfect
 - who do make mistakes (errors)
- under increasing pressure to deliver to strict deadlines
 - no time to check but assumptions may be wrong
 - systems may be incomplete
- if you have ever written software ...



Do search ,you will find lots of
interesting facts

What do software faults cost?

- huge sums
 - Ariane 5 (\$7billion)
 - Mariner space probe to Venus (\$250m)
 - American Airlines (\$50m)
- very little or nothing at all
 - minor inconvenience
 - no visible or physical detrimental impact
- software is not “linear”:
 - small input may have very large effect



Safety-critical systems

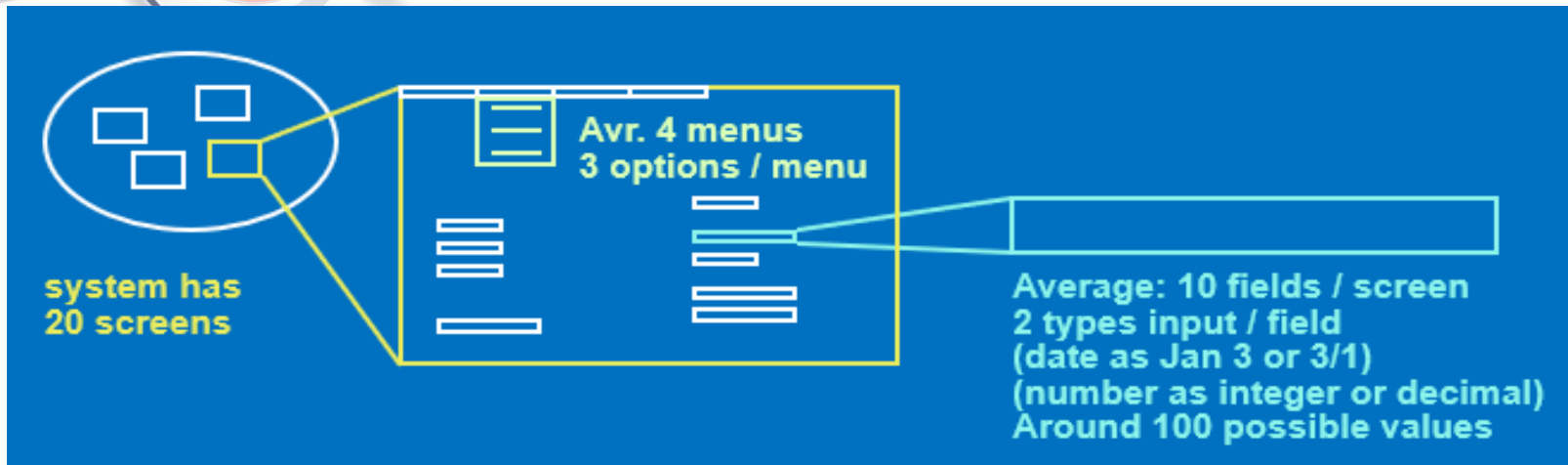
- software faults can cause death or injury
 - radiation treatment kills patients (Therac-25)
 - train driver killed
 - aircraft crashes (Airbus & Korean Airlines)
 - bank system overdraft letters cause suicide



So why is testing necessary?

- because software is likely to have faults ✓
- to learn about the reliability of the software ✓
- to fill the time between delivery of the software and the release date ✗
- to prove that the software has no faults ✗
- because testing is included in the project plan ✗
- because failures can be very expensive ✓
- to avoid being sued by customers ✓
- to stay in business ✓

Why not just "test everything"?



Total for 'exhaustive' testing:

$$20 \times 4 \times 3 \times 10 \times 2 \times 100 = 480,000 \text{ tests}$$

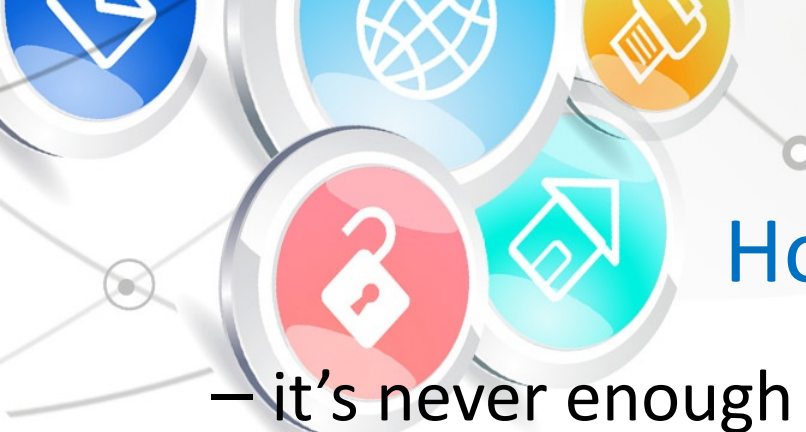
If 1 second per test, 8000 mins, 133 hrs, 17.7 days
(not counting finger trouble, faults or retest)

10 secs = 34 wks, 1 min = 4 yrs, 10 min = 40 yrs



Exhaustive testing?

- What is exhaustive testing?
 - when all the testers are exhausted
 - when all the planned tests have been executed
 - exercising all combinations of inputs and preconditions
- How much time will exhaustive testing take?
 - infinite time
 - not much time
 - impractical amount of time



How much testing is enough?

- it's never enough
- when you have done what you planned
- when your customer/user is happy
- when you have proved that the system works correctly
- when you are confident that the system works correctly
- it depends on the risks for your system



How much testing?

- It depends on **RISK**
 - risk of missing important faults
 - risk of incurring failure costs
 - risk of releasing untested or under-tested software
 - risk of losing credibility and market share
 - risk of missing a market window
 - risk of over-testing, ineffective testing



So little time, so much to test ..

- test time will always be limited
 - use **RISK** to determine:
 - what to test first
 - what to test most
 - how thoroughly to test each item
- } i.e. where to place emphasis
- what not to test (this time)
-
- allocate the time available for testing by prioritising testing ...

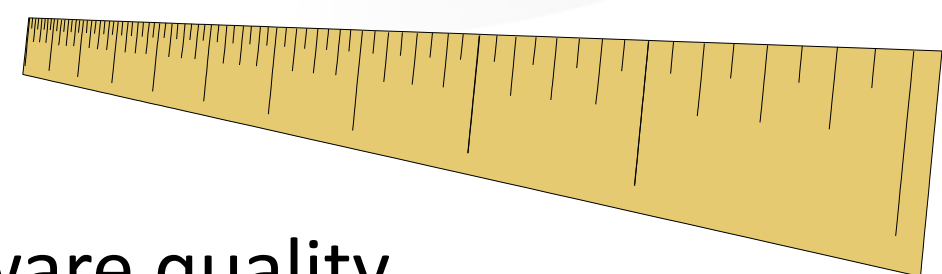


Most important principle

**Prioritise tests
so that,
whenever you stop testing,
you have done the best testing
in the time available.**



Testing and quality



- testing measures software quality
- testing can find faults; when they are removed, software quality (and possibly reliability) is improved
- what does testing test?
 - system function, correctness of operation
 - non-functional qualities: reliability, usability, maintainability, reusability, testability, etc.



Other factors that influence testing

- contractual requirements
- legal requirements
- industry-specific requirements
 - e.g. pharmaceutical industry (FDA), compiler standard tests, safety-critical or safety-related such as railroad switching, air traffic control

**It is difficult to determine
how much testing is enough
but it is not impossible**



Tute

- Briefly explain the important of testing.
- How can you determine how much testing is enough?
- Briefly explain why exhaustive testing is not possible.





Software Quality Assurance



Pavithra Subashini

Senior lecturer

Faculty of Computing



How the customer explained it



How the project leader understood it



How the analyst designed it



How the programmer wrote it



What the beta testers received



How the business consultant described it



How the project was documented



What operations installed



How the customer was billed



How it was supported



What marketing advertised



What the customer really needed



Test Case Development

- ID
- Title
- Description
- Pre-condition
- Test Steps
- Test Data
- Expected Results
- Actual Results
- Priority
- Release
- Status



Test Environment Setup

- Hardware Configurations
- Operating Systems
- Integrations
- Test Data
- Access/Permissions

Test Execution

- Prioritize
- Smoke Test
- Functional Test
- Non Functional Test
- Test Automation



What is a defect?

- A defect is any form of error or obstacle that comes between the software testers or users and seamless execution of the application.
- This error tries to hinder the execution by mismatching the expected behavior of the software with the actual one.



Compulsory attributes of Software Defects

- ID
- Title
- Description
- Steps to recreate
- Test Data
- Test Environment
- Expected results
- Actual Results
- Screen Shots
- Severity
- Priority
- Status

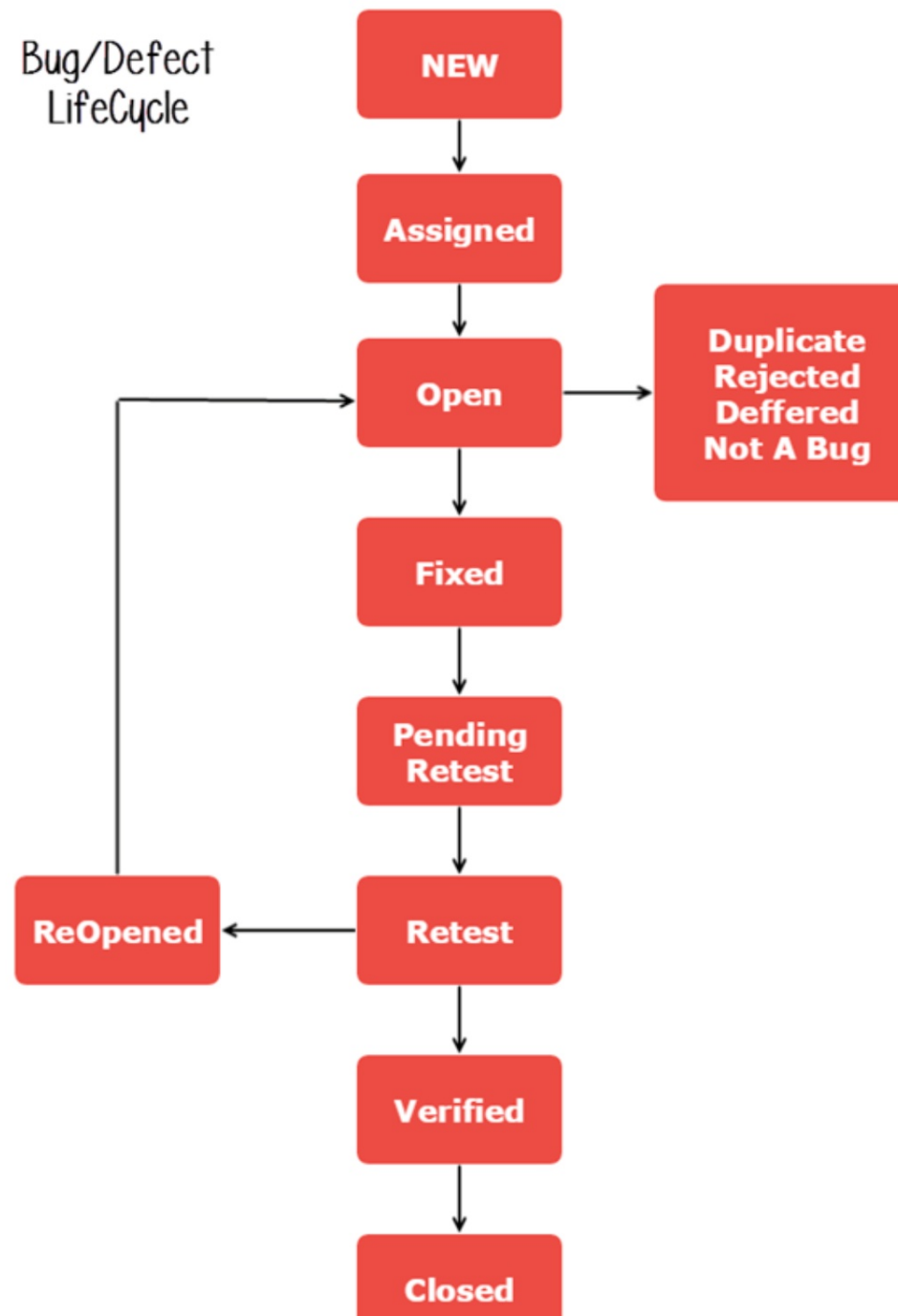


Defect Status

- **Defect Status** or Bug Status in defect life cycle is:
- The present state from which the defect or a bug is currently undergoing.
- The goal of defect status is to precisely convey the current state or progress of a defect or bug in order to better track and understand the actual progress of the defect life cycle.



Bug/Defect LifeCycle





Make a checklist and ensure whether you have passed all the points before reporting a bug.

- i. Have I reproduced the bug 2-3 times.
- ii. Have I verified in the Defect Tracking Tool (using keywords) whether someone else already posted the same issue.
- iii. Have I verified the similar issue in the related modules.
- iv. Have I written the detailed steps to reproduce the bug.
- v. Have I written proper defect summary.
- vi. Have I attached relevant screenshots.
- vii. Have I missed any necessary fields in the bug report?



Sample bug report

- **Defect ID:**

Add a Defect ID using a naming convention followed by your team. The Defect ID will be generated automatically in case of defect management tool.

- **Reporter Name:**

Name of the one who found the defect (Usually tester's name but sometimes it might be Developer, Business Analyst, Subject Matter Expert (SME), Customer)

- **Defect Reported Date:** Date of the defect reported
- **Who Detected:** Add the designation of the one who found the defect. E.g. QA, Developer, Business Analyst, SME, Customer
- **How Detected:** Testing, Review, Walkthrough
- **Project Name:** Add name of the project. (If it's a product, add product name)
- **Release/Build Version:** Add the build version details here



Sample bug report

- **Defect/Enhancement:** Add whether it is defect or improvement
- **Environment:** Add Operation Systems details, Browser Details and any other related to the test environment.
 - (E.g. Windows 8/Chrome 48.0.2564.103)
- **Priority:** Add the priority of the bug
 - (E.g. High/Medium/Low)
- **Severity:** Add the severity of the bug
 - (E.g. Critical/High/Medium/Low)
- **Status:** Add the status of the bug. If you just found and posting it then it will be New. The status of the bug will change.
 - (E.g. New/ Assigned/ Open/ Fixed/ Test/ Verified/ Closed/ Reopen/ Duplicate/ Deferred/ Rejected/ cannot be fixed/ Not Reproducible/ Need more information)
- **Description:** Add a detailed description.
- **Steps to reproduce:** Mention steps in detail. So that even the one who has no idea about the application also could reproduce the bug.



Sample bug report

- **URL:** Add the URL of the application (If available)
- **Expected Result:** Mention the expected result here which is available in your test case document.
- **Actual Result:** Mention the actual result here which is available in your test case document.
- **Defect Close Date:** Add the defect close date only once you ensure that the defect is not reproducible





Software Quality Assurance



Pavithra Subashini

Senior lecturer

Faculty of Computing



Psychology of testing



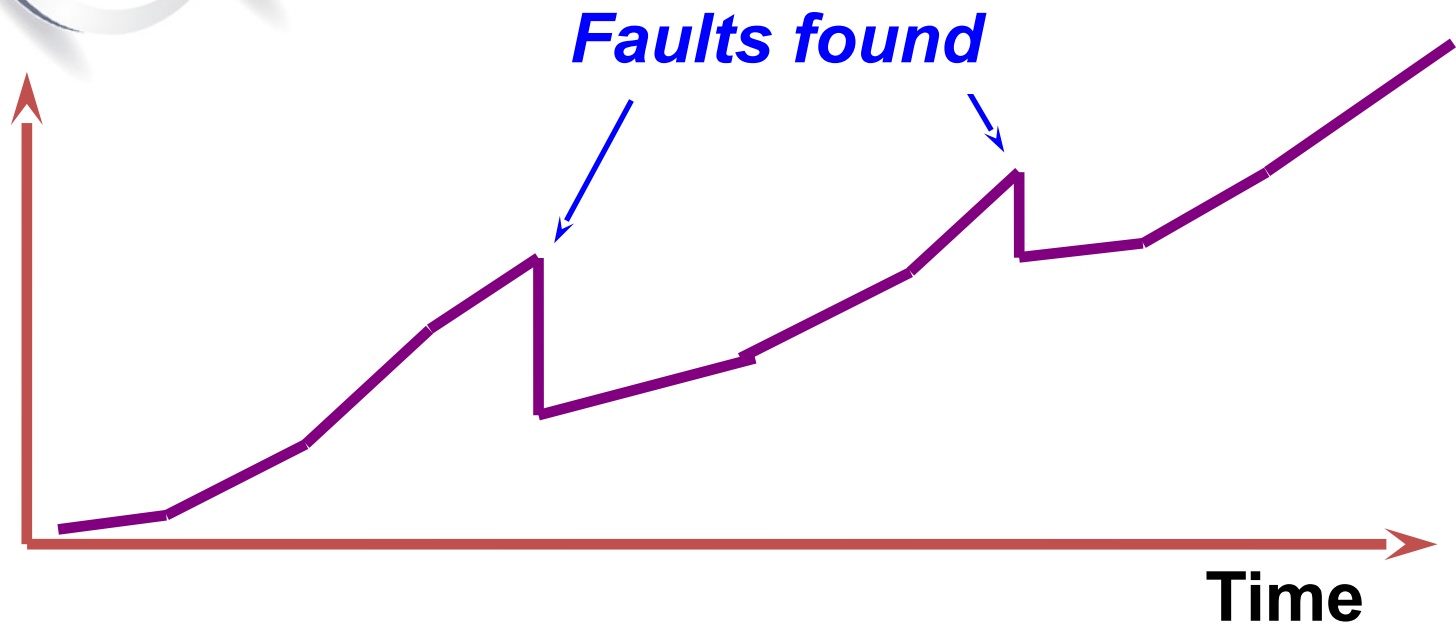
Why test?

- build confidence ✓
- prove that the software is correct ✗
- demonstrate conformance to requirements ✓
- find faults ✓
- reduce costs ✓
- show system meets user needs ✓
- assess the software quality ✓



Confidence

Confidence



No faults found = confidence?



A traditional testing approach

■ Show that the system:

- does what it should
- doesn't do what it shouldn't

Goal: show
working

Success: system works

Fastest achievement: easy test cases

—————➔ **Result: faults left in**



A better testing approach

■ Show that the system:

- does what it shouldn't
- doesn't do what it should

Goal: find faults

Success: system fails

Fastest achievement: difficult test cases

—————➔ **Result: fewer faults left in**



The testing paradox

Purpose of testing: to find faults
Finding faults destroys confidence
∴ Purpose of testing: destroy confidence

Purpose of testing: build confidence

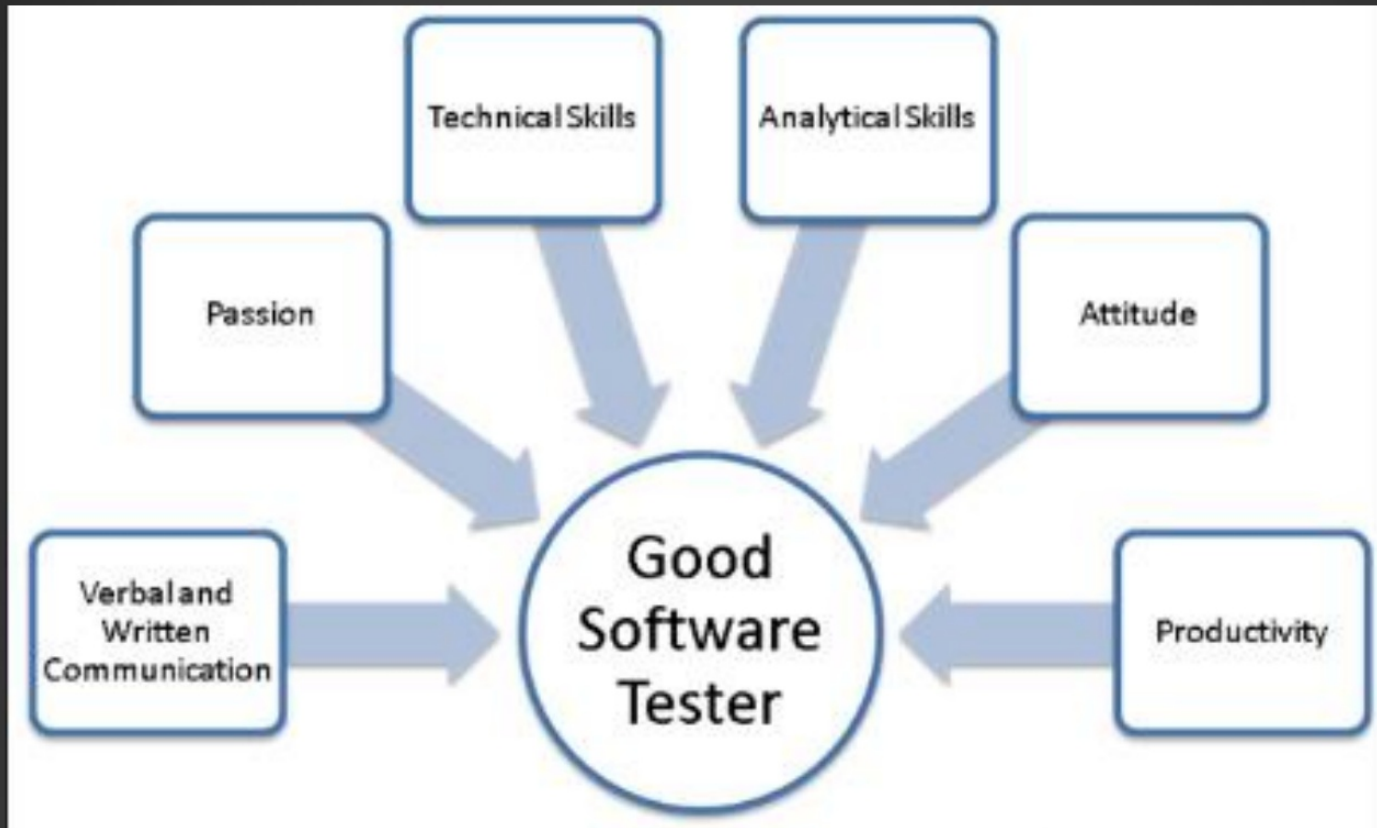
**The best way to build confidence
is to try to destroy it**



Who wants to be a tester?

- A critical process
- Bring bad news (“your baby is ugly”)
- Under worst time pressure (at the end)
- Need to take a different view, a different mindset (“What if it isn’t?”, “What could go wrong?”)
- How should fault information be communicated (to authors and managers?)

SKILLS TO BECOME A GOOD TESTER





SOFT SKILLS TO BECOME A GOOD TESTER



QUALITIES OF A GOOD SOFTWARE TESTER

An ideal tester should have:

- Heightened degree of responsibility
- Strong communication skills
- Knack to express his ideas precisely and clearly
- Orderliness
- Patience, assiduity, attention to details, observationЖ
- Flexible thinking, high education potential
- Intelligent abstract thinking, ability for analytical work
- Experimenter's psychology
- Software engineering skills
- Detective skills



Tester's have the right to:

- Accurate information about progress and changes
- Insight from developers about areas of the software
- Delivered code tested to an agreed standard
- Be regarded as a professional (no abuse!)
- Find faults!
- Challenge specifications and test plans
- Have reported faults taken seriously (non-reproducible)
- Make predictions about future fault levels
- Improve your own testing process



Testers have responsibility to:

- Follow the test plans, scripts etc. as documented
- Report faults objectively and factually (no abuse!)
- Check tests are correct before reporting s/w faults
- Remember it is the software, not the programmer, that you are testing
- Assess risk objectively
- Prioritise what you report
- Communicate the truth



Independence

■ Test your own work?

- find 30% - 50% of your own faults
- same assumptions and thought processes
- see what you meant or want to see, not what is there
- emotional attachment
 - don't want to find faults
 - actively want NOT to find faults



Levels of independence

- None: tests designed by the person who wrote the software
- Tests designed by a different person
- Tests designed by someone from a different department or team (e.g. test team)
- Tests designed by someone from a different organisation (e.g. agency)
- Tests generated by a tool (low quality tests?)



Software Development Tools & Practices

PUSL2020

Level 2

“Introduction to Web Technologies (Part II)”

Dr. Rasika Ranaweera (ranaweera.r@nsbm.ac.lk | +94 11 544 6126)

Faculty of Faculty of Postgraduate Studies & Professional Advancement | NSBM Green University

Introduction to State Management

- Remember that ASP.NET (and the Web) is stateless
 - The Web server does not keep track of past client requests
- Different technologies handle the issue of state management differently
 - ASP.NET is somewhat unique in this regard

Form posting: ASP.net vs PHP

- Write a script to design following form. Then write necessary codes/scripts to display user entered values in new web page.



A web form with two input fields and a submit button. The first field is labeled "Name:" and the second field is labeled "E-mail:". Below the fields is a "Submit" button.

Name:

E-mail:

Form posting: ASP.net vs PHP (Cont.)

•PHP

```
<!DOCTYPE HTML>
<html>
<body>

<form action="welcome_get.php" method="get">
Name: <input type="text" name="name"><br>
E-mail: <input type="text" name="email"><br>
<input type="submit">
</form>

</body>
</html>
```

```
<html>
<body>

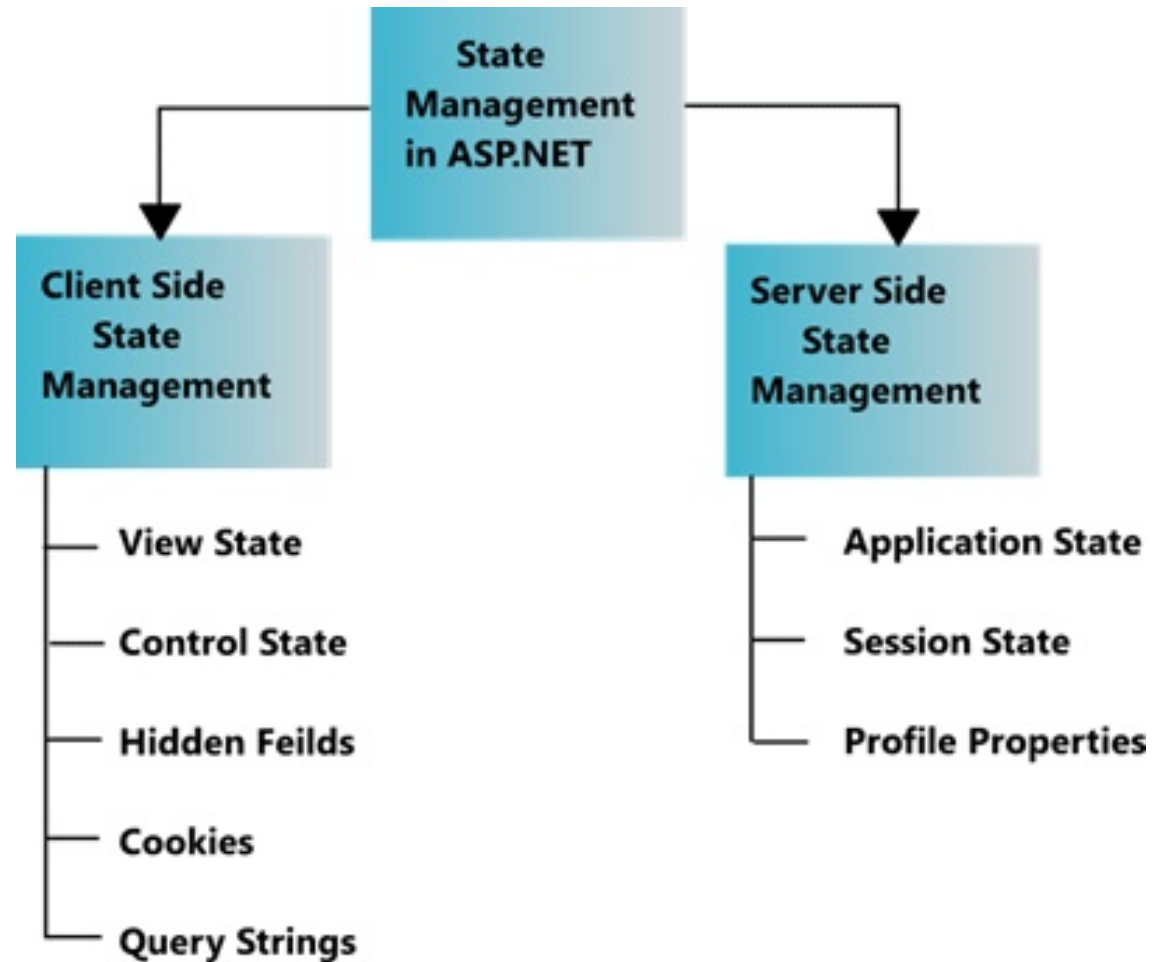
Welcome <?php echo $_GET["name"]; ?><br>
Your email address is: <?php echo $_GET["email"]; ?>

</body>
</html>
```

Types of State Management

- ASP.NET offers two categories of state management
 - Pure client-side state management
 - Server-side state management

State Management Techniques



Cookies

- Cookie is a small text file which is created by the client's browser and also stored on the client hard disk by the browser.
- It does not use server memory.
- Generally a cookie is used to identify users.
- Whenever a user makes a request for a page the first time, the server creates a cookie and sends it to the client along with the requested page and the client browser receives that cookie and stores it on the client machine either permanently or temporarily (persistent or non persistence).
- The next time the user makes a request for the same site, either the same or another page, the browser checks the existence of the cookie for that site in the folder. If the cookie exists it sends a request with the same cookie, else that request is treated as a new request.

Cookies (Cont.)

- **Persistence Cookie:** Cookies which you can set an expiry date time are called persistence cookies. Persistence cookies are permanently stored till the time you set.
- **Non-Persistence Cookie:** Non persistence cookies are not permanently stored on the user client hard disk folder. It maintains user information as long as the user accesses the same browser. When user closes the browser the cookie will be discarded. Non Persistence cookies are useful for public computers.

Setting cookies



```
Response.Cookies["userName"].Value =  
"patrick";  
Response.Cookies["userName"].Expires =  
DateTime.Now.AddDays(1);  
//or  
HttpCookie aCookie = new  
HttpCookie("lastVisit");  
aCookie.Value = DateTime.Now.ToString();  
aCookie.Expires = DateTime.Now.AddDays(1);  
Response.C
```

Set Cookies

```
if(Request.Cookies["userName"] != null)  
    Label1.Text =  
Server.HtmlEncode(Request.Cookies["userName"].Value)  
;  
  
if(Request.Cookies["userName"] != null)  
{  
    HttpCookie aCookie = Request.Cookies["userName"];  
    Label1.Text = Server.HtmlEncode(aCookie.Value);  
}
```

Read Cookies

Setting cookies (Cont.)



Set Cookies

```
<?php
$cookie_name = "user";
$cookie_value = "John Doe";
setcookie($cookie_name, $cookie_value, time() + (86400 *
30), "/");
// 86400 = 1 day
?>
<html>
<body>

<?php
echo "Cookie is set.";
?>
</body>
</html>
```

Read Cookies

```
<html>
<body>
<?php
$cookie_name = "user";
if(!isset($_COOKIE[$cookie_name])) {
    echo "Cookie named '" . $cookie_name . "' does
not exist!";
} else {
    echo "Cookie is named: " . $cookie_name .
"<br>Value is: " . $_COOKIE[$cookie_name];
}
?>
</body>
</html>
```

Cookies usage

- ◆ Cookies do not become visible until the next page load.
- ◆ In PHP Cookies must be deleted using the same parameters with which they were set.
- ◆ PHP Cookie names can be set as array names, accessed through arrays but are stored as separate cookies on users system.

Query String

- Query strings are an easy way to pass information from one page to another or even back to the same from one page to another or even back to the same page.
 - `Response.Redirect("newpage.aspx?recordID=10");`
 - `Response.Redirect("newpage.aspx?recordID=10&mode=full");`
 - `string ID = Request.QueryString["recordID"];`

Some browsers and client devices impose a 2083 character limit on the length of the URL.

SESSIONS

- ◆ Cookies don't work with shopping carts!
- ◆ Items in session state linked to particular user
 - ◆ used to store user preferences across multiple page requests
 - ◆ Can store objects - not just text
- ◆ Item is stored on server - not client
 - ◆ BUT a cookie is sent to client to identify the session id
- ◆ Be wary of it's use
 - ◆ How many people using your application?
 - ◆ Separate copy of each item for each user?

ASP.NET sessions

- ◆ `Session["message"] = "hello"`
- ◆ When item stored, ASP.NET _SessionID cookie sent to browser to link the session to the user.
- ◆ Session state does not work if cookies are disabled.
 - ◆ No error.
 - ◆ Just not there.
- ◆ ASP.NET framework assumes if no interaction in 20 minutes sessions state is discarded.

PHP Sessions

- ◆ Must invoke the `session_start()` function
 - ◆ checks if a session created for user - if not creates one.
 - ◆ Must be called before any output to the page
- ◆ `$_SESSION["message"] = "hello"`

```
<? php
    session_start()
    if(!$_SESSION["message"])
        //means we haven't got one
        $_SESSION["message"] = "hello";
    else
        msg = $_SESSION["message"];
?>
```



TEST STATUS REPORTING

Contents of Test Status Reporting

- Introduction
- Target audience for Status Reporting
- Why Status Reporting?
- Status Reporting Guidelines
- Metrics Based Status Reporting - Standard Chartered Bank example

Introduction

Test Status Reporting is a formalized reporting on the status of the project from a testing point of view. This report contains quantitative information about the project.

Test Status Reporting is the component that informs key project stakeholders of the critical aspects of the project's status. Periodic status reporting prevents surprises to project sponsors and stakeholders.

Target audience for Status Reporting

Test Status Reports will be used by the Project Manager to understand and communicate the status of the project, which then becomes very useful in terms of tracking progress and the quality of the application/product.

Target audience :

- Customer
- Test Team
- Management
- Any other key stakeholders

Why Status Reporting?

The purpose of a Test Status Report is to provide an ongoing status of the project, which becomes very useful in terms of tracking progress, evaluation and review.

Each status report from the team to the intended audience acts as a source of information which enables them to make timely decisions.

Some of the typical questions management asks are

- Is the test cycle execution happening as planned?
- What percentage of test execution is completed till date?
- No. of Open Defects as on date
- How many more test cycles would be needed before we can release?
- When can we make a release?

So it's a challenge to provide the right kind of data that will help management determine the quality of the application under test and progress of the project.

Status Reporting Guidelines

A good status report should effectively communicate the relevant information to the target audience. The structure of the document should be tailored to fit the stakeholder needs.

A brief description/contents of a typical test status report is presented below.

- **Project Name** - The project name that you are reporting status for.
- **Duration** - <The reporting period> From and To Date
- **Report by** - <Owner / Author of the Report> Test Lead/Test Manager prepares this document
- **Report to** - Target Audience.
- **Planned Activities/Tasks** - List of planned activities/tasks and owners for the same for the reporting period
- **Activities/Tasks Accomplished in the Reporting Period** - Activities performed after last reporting date.
- **Project Milestones Reached** - Important milestones from a testing point of view with current status.

Status Reporting Guidelines

- **Activities/Tasks not Accomplished/missed** - Activities not performed as per plan
- **Planned Activities for the next Reporting Period** - Activities/Tasks for the current reporting period
- **Test Project Execution Details** : # Pass , # Fail , # Blocked , # Not Executed
- **Test Summary** : Test Coverage details
- **Defect Status** : Details of # defects / severity wise
- Status of Defect Re-testing
- **Issues** - The problems faced during test lifecycle which affects the planned test activity listed according to their criticality. (For example: environment problems, blocking factors that prevent the progress of testing, unscheduled releases, unavailability of planned resources etc)
- **Unresolved Issues** - Issues from previous reporting period that has not been addressed
- **Risks** - Providing the stake holder note of important risks that might affect the testing schedule
- **Environment Downtime Tracking**: Report # hours lost due to server down or any other environment issues faced during test execution

Software Development Tools & Practices

PUSL2020

Level 2

“Introduction to Web Technologies”

Dr. Rasika Ranaweera (ranaweera.r@nsbm.ac.lk | +94 11 544 6126)

Faculty of Faculty of Postgraduate Studies & Professional Advancement | NSBM Green University

Overview

- ◆ Introduce the module
- ◆ Core principles
- ◆ Tools in use
- ◆ Revision – HTML, CSS, JavaScript

Web Technologies

- ◆ Server side technologies
 - ◆ ASP.NET, JSP, PHP, Python, RoR, etc.
- ◆ Implement server-side solution integrated with data store
 - ◆ MySQL & SQL Server
- ◆ Web Frameworks & Open source
- ◆ Test & Secure

Core Principles

- ◆ What is a web page?
- ◆ URI's & URLs
- ◆ Architecture
- ◆ HTTP headers

Static vs Dynamic web pages

A STATIC WEBSITE

A static website is the simplest kind of website you can build.

Static websites are written in [HTML](#) and [CSS](#) only, with no scripting.

The only form of interactivity on a static website is hyperlinks.

A DYNAMIC WEBSITE

A dynamic website is a website that not only uses HTML and CSS, but includes website scripting as well.

There are two main reasons why you'd want to use website scripting on your site:

- you want an interactive web app that people can use, not just read
- you want to be able to share HTML code between your pages.



URIs, URLs & URNs

- ◆ Uniform Resource Identifier

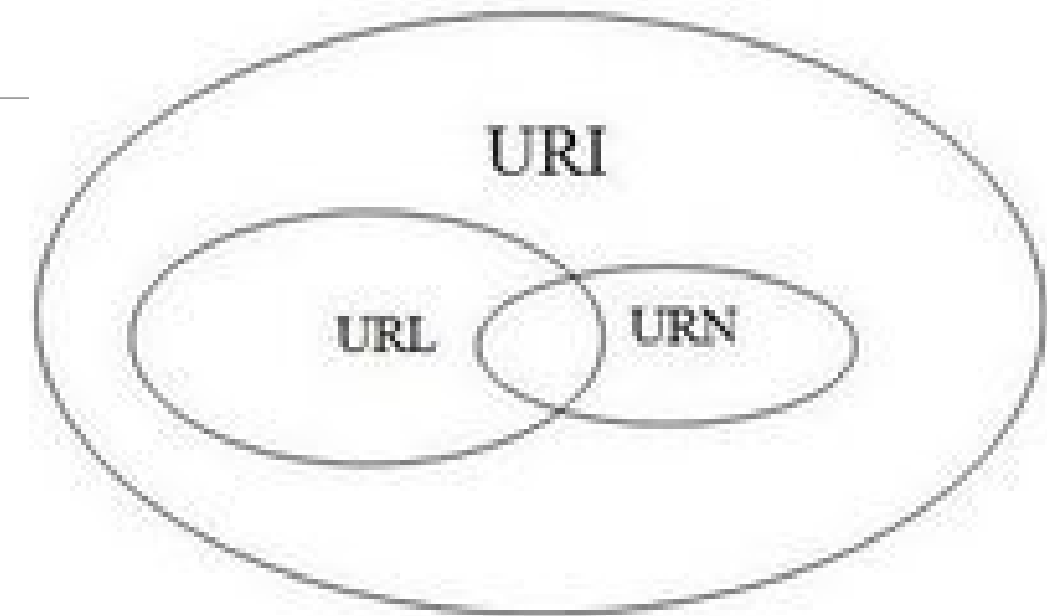
- ◆ Identifies a “thing” on the web

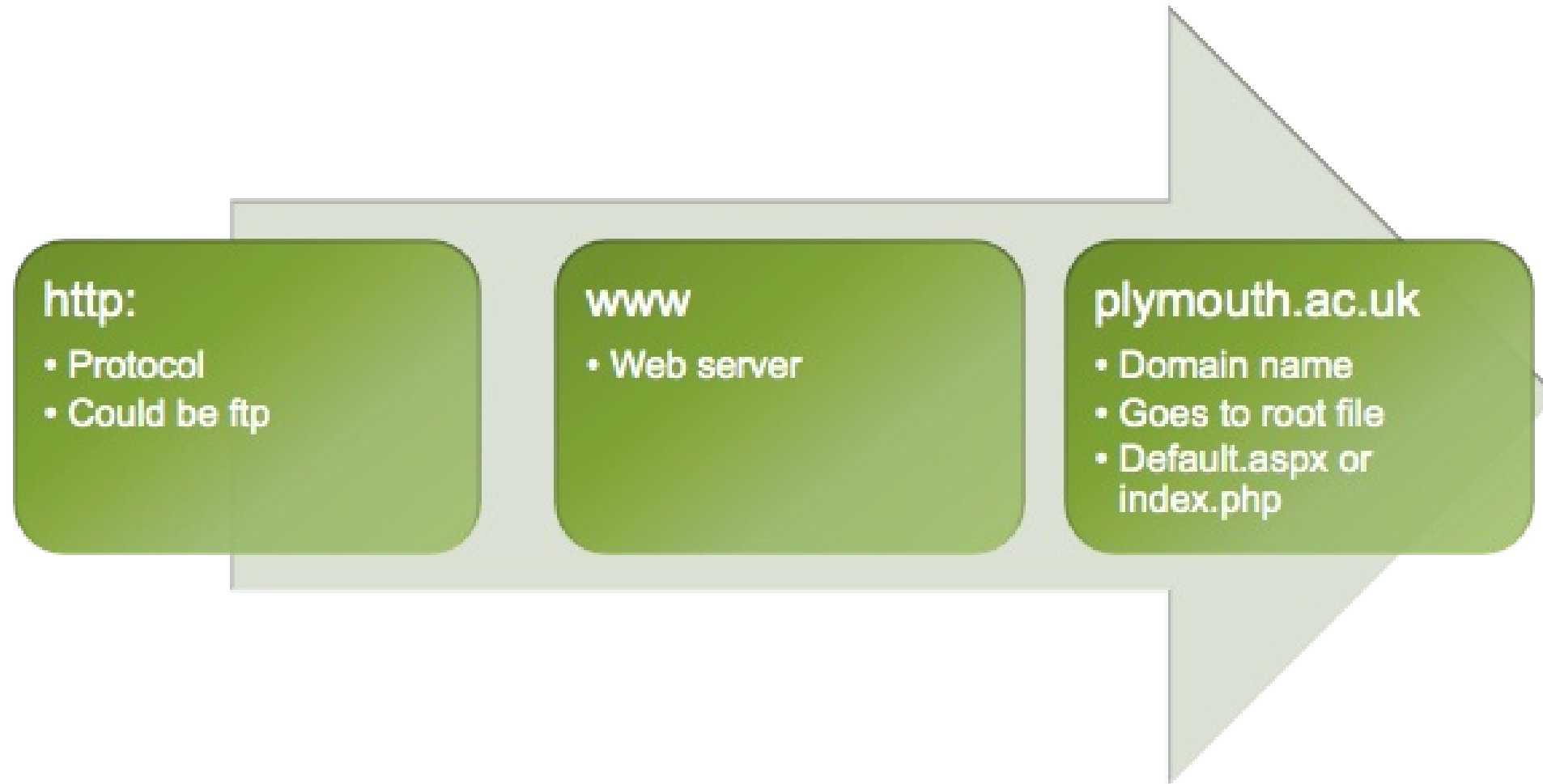
- ◆ Uniform Resource Locator

- ◆ A URI that starts with http
 - ◆ A URL contains information that directs a browser to the resource that the user wishes to access.
 - ◆ Web servers make such resources available to web clients.

- ◆ Uniform Resource Name

- ◆ The URN is the “globally unique” part of the identification; it’s a unique name.
 - ◆ Ex: urn:isbn:0451450523





http://www.webhomecover.com:80/help/callcentres.html



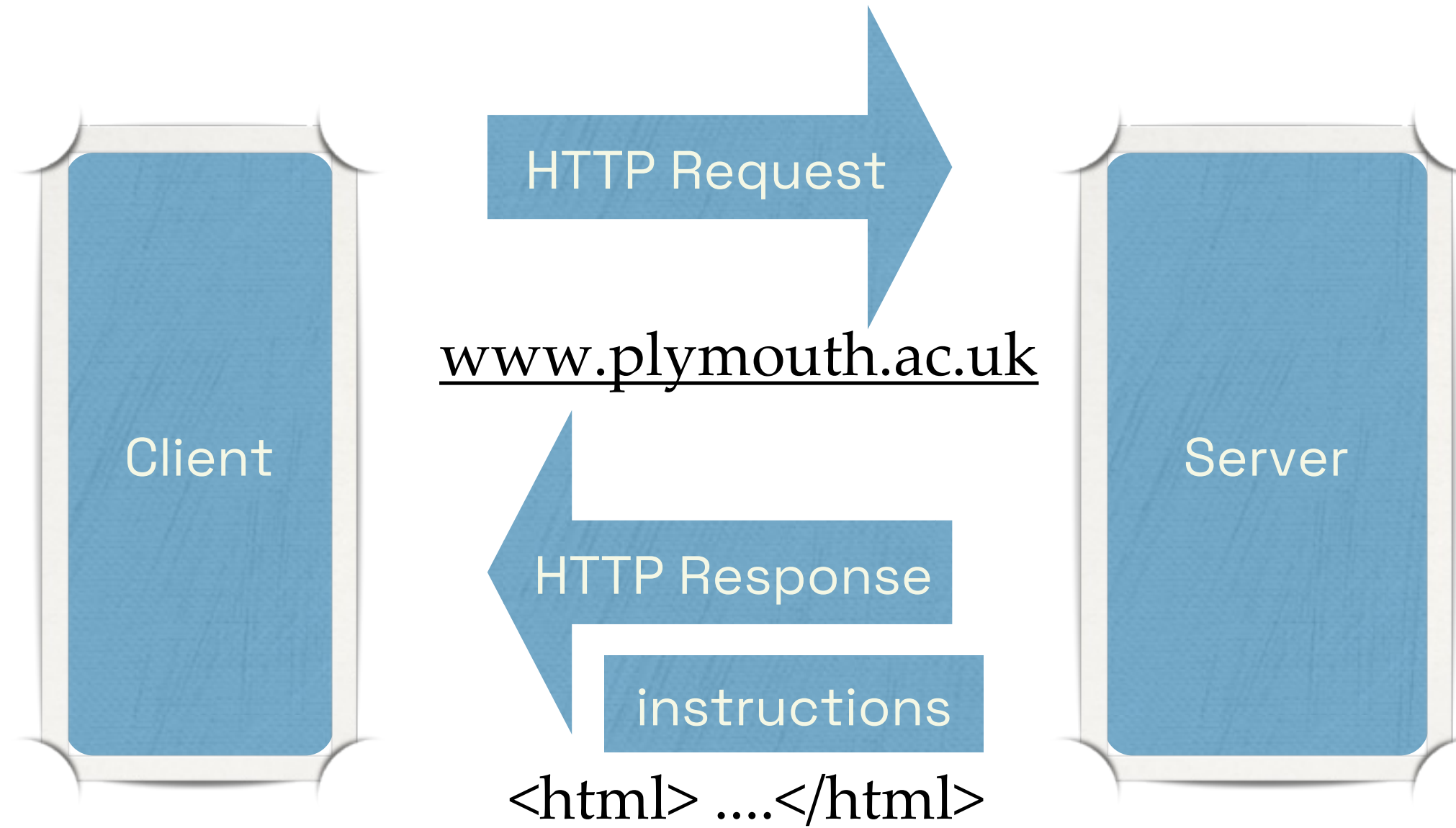
Can be the name of a resource that generates dynamic content

Exercise

- ◆ `ftp://ftp.is.co.za/rfc/rfc1808.txt`
- ◆ `http://www.ietf.org/rfc/rfc2396.txt`
- ◆ `ldap://[2001:db8::7]/c=GB?objectClass?one`
- ◆ `mailto:John.Doe@example.com`
- ◆ `news:comp.infosystems.www.servers.unix`
- ◆ `telnet://192.0.2.16:80/`
- ◆ `urn:oasis:names:specification:docbook:dtd:xml:4.1.2 1.1.2`
- ◆ `tel:+1-816-555-1212 (?)`

IP Addresses

- ◆ Routers “glue” all the IP addresses together
- ◆ Important concept to remember
- ◆ IP convert to domain names
- ◆ Domain Name System (DNS) allows domain names to be resolved to the correct IP address.
- ◆ The format of an IP address is a 32-bit numeric address written as four numbers separated by periods. Each number can be zero to 255. For example, 1.160.10.240 could be an IP address.



HTTP headers

Content-type : text/html

- ◆ Additional information is provided in the HTTP headers.
 - ◆ The information provided in the header specifies the Multipurpose Internet Mail Extensions (MIME) type of the content that the server is transmitting to the browser.
 - ◆ The MIME standard specifies data formats, which programs can use to interpret data correctly.
- ◆ Provides information about what the file is that the server is sending to the client (MIME type).
- ◆ HTTP Request types
 - ◆ Get
 - ◆ Post
 - ◆ ...

www.google.co.uk/search?q=atkinson

`$_GET["q"]` or `Request.QueryString("q")`

`$_POST["q"]` or `Request.Form("q")`

HTTP Get and Post Requests

The two most common HTTP request types are get and post.

A get request typically gets (or retrieves) information from a server, such as an HTML document, an image or search results based on a user-submitted search term.

A post request typically posts (or sends) data to a server.

Common uses of post requests are to send form data or documents to a server.

An HTTP request often posts data to a server-side form handler that processes the data.

HTTP Get and Post Requests (Cont.)

A get request appends data to the URL, e.g., `www.google.com/search?q=deitel`.

The server uses data passed in a **query string** to retrieve an appropriate resource from the server.

The server then sends a response to the client. A get request may be initiated by submitting an HTML form whose method attribute is set to "get", or by typing the URL (possibly containing a query string) directly into the browser's address bar.

A post request sends form data as **part of the HTTP message**, not as part of the URL.

A get request typically limits the query string (i.e., **everything to the right of the ?**) to a specific number of characters, so it's often necessary to send large amounts of information using the post method.

The form data still reaches the server and is processed in a similar fashion to a get request, but the user does not see the exact information sent.

Stateless

- ◆ Client makes request to server
- ◆ Server does not remember what got sent before
- ◆ Client runs browser software, uses http to request pages from server
- ◆ Server listens for clients asking for pages so runs server software
 - ◆ Apache
 - ◆ IIS
- ◆ Server transmits status code, web page and associated files

Client-Side Scripting versus Server-Side Scripting

Client-side scripting with JavaScript can be used to validate user input, to interact with the browser, to enhance web pages, and to add client/server communication between a browser and a web server.

Client-side scripting does have limitations, such as browser dependency; the browser or [scripting host](#) must support the scripting language and capabilities.

Scripts are restricted from arbitrarily accessing the local hardware and file system for security reasons.

Another issue is that client-side scripts can be viewed by the client by using the browser's source-viewing capability.

Client-Side Scripting versus Server-Side Scripting (cont.)

Programmers have more flexibility with [server-side scripts](#), which often generate custom responses for clients.

For example, a client might connect to an airline's web server and request a list of flights from Boston to San Francisco between April 19 and May 5. The server queries the database, dynamically generates an HTML document containing the flight list and sends the document to the client. This technology allows clients to obtain the most current flight information from the database by connecting to an airline's web server.

Server-side scripting languages have a wider range of programmatic capabilities than their client-side equivalents.

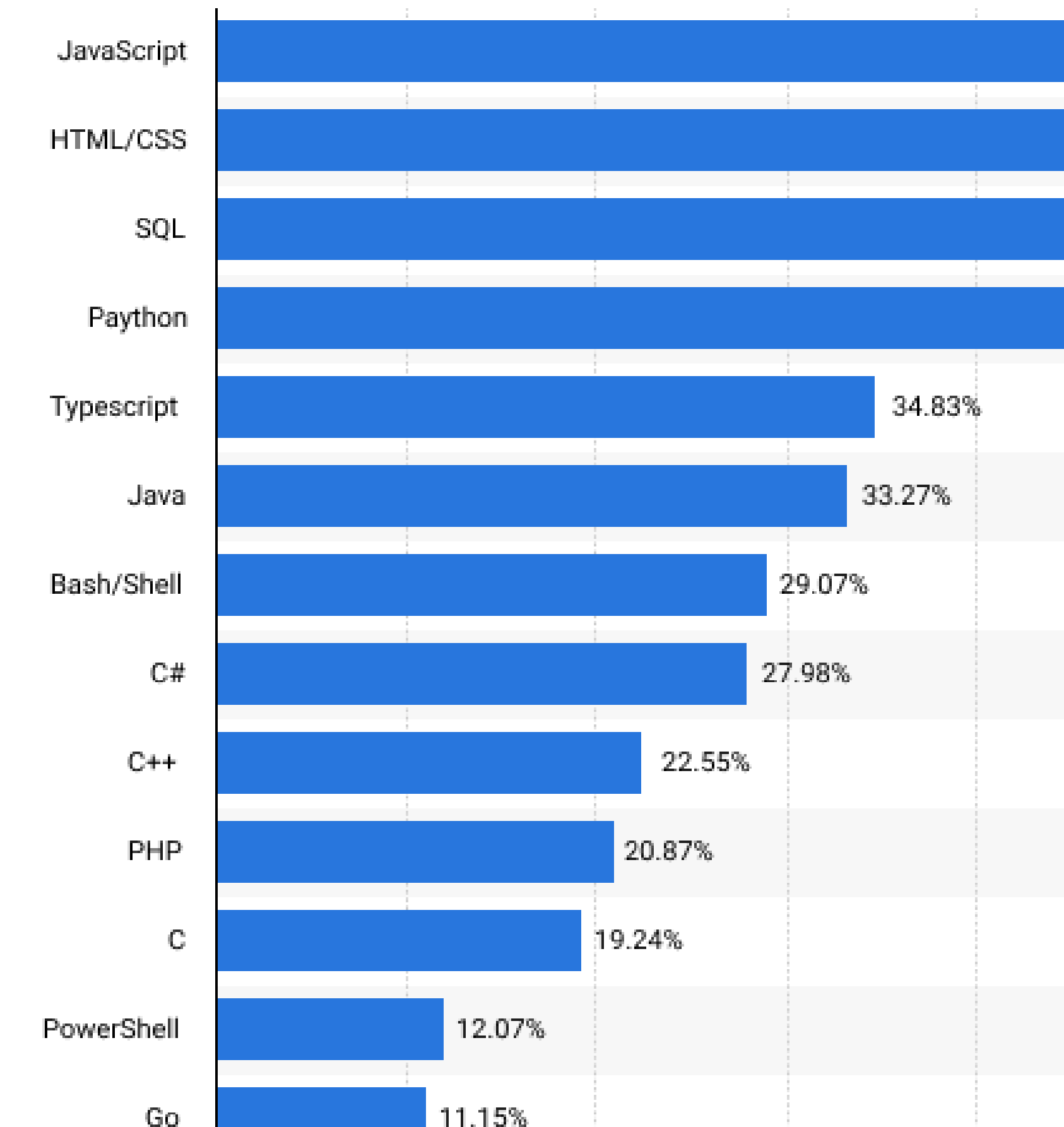
IIS web server

- ◆ Automatic with some versions of windows
- ◆ Set up through the control panel – Program Features
- ◆ Documents for serving on web go into c:\inetput\www
 - ◆ or you create a virtual directory

Web servers- Apache

- ◆ Open source software
- ◆ Runs on Unix, Linux, Mac & Windows
- ◆ Comes as a bundle with PHP & MySQL (WAMP, XAMP etc)
- ◆ Documents served from default directory
 - ◆ htdocs
 - ◆ www

In-Demand Programming Languages in 2024



Language Rank	Types	Spectrum Ranking
1. Python	 	100.0
2. C	  	99.7
3. Java	  	99.5
4. C++	  	97.1
5. C#	  	87.7
6. R		87.7
7. JavaScript	 	85.6
8. PHP		81.2
9. Go	 	75.1
10. Swift	 	73.7

	PHP	ASP.Net	JSP	Python
Developer	The PHP Group	Microsoft Corporation	Adobe (previously Macromedia)	Python Software Foundation
Cost	Free download and hosting on open-source Linux server.	Free download but possible Windows hosting cost (although see Operating System comparison below).	Free download and hosting on open source Linux server.	Free download and hosting on open-source Linux server.
Ease of use	Simple, text based language with numerous online tutorials.	Easy to use particularly with Microsoft's integrated development environment software, WebMatrix.	More complex to learn. Also, using JSP requires some knowledge of Java programming language.	Programming language. More complex to learn but voted best language for beginners serious about learning programming.

	PHP	ASP.Net	JSP	Python
Implementation language	C based language.	.NET languages	Java language	C/C++ based languages and Java language.
Operating system	All modern operating systems including mobile platforms (iOS and Android).	Platform dependent – Windows operating system required to develop in ASP.NET	All modern operating systems including mobile platforms (iOS and Android).	All modern operating systems.



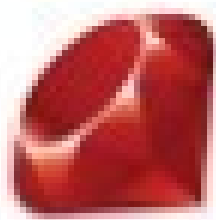
Static | Multiple languages | Mono on Linux | Good Docs & Support | Proprietary | \$\$\$



Static | Multiple languages on the JVM | Good Docs | Big Developer Community | Heavy Frameworks | Oracle?



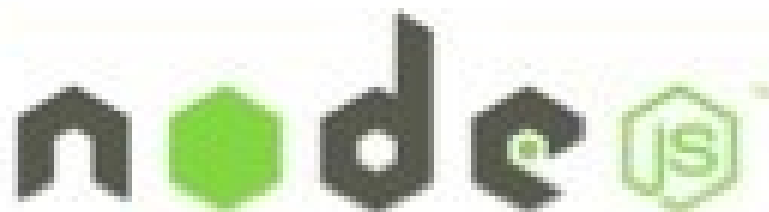
Dynamic | Scientific | Easy | Strong Cloud Support | Scattered Docs & Libs | Light | Single Threaded



Dynamic | Easy | Learning Resource | Web Support | Light | Scaling Limits



Dynamic | Vast Libraries | Big Developer Community | Long History | Spaghetti Code | Horrible to Awesome



Dynamic | New | Skill Reuse | Event Driven | Limited Libraries and Breadth | Smaller Community

FRONT END

- ▶ Html
- ▶ Css3
- ▶ Java Script
- ▶ Angular JS
- ▶ ReactJS

BACK END

- ▶ Ruby
- ▶ Java
- ▶ Python
- ▶ PHP
- ▶ .Net

DATABASE

- ▶ My SQL
- ▶ MongoDB
- ▶ CouchDB
- ▶ MS SQL

VERSION CONTROL

- ▶ Git
- ▶ Grunt
- ▶ Xdebug
- ▶ Sub Version

PROJECT MGMT TOOLS

- ▶ Basecamp
- ▶ Jira
- ▶ Trello
- ▶ Red mine
- ▶ Team work



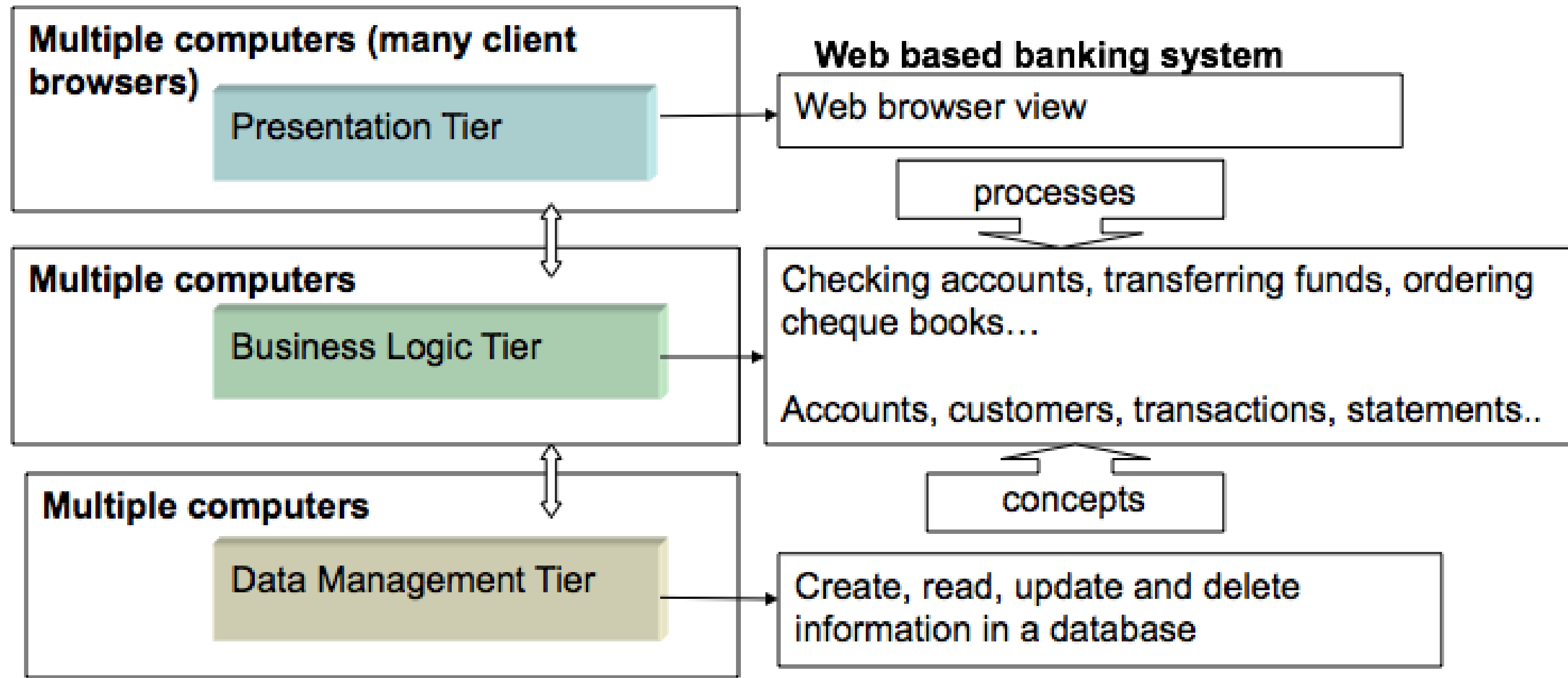
Revision – HTML, CSS, JavaScript (Cont.)

2. Write the Javascript code to fill the selection box below with the following five choices: Garfield, Odie, Nermal, Pooky, and Jon. When the selected item in the box changes, the image below the selection box changes to display a picture of the character. The images are located in the images/ subfolder from the web page and are saved in JPG format with file names matching the character names.

After your Javascript code runs, the appearance and behavior should be the following



Coding Architecture



Helpful Links

- www.w3schools.com
 - HTML
 - CSS
 - JavaScript
- <https://www.linkedin.com/>
 - Web Development Foundations: Web Technologies

Software Quality Assurance



Pavithra Subashini

Senior lecturer

Faculty of Computing



Test case is defined as

- A set of test inputs, execution conditions and expected results, developed for a particular objective.
- Documentation specifying inputs, predicted results and a set of execution conditions for a test item.
- Specific inputs that will be tried and the procedures that will be followed when the software tested.
- Sequence of one or more subtests executed as a sequence as the outcome and/or final state of one subtests is the input and/or initial state of the next.
- Specifies the pretest state of the AUT and its environment, the test inputs or conditions.
- The expected result specifies what the AUT should produce from the test inputs.

Test Cases

Project Name		Test Designed by:	
Module Name		Test Designed Date:	
Release Version		Test Executed by	
		Test Execution date	

Precondition							
Dependencies							
Test Priority							

Test Case ID	Test Title	Test Summary	Test Steps	Test Data	Expected Result	Post Condition	Actual Results	Status	Notes
TC_GMAIL_001	Gmail Registration	Verify Create account / registration	1. Open IE browser 2. Type https://accounts.google.com/SignUp?continue 3. Press Enter	Gmail URL	Gmail Registration page should open	None	Gmail Registration page open	PASS	

- Test Case ID
- Test Title
- Test Summary/ Objective/ Description
- Test Case Steps
- Test Data
- Expected result
- Post Condition
- Actual result
- Status / Pass/fail
- Notes /Comments



Good Test Cases

Find Defects

- Have high probability of finding a new defect.
- Unambiguous tangible result that can be inspected.
- Repeatable and predictable
- Traceable to requirements or design documents
- Push systems to its limits
- Execution and tracking can be automated
- Do not mislead
- Feasible



Test Case Design Technique

Black-box testing (or functional testing):

- Equivalence partitioning
- Boundary value analysis
- Cause-effect graphing
- Behavioural testing
- Random testing



Black-box: Three major approaches

- Analysis of the input/output domain of the program:
Leads to a logical partitioning of the input/output domain into 'interesting' subsets
- Analysis of the observable black-box behavior:
Leads to a flow-graph-like model, which enables application of techniques from the white-box world (on the black-box model)
- Techniques like risk analysis, random input, stress testing



Black-box : Equivalence Partitioning

Equivalence Partitioning also called as equivalence class partitioning. It is abbreviated as ECP. It is a software testing technique that divides the input test data of the application under test into each partition at least once of equivalent data from which test cases can be derived.

An advantage of this approach is it reduces the time required for performing testing of a software due to less number of test cases.



Black-box : Equivalence Partitioning

Example:

If one application is accepting input range from 1 to 100, using equivalence class we can divide inputs into the classes, for example, one for valid input and another for invalid input and design one test case from each class.

In this example test cases are chosen as below:

- One is for valid input class i.e. selects any value from input between ranges 1 to 100. So here we are not writing hundreds of test cases for each value. Any one value from this equivalence class should give you the same result.
- One is for invalid data below lower limit i.e. any one value below 1.
- One is for invalid data above upper limit i.e. any value above 100.



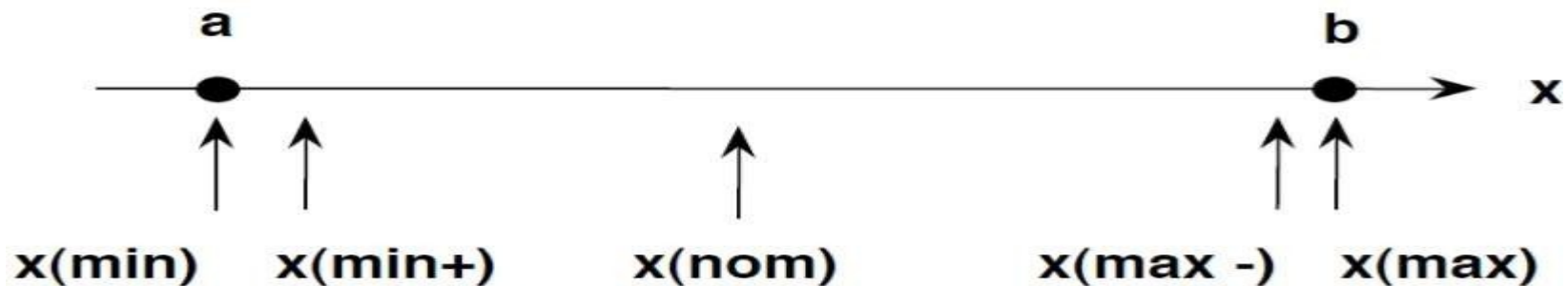
Boundary value analysis

- Boundary value analysis is a type of black box or specification based testing technique in which tests are performed using the boundary values.
- Boundary Testing comes after the Equivalence Class Partitioning.
- Testing between extreme ends or boundaries between partitions of the input values.

Example:

Input data 1 to 10 (boundary value)

Test input data 0, 1, 2 to 9, 10, 11





Decision Table

- A decision table is a good way to deal with combinations of Inputs
- Also referred as cause- effect table
- Focused on business logic or business rules
- Decision tables provide a systematic way of stating complex business rules,

Conditions / Inputs		Actions / Output
UserName	Password	
Valid (T)	Valid (T)	Login
Invalid (F)	Valid (T)	Error message
Valid (T)	Invalid (F)	Error message
Invalid (F)	Invalid (F)	Error message

Note:- Number of possible Combinations is given by 2^n , where n is the number of Inputs.

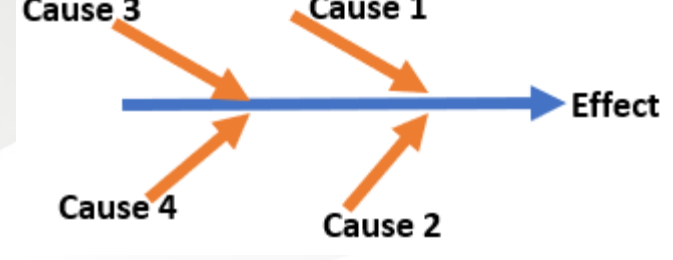
In the above example :- we would need $2^2 = 4$ combinations



Decision Table

Advantages:-

- 1) It provide complete coverage of test cases which help to reduce the rework on writing test scenarios & test cases.
- 2) Any complex business flow can be easily converted into the test scenarios & test cases using this technique.
- 3) Simple to understand and everyone can use this method design the test scenarios & test cases.
- 4) These tables guarantee that we consider every possible combination of condition values. This is known as its “completeness property”.



Cause Effect Graph

A “Cause” represents a distinct input condition that brings about an internal change in the system.

An “Effect” represents an output condition, a system transformation or a state resulting from a combination of causes.

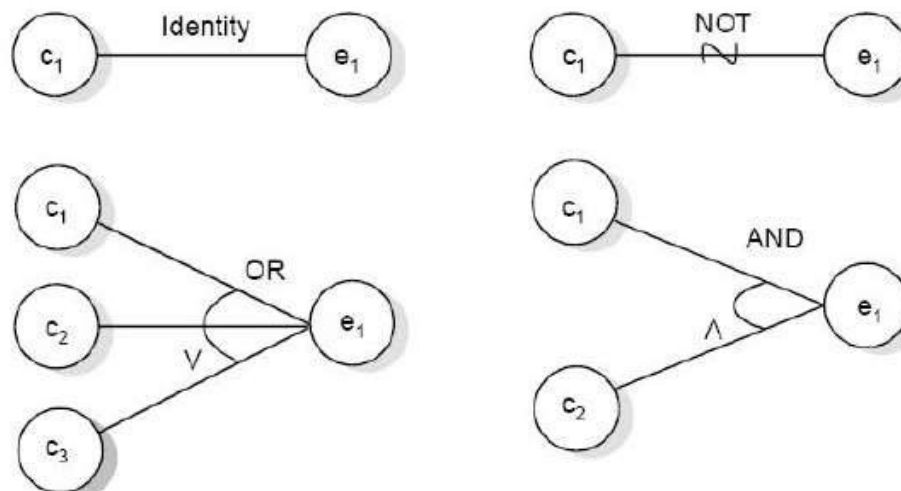
It is also known as Ishikawa diagram as it was invented by Kaoru Ishikawa or fish bone diagram because of the way it looks.

Circumstances - under which Cause-Effect Diagram used

- To Identify the possible root causes, the reasons for a specific effect, problem, or outcome.
- To Relate the interactions of the system among the factors affecting a particular process or effect.
- To Analyze the existing problems so that corrective action can be taken at

Benefits:

- It Helps us to determine the root causes of a problem or quality using a structured approach.
- It Uses an orderly, easy-to-read format to diagram cause-and-effect relationships.
- It Indicates possible causes of variation in a process.
- It Identifies areas, where data should be collected for further study.
- It Increases knowledge of the process by helping everyone to learn more about



Basic cause effect graph symbols



Error Guessing

- Make use of a tester's skill intuition and experience in testing similar applications to identify defects that may not be easy to capture by the more formal techniques. It is usually done after more formal techniques are completed.
- Mainly used during release of application
- Usually used by the experts
- Its less reproducible
- Coverage is limited by experience, knowledge and time available
- Types of defects depends on the scenarios that were guessed
- Checklist based testing
- Typical conditions to try include division by zero, blank (or no) input, empty files and the wrong kind of data (e.g. alphabetic characters where numeric are required).



Random Testing

What is Random Testing?

Random Testing, also known as monkey testing, is a form of functional black box testing that is performed when there is not enough time to write and execute the tests.

Random Testing Characteristics:

- Random testing is performed where the defects are NOT identified in regular intervals.
- Random input is used to test the system's reliability and performance.
- Saves time and effort than actual test efforts.
- Other Testing methods cannot be used to.

Random Testing Steps:

- Random Inputs are identified to be evaluated against the system.
- Test Inputs are selected independently from test domain.
- Tests are Executed using those random inputs.
- Record the results and compare against the expected outcomes.
- Reproduce/Replicate the issue and raise defects, fix and retest.



Items in test cases



- **Test Case ID**
- **Test Description/Test case name**
- **Prerequisite**
- **Test Steps**
- **Input Data**
- **Expected Result**
- **Actual Result**
- **Status**
- **Severity**
- **Priority**
- **Executed by**
- **Comments**

Project details

- **Project name**
- **Module name**
- **Created by**
- **Created date**
- **Reviewed by**
- **Reviewed date**



Sample 1

- Let's consider the behavior of Order Pizza Text Box Below
- Pizza values 1 to 10 is considered valid. A success message is shown.
- While value 11 to 99 are considered invalid for order and an error message will appear, **"Only 10 Pizza can be ordered"**

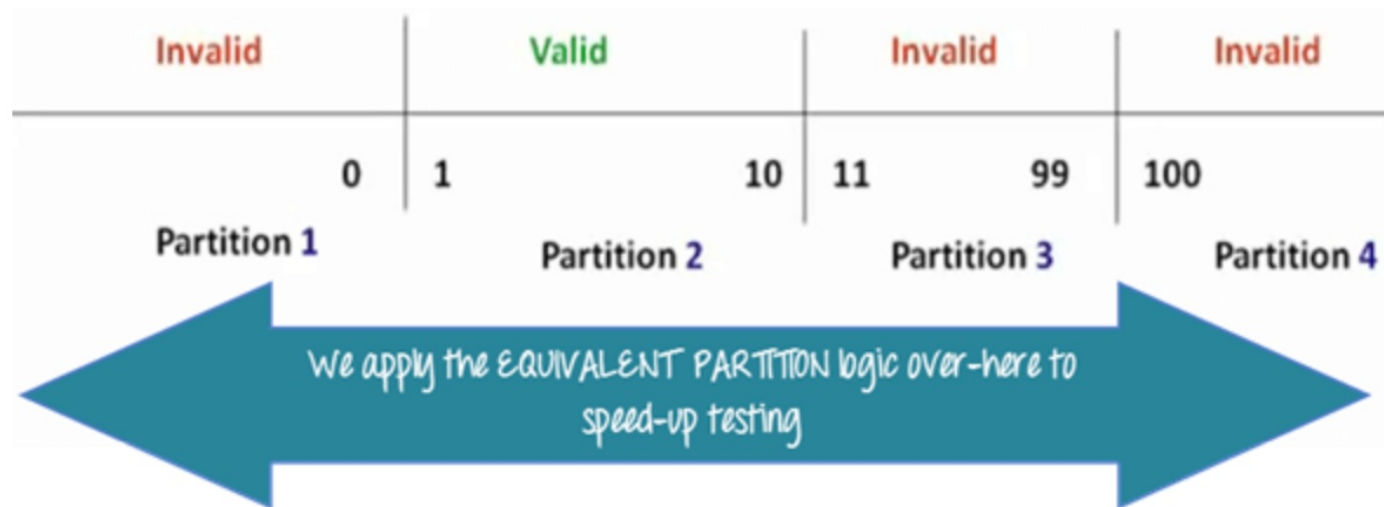
Order Pizza:

Submit

Here is the test condition

1. Any Number greater than 10 entered in the Order Pizza field(let say 11) is considered invalid.
2. Any Number less than 1 that is 0 or below, then it is considered invalid.
3. Numbers 1 to 10 are considered valid
4. Any 3 Digit Number say -100 is invalid.

Equivalence Partitioning





Boundary value analysis

Invalid	Valid	Invalid	Invalid
0	1 10	11 99	100
Partition 1	Partition 2	Partition 3	Partition 4

In BOUNDARY VALUE ANALYSIS you will check the boundary values like 0, 1, 10, 11, 99, 100



Example 2: Equivalence and Boundary Value

Following password field accepts minimum 6 characters and maximum 10 characters

That means results for values in partitions 0-5, 6-10, 11-14 should be equivalent

Enter Password:

Submit

Test Scenario #	Test Scenario Description	Expected Outcome
1	Enter 0 to 5 characters in password field	System should not accept
2	Enter 6 to 10 characters in password field	System should accept
3	Enter 11 to 14 character in password field	System should not accept



Open excel sheet
Start test case writing



Activity

- Write test cases to test below web page.

Keells

Track Order

Enter your order number to start tracking the status of the order

Your Order Number

Order number can be found on the order confirmation email

Your Contact Number / Email Address

Submit

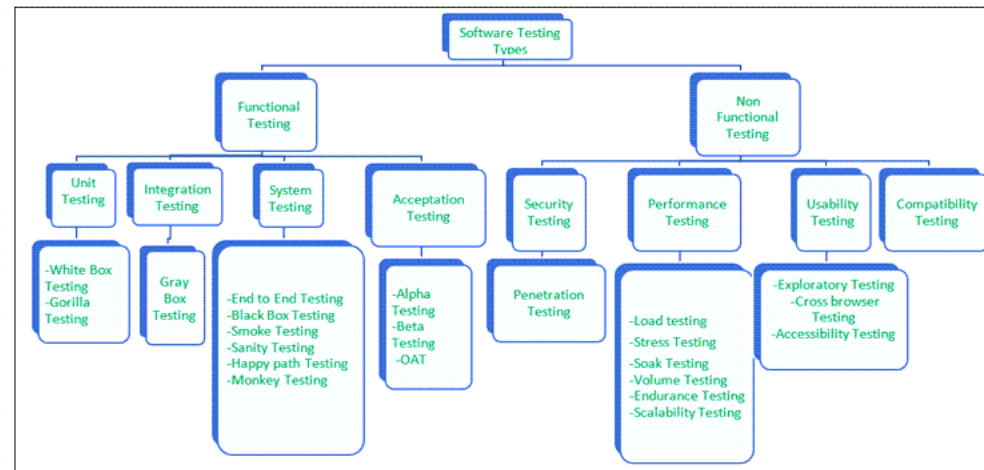




Types of testing

K.K.Pavithra Subashini

Types of testing



Functional Testing

1) Unit Testing

- **Unit testing** is a type of software testing which is done on an individual unit or component to test its corrections.
- Unit testing is done by the developer at the application development phase.
- Developers often use test automation tools such as NUnit, Xunit, JUnit for the test execution.
- Unit testing is important because we can find more defects at the unit test level.

a) White Box Testing

- **White box** testing is a test technique in which the internal structure or code of an application is visible and accessible to the tester.
- In this technique, it is easy to find loopholes in the design of an application or fault in business logic.

b) Gorilla Testing

Gorilla testing is a test technique in which the tester and/or developer test the module of the application thoroughly in all aspects.

Gorilla testing is done to check how robust your application is.

2) Integration Testing

- Integration testing is a type of software testing where two or more modules of an application are logically grouped together and tested as a whole.
- The focus of this type of testing is to find the defect on interface, communication, and data flow among modules.

a) Gray box testing

- gray box testing is a combination of white-box testing and black-box testing.
- Testers have partial knowledge of the internal structure or code of an application.

3) System Testing

- System testing is types of testing where tester evaluates the whole system against the specified requirements.

a) End to End Testing

- It involves testing a complete application environment

b) Black Box Testing

- testing is performed without knowing the internal structure, design, or code of a system under test.
- Testers should focus only on the input and output of test objects.

c) Smoke Testing

- Smoke testing is performed to verify that basic and critical functionality of the system under test is working fine at a very high level.

d) Sanity Testing

- Sanity testing is performed on a system to verify that newly added functionality or bug fixes are working fine.
- Sanity testing is done on stable build.
- It is a subset of the regression test.

e) Happy path Testing

- The objective of Happy Path Testing is to test an application successfully on a positive flow.
- It does not look for negative or error conditions.
- The focus is only on valid and positive inputs through which the application generates the expected output.
- **f) Monkey Testing**
- **Monkey Testing** is carried out by a tester, assuming that if the monkey uses the application, then how random input and values will be entered by the Monkey without any knowledge or understanding of the application.
- The objective of Monkey Testing is to check if an application or system gets crashed by providing random input values/data.

4) Acceptance Testing

- Acceptance testing is a type of testing where client/business/customer test the software with real time business scenarios.
- The client accepts the software only when all the features and functionalities work as expected.
- This is the last phase of testing, after which the software goes into production.
- This is also called User Acceptance Testing (UAT).

Non-Functional Testing

1) Security Testing

- Security Testing is done to check how the software, application, or website is secure from internal and/or external threats.
- This testing includes how much software is secure from malicious programs, viruses and how secure & strong the authorization and authentication processes are.
- It also checks how software behaves for any hacker's attack & malicious programs and how software is maintained for data security after such a hacker attack.

2) Performance Testing

- Performance testing is testing of an application's stability and response time by applying load.
- Performance testing is done with the help of tools. Loader.IO, JMeter, LoadRunner, etc

a) Load testing

- Load testing is testing of an application's stability and response time by applying load, which is equal to or less than the designed number of users for an application.

b) Stress Testing

- Stress testing is testing an application's stability and response time by applying load, which is more than the designed number of users for an application.

d) Volume testing

- Volume testing is testing an application's stability and response time by transferring a large volume of data to the database. Basically, it tests the capacity of the database to handle the data.

e) Endurance Testing

- Endurance testing is testing an application's stability and response time by applying load continuously for a longer period to verify that the application is working fine.

3) Usability Testing

- Usability testing is testing an application from the user's perspective to check the look and feel and user-friendliness.

a) Exploratory testing

- Exploratory Testing is informal testing performed by the testing team.
- The objective of this testing is to explore the application and look for defects that exist in the application.
- Testers use the knowledge of the business domain to test the application.

b) Cross browser testing

- Cross browser testing is testing an application on different browsers, operating systems, mobile devices to see look and feel and performance.

c) Accessibility Testing

- The aim of Accessibility Testing is to determine whether the software or application is accessible for disabled people or not.

4) Compatibility testing

- **Compatibility testing** ensures that software can run on different configuration, different databases, different browsers, and their versions. The testing team performs compatibility testing.

Other Types of Testing

- **Ad-hoc Testing**

- This testing is performed on an ad-hoc basis, i.e., with no reference to the test case and also without any plan or documentation in place for this type of testing.
- The objective of this testing is to find the defects and break the application by executing any flow of the application or any random functionality.

- **Browser Compatibility Testing**

- Browser Compatibility Testing is performed for web applications and ensures that the software can run with a combination of different browsers and operating systems.
- This type of testing also validates whether a web application runs on all versions of all browsers or not.

- **Black Box Testing**

- Internal system design is not considered in this type of testing. Tests are based on the requirements and functionality.

Regression Testing

- Regression testing is testing of unchanged features of the application to make sure that any bug fixes, adding new features, deleting, or updating existing features, are not impacting the working application.

Risk-Based Testing (RBT)

- For [Risk-Based Testing](#), the functionalities or requirements are tested based on their priority. Risk-Based Testing includes testing of highly critical functionality, which has the highest impact on business and in which the probability of failure is very high.

Negative Testing

- Negative Testing technique is performed using incorrect data, invalid data, or input. It validates if the system throws an error of invalid input and behaves as expected.

Software Quality Assurance



Pavithra Subashini

Senior lecturer

Faculty of Computing



Fundamental Test Process



Fundamental Test Process

1. Planning and control



2. Test analysis and design



3. Test implementation and execution



4. Evaluating exit criteria and reporting



5. Test closure activities:



Fundamental Test Process cont.....

1. Planning and control:

- Defining test strategy and policies are fundamental for establishing a clear roadmap and set of rules which could be shared with the stakeholders. This is crucial for ensuring effective testing in a timely fashion.
- Main 6 key points for identifying required planning and control activities:
 - determine scope, risks and test objectives: the goal is to make sure that each requirement is covered by a test plan.
 - determine test approach in term of test procedures, test techniques, teams, environment and data involved: this ensures to clearly identify what's required (feasibility) from coming testing activities.
 - implement test policy/strategy previously determined.
 - determine test resources (environment, people) needed so they can be planned/allocated.
 - schedule all test activities: plan, design, implementation, execution, evaluation.
 - determine the exit criteria which could be test coverage (% expected), number of test executed... It's a useful step which has to be agreed with stakeholders because it determines the end of testing and therefore software/product release.



Planning and control cont..

- Regarding controlling test activities, the objective is ensure that what was previously planned is well implemented and communicated. This includes the following actions:
- measure and analyze the results: how much test has been executed, how much defects have been found classified by their severity..
- monitor and document progress, because stakeholders, project managers and the testing team want to be aware of what has been performed so far, what has been identified and what's left to verify.
- initiate corrective actions as a test strategy can be tailored or refined during progress.
- make decisions regarding stopping or restarting testing activities or simply confirming GO for release.



Fundamental Test Process cont.....

2. Test analysis and design:

- This is in relation with understanding software requirements and defining test objectives and in order to allow that, we will:
- review test basis and understand clearly the software specifications and deliverable.
- identify test conditions, so list every feature, transaction, function, quality attribute or other structural element that could be verified by one or more test cases.
- evaluate testability of requirements: specify expected results and make sure requirements can be verified.
- design test environment: identify any tool or infrastructure required for testing the software/product.



Fundamental Test Process cont.....

3. Test implementation and execution:

- All high-level design performed has now to be built in test cases and manual or automated test procedures. So before execution, following preparation tasks are required:
 - develop and prioritize our test cases by describing step by step instructions needed to perform the test.
 - create test suites or a logical collection of common test cases to execute for ensuring that test scheduling is well managed and the feature/transaction clearly end-to-end tested.
 - implement and verify the environment to make sure that it's ready and will not introduce errors or block us in test execution.
- Then, during execution :
 - ensure that test suites and individual test cases are run in the planned sequenced with the right tools and environment.
 - log the outcome of each test execution and record software/product identities and version tested, data used, tools and environment involved. This is very important and contributes to make sure that configuration items are well managed.
 - compare actual results to expected results and provide additional information and details on how the defect occurs, ensuring reproducibility.
 - retest after the defect has been fixed and automatically perform regression test.



Fundamental Test Process cont.....

4. Evaluating exit criteria and reporting

- For each test level, exit criteria has to be defined beforehand and a reporting has to be provided to inform stakeholders for recording official decision taken:
- checks test logs against exit criteria in test planning and identify what's remain to test or to fix.
- assess if more tests are needed and if initial exit criteria has to be reset and agreed again with stakeholders.
- write a summary report as a test deliverable on which clear decision could be documented. .



Fundamental Test Process cont.....

5. Test closure activities

- Ending testing includes various and sequential closing activities that would lead to lesson learning. Therefore, we have to:
- check which planned acceptance or rejection deliverable has been delivered, defects resolved or differed in a future release.
- finalize and archive test ware for an eventual reuse (script, data, tools, environment).



Software Development Tools & Practices

PUSL2020

Level 2

“APS.NET Development”

Dr. Rasika Ranaweera (ranaweera.r@nsbm.ac.lk | +94 11 544 6126)

Faculty of Computing | NSBM Green University

ASP.NET Web Forms

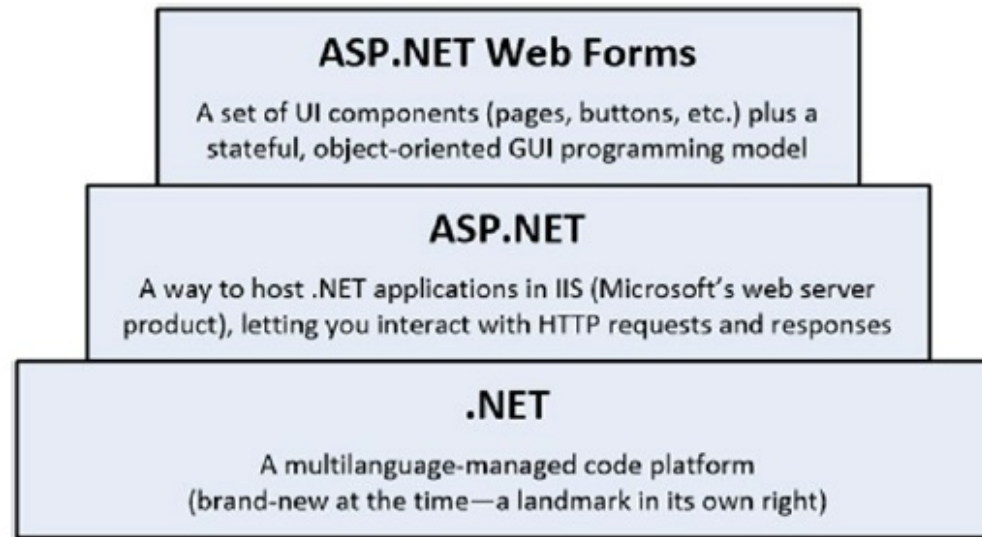
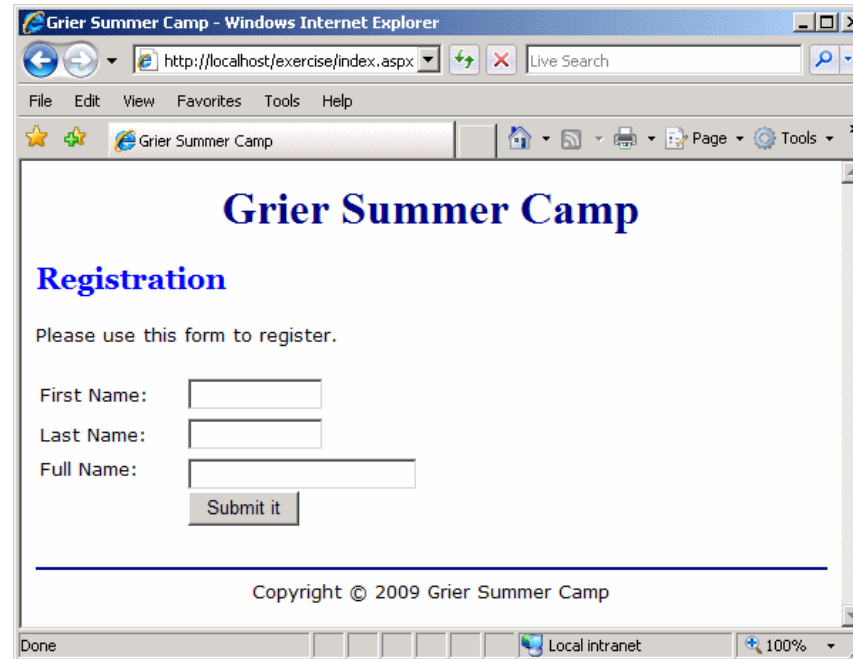


Figure 1-1. The ASP.NET Web Forms technology stack

With Web Forms, Microsoft attempted to hide both *Hypertext Transfer Protocol* (HTTP), with its intrinsic statelessness, and *Hypertext Markup Language* (HTML), which at the time was unfamiliar to many developers, by modeling the user interface (UI) as a hierarchy of server-side control objects.

ASP.NET Web Forms



Each control kept track of its own state across requests, rendering itself as HTML when needed and automatically connecting client-side events (for example, a button click) with the corresponding server-side event handler code. In effect, Web Forms is a giant abstraction layer designed to deliver a classic event-driven graphical user interface (GUI) over the Web.

The problems of ASP.NET Web Forms

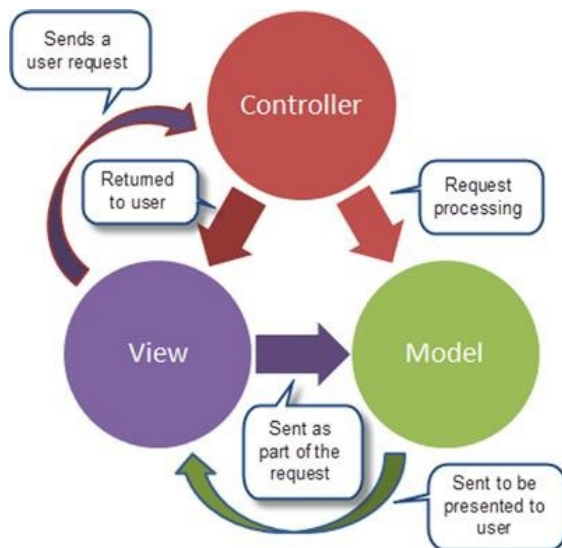
1. *View State weight*: The actual mechanism for maintaining state across requests (known as View State) resulted in large blocks of data being transferred between the client and server;
2. Page life cycle: The mechanism for connecting client-side events with server-side event handler code, part of the page life cycle, could be complicated and delicate;
3. False sense of separation of concerns;
4. Limited control over HTML: In early versions of ASP.NET, the HTML output failed to meet web standards or make good use of Cascading Style Sheets;
5. Leaky abstraction
6. Low testability

The development of Web Forms

- Web Forms wasn't all bad, and eventually, Microsoft put effort into improving standards compliance and simplifying the development process and even took some features from the original ASP.NET MVC Framework and applied them to Web Forms.
- Web Forms excelled when you needed quick results, and you could have a reasonably complex web app up and running within a day. But unless you were careful during development, you would find that the application you created was hard to test and hard to maintain.

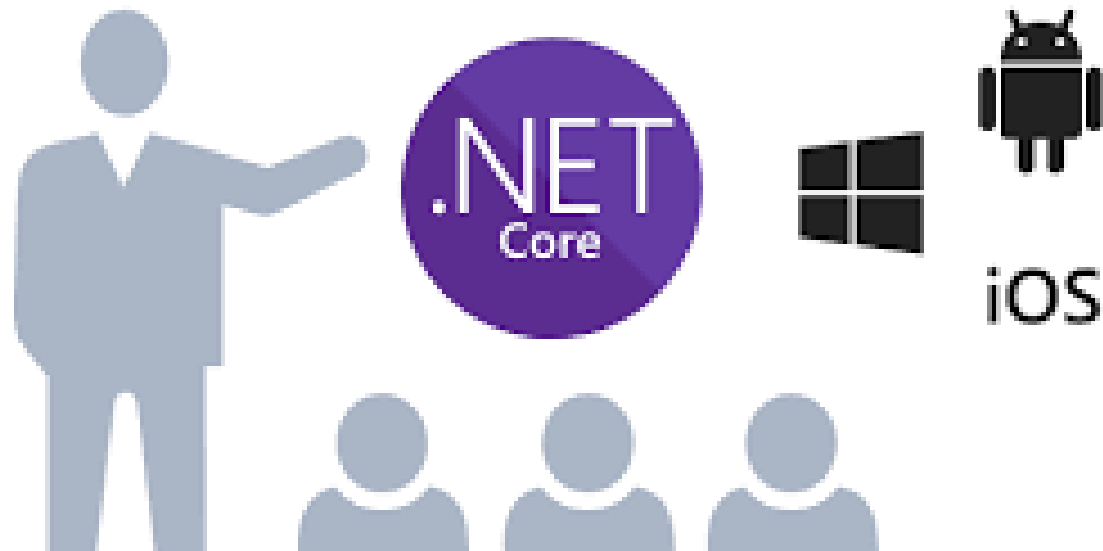
Original MVC Framework

- In October 2007, Microsoft announced a new development platform, built on the existing ASP.NET platform, that was intended as a direct response to the criticisms of Web Forms and the popularity of competing platforms such as Ruby on Rails. The new platform was called the ASP.NET MVC Framework and reflected the emerging trends in web application development, such as HTML and CSS standardization, RESTful web services, effective unit testing, and the idea that developers should embrace the stateless nature of HTTP.



Understanding ASP.NET Core

- ASP.NET Core is built on .NET Core, which is a cross-platform version of the .NET Framework without the Windows-specific application programming interfaces (APIs).
- Microsoft has extended the reach of .NET, made it possible to deploy ASP.NET Core applications to a broader set of hosting environments, and, as a bonus, made it possible for developers to create ASP.NET Core web applications on Linux and macOS.



ASP.NET Core MVC

- ASP.NET Core MVC provides the functionality of the original ASP.NET MVC Framework built on the new ASP.NET Core platform.
- It integrates the features that were previously provided by Web API, it includes a more natural way of generating complex content, and it makes key development tasks, such as unit testing, simpler and more predictable.
- The ASP.NET Core MVC 2 release focuses on consolidation, working through some of the tooling and platform changes that were introduced in earlier versions.

Key Benefits of ASP.NET Core MVC

- 1. MVC Architecture

- User interaction with an application that adheres to the MVC pattern follows a natural cycle: the user takes an action, and in response, the application changes its data model and delivers an updated view to the user.
- Web applications necessitate combining several technologies (databases, HTML, and executable code, for example), usually split into a set of tiers or layers.

- 2. Extensibility

- We can easily replace key components with ones of your own implementation

- 3. Tight Control over HTML and HTTP

- 4. Testability

- 5. Powerful Routing System

- 6. Modern API and Cross-Platform

- 7. Open Source

ASP.NET Core in Visual Studio

new ASP.NET Core Web Application - webApplication24

The image shows the Visual Studio interface for creating a new ASP.NET Core Web Application. The left pane displays the 'Create a new ASP.NET Core Web Application' dialog. The 'ASP.NET Core' dropdown is open, showing 'ASP.NET Core 2.1' as the selected option. Below the dropdown, there are icons for 'Empty', 'API', 'Web Application', 'Web Application (Model-View-Controller)', and 'Razor Class Library'. The 'Web Application (Model-View-Controller)' template is highlighted in blue. The right pane shows the configuration options for the selected template. The 'Authentication' section has 'No Authentication' selected. The 'Advanced' section has 'Configure for HTTPS' checked and 'Enable Docker Support' unchecked. The 'OS' dropdown is set to 'Linux'. The 'Create' button is visible at the bottom right.

Create a new ASP.NET Core Web Application

.NET Core ASP.NET Core 2.1 ASP.NET Core 2.1

Empty API Web Application Web Application (Model-View-Controller) Razor Class Library

Angular React.js React.js and Redux

[Get additional project templates](#)

☐ Enable Docker Support (Requires [Docker for Windows](#))

OS: Linux

☐ Configure for HTTPS

Authentication

No Authentication [Change](#)

Advanced

☒ Configure for HTTPS

☐ Enable Docker Support (Requires [Docker Desktop](#))

Linux

Web Application (Model-View-Controller)

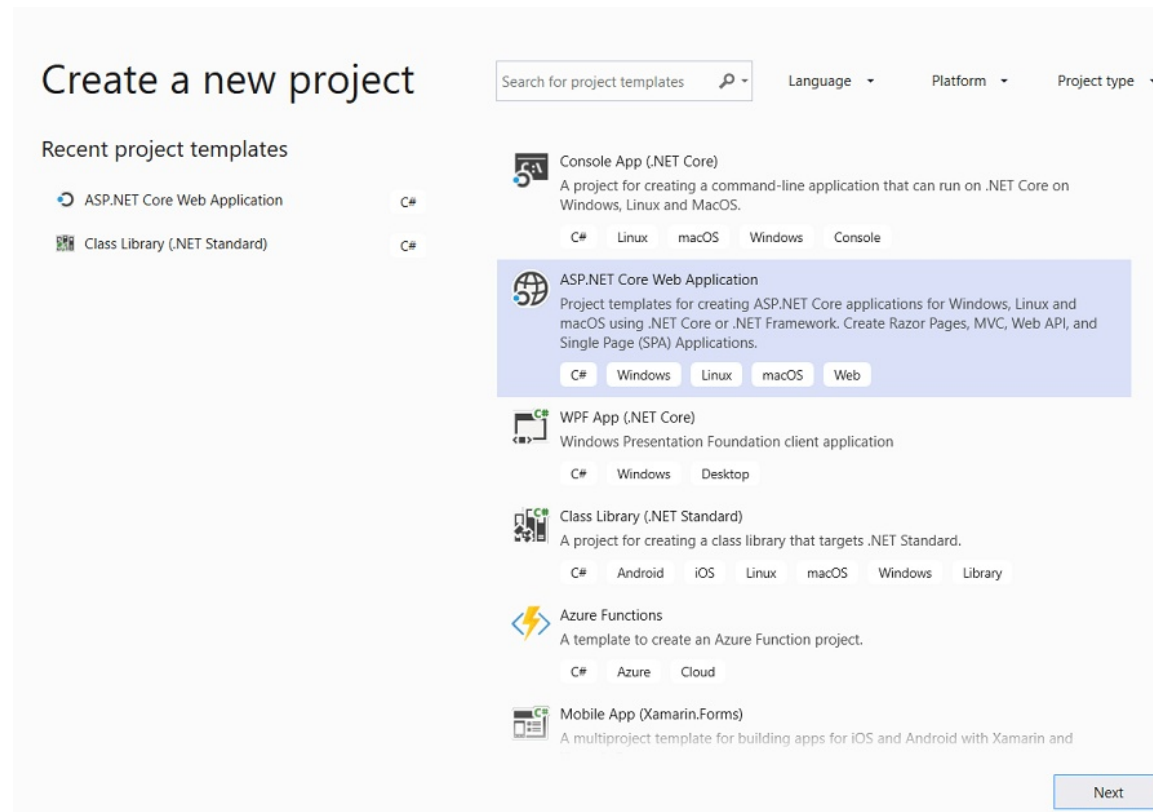
A project template for creating an ASP.NET Core application with example ASP.NET Core MVC Views and Controllers. This template can also be used for RESTful HTTP services.

[Get additional project templates](#)

Author: Microsoft
Source: SDK 3.0.100-preview5-011568

[Back](#) [Create](#)

Practice I: Creating a New ASP.NET Core MVC Project

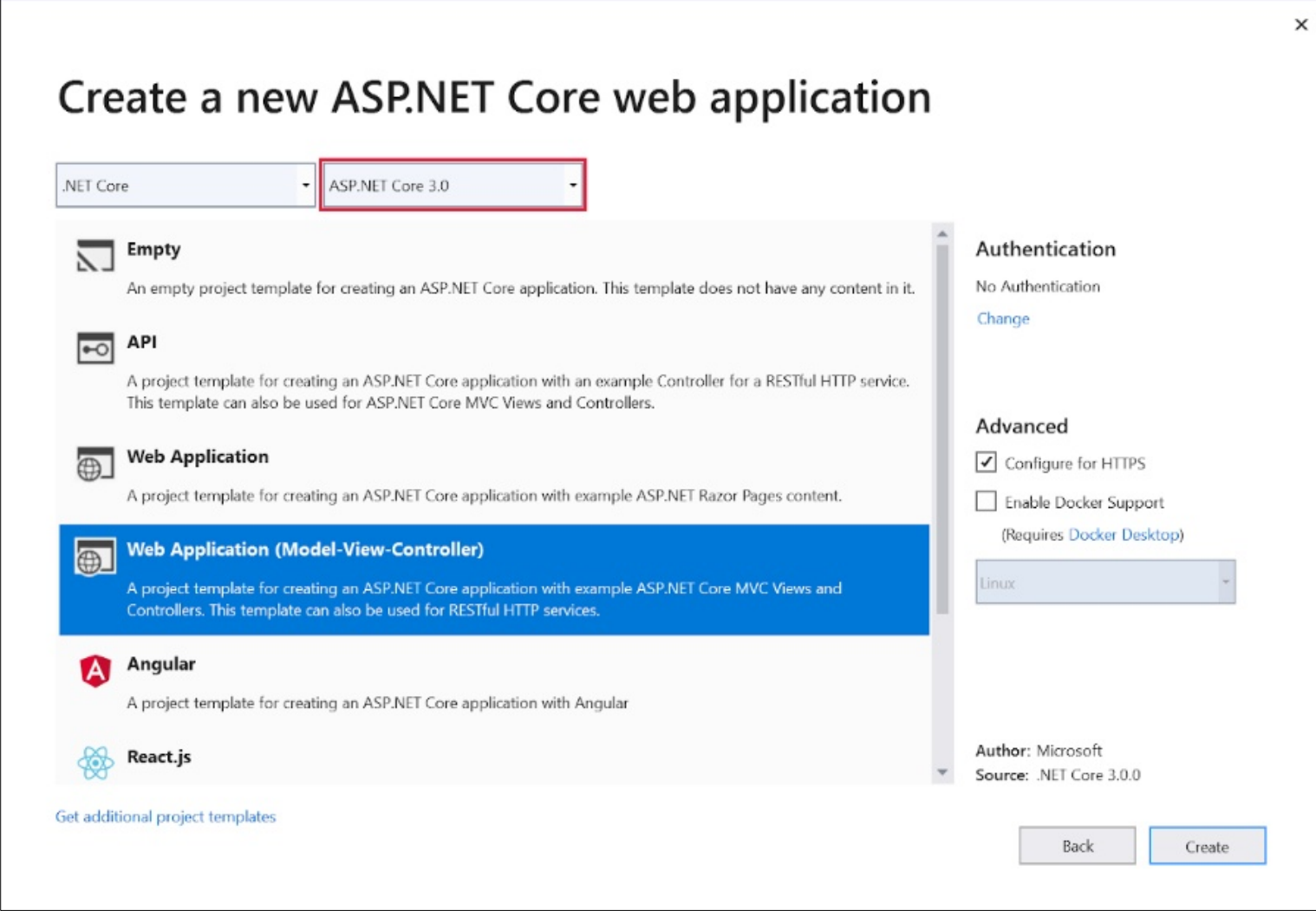


1. From the Visual Studio select **Create a new project**.

2. Select **ASP.NET Core Web Application** and then select **Next**.

3. Name the project **MvcMovie** and select Create. It's important to name the project **MvcMovie** so when you copy code, the namespace will match.

4. Select **Web Application(Model-View-Controller)**, and then select **Create**.



Create a new ASP.NET Core web application

.NET Core ASP.NET Core 3.0

Empty
An empty project template for creating an ASP.NET Core application. This template does not have any content in it.

API
A project template for creating an ASP.NET Core application with an example Controller for a RESTful HTTP service. This template can also be used for ASP.NET Core MVC Views and Controllers.

Web Application
A project template for creating an ASP.NET Core application with example ASP.NET Razor Pages content.

Web Application (Model-View-Controller)
A project template for creating an ASP.NET Core application with example ASP.NET Core MVC Views and Controllers. This template can also be used for RESTful HTTP services.

Angular
A project template for creating an ASP.NET Core application with Angular

React.js

[Get additional project templates](#)

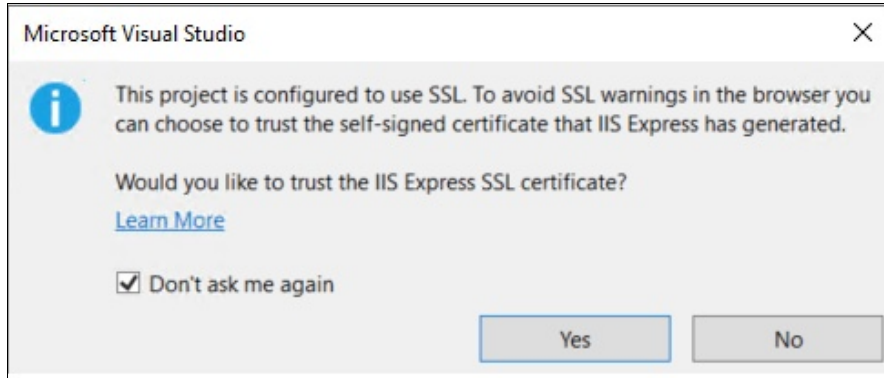
Authentication
No Authentication
[Change](#)

Advanced
☒ Configure for HTTPS
☐ Enable Docker Support
(Requires [Docker Desktop](#))
Linux

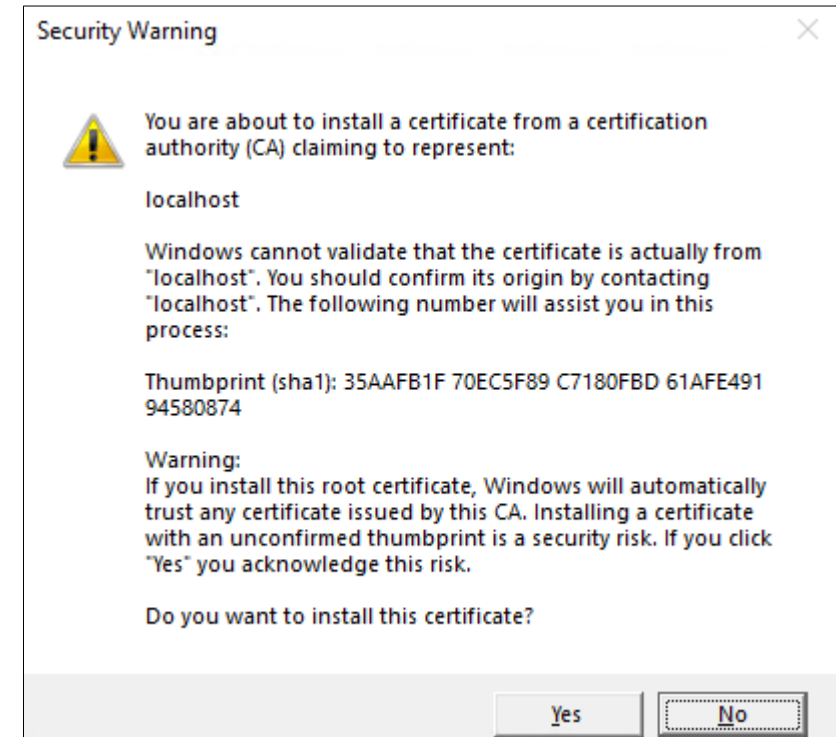
Author: Microsoft
Source: .NET Core 3.0.0

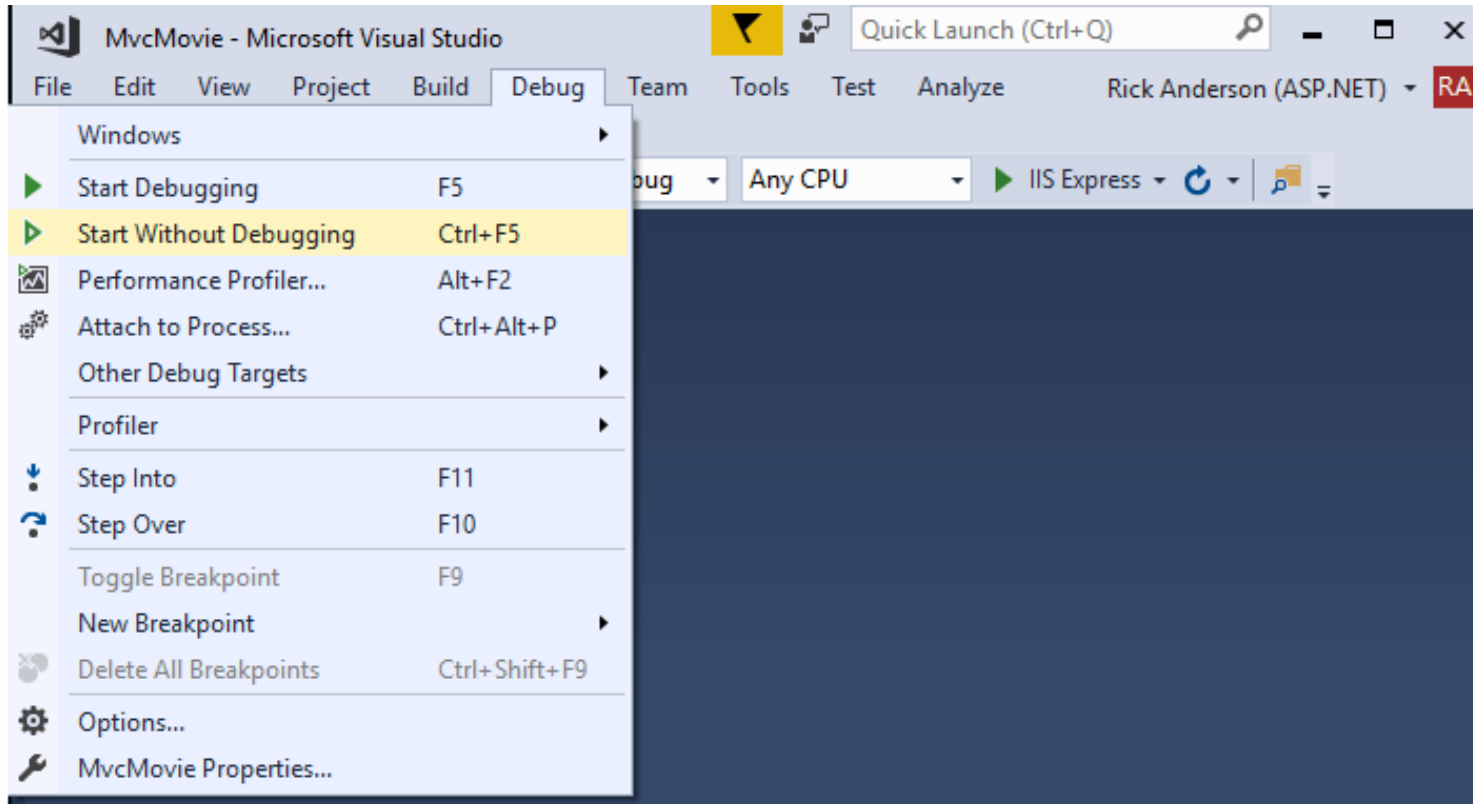
[Back](#) [Create](#)

Select **Ctrl-F5** to run the app in non-debug mode.
Visual Studio displays the following dialog:



Select **Yes** if you trust the IIS Express SSL
certificate.
The following dialog is displayed:

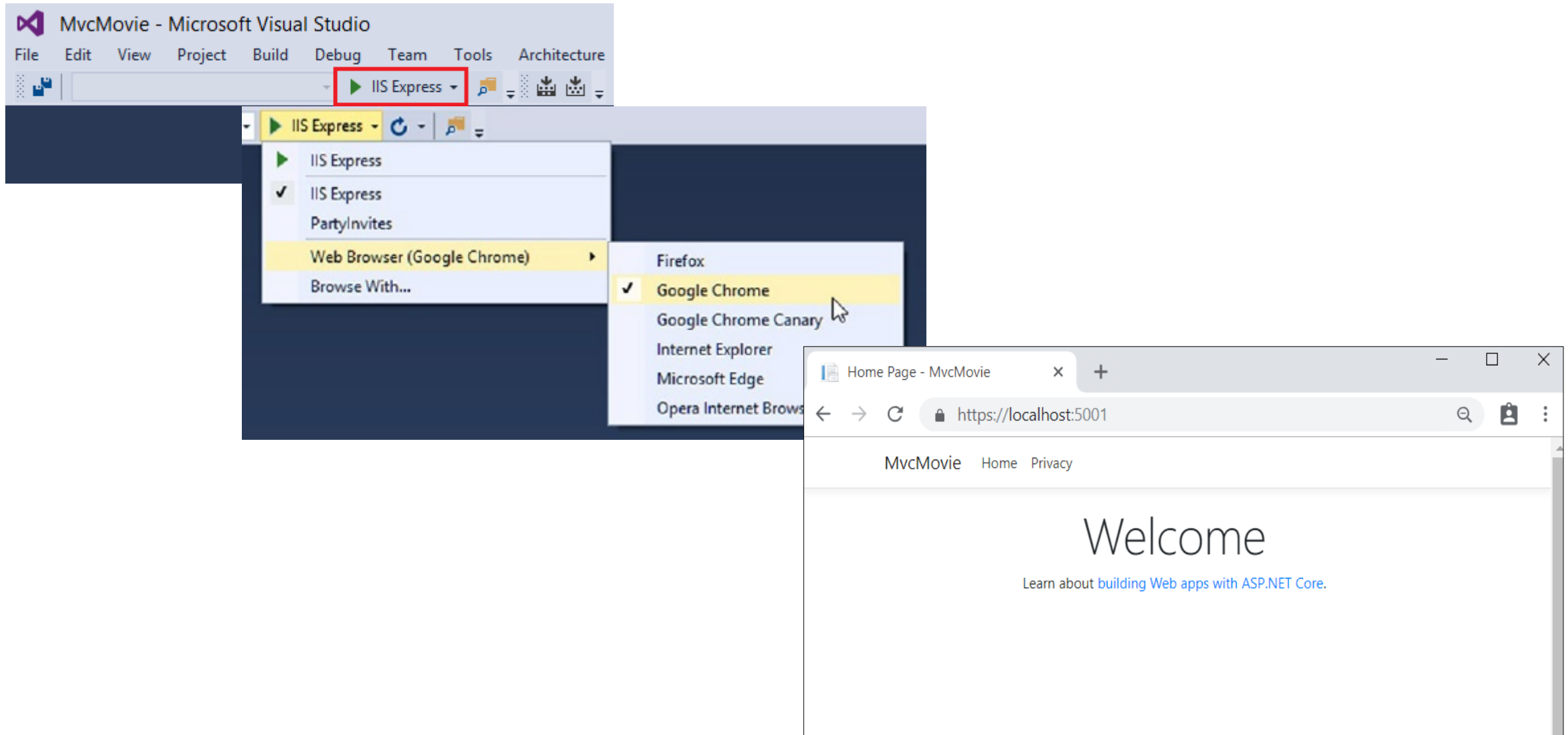




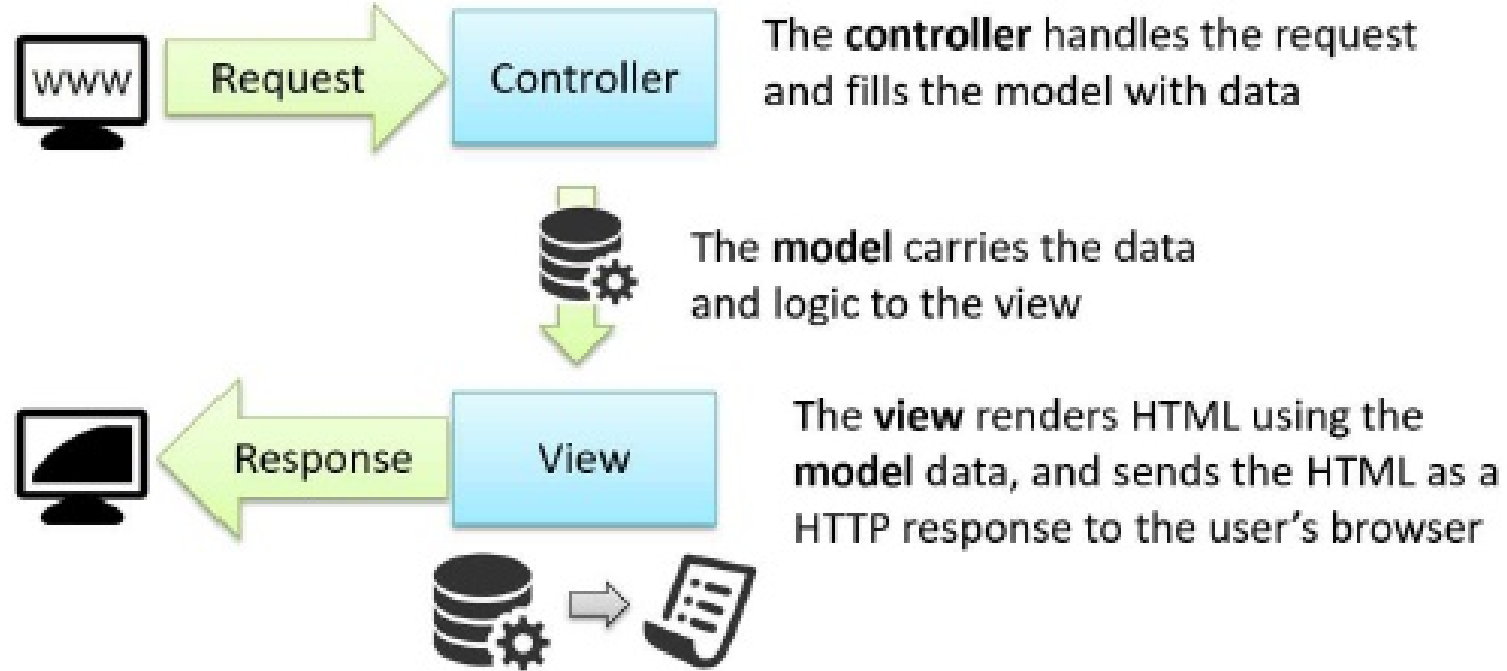
Select **Yes** if you agree to trust the development certificate.

- Visual Studio starts [IIS Express](#) and runs the app. Notice that the address bar shows localhost:port# and not something like example.com. That's because localhost is the standard hostname for your local computer. When Visual Studio creates a web project, a random port is used for the web server.
- Launching the app with Ctrl+F5 (non-debug mode) allows you to make code changes, save the file, refresh the browser, and see the code changes. Many developers prefer to use non-debug mode to quickly launch the app and view changes.
- You can launch the app in debug or non-debug mode from the **Debug** menu item:

You can debug the app by selecting the **IIS Express** button



Understanding the MVC Pattern



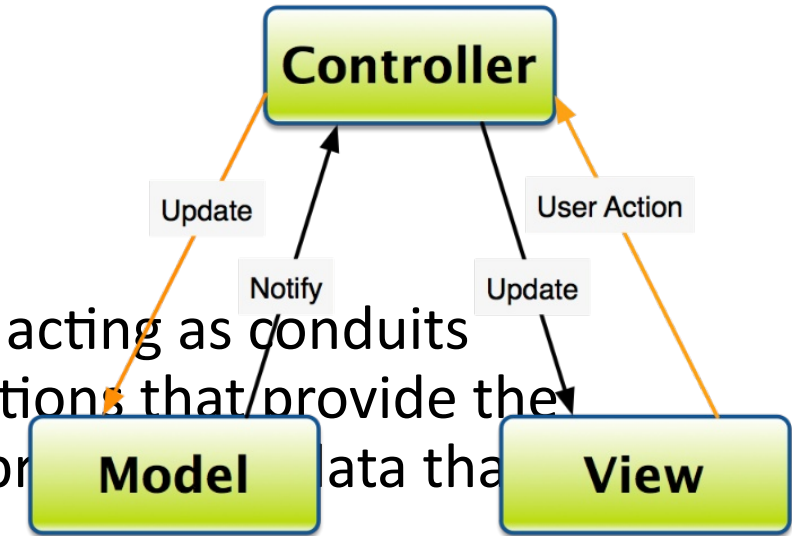
The term *model-view-controller* has been in use since the late 1970s and arose from the Smalltalk project at Xerox PARC, where it was conceived as a way to organize some early GUI applications. Some of the fine detail of the original MVC pattern was tied to Smalltalk-specific concepts, such as *screens* and *tools*, but the broader concepts are still applicable to applications, and they are especially well-suited to web applications.

MVC- Models

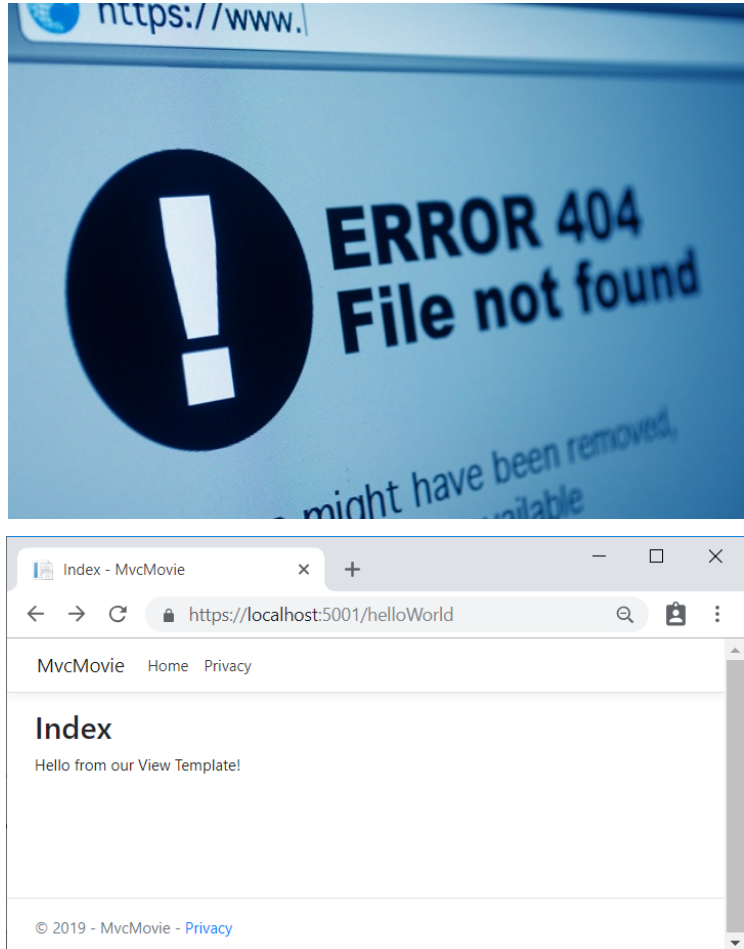
- For each of the components in the MVC pattern, I'll describe what should and should not be included.
- The model in an application built using the MVC pattern *should*
 - Contain the domain data
 - Contain the logic for creating, managing, and modifying the domain data
 - Provide a clean API that exposes the model data and operations on it
- The model *should not*
 - Expose details of how the model data is obtained or managed (in other words, details of the data storage mechanism should not be exposed to controllers and views)
 - Contain logic that transforms the model based on user interaction (because that is the controller's job)
 - Contain logic for displaying data to the user (that is the view's job)

MVC- Controllers

- Controllers are the connective tissue in the MVC pattern, acting as conduits between the data model and views. Controllers define actions that provide the business logic that operates on the data model and that provide data that the views display to the user.
- A controller built using the MVC pattern *should*
 - Contain the actions required to update the model based on user interaction
- The controller *should not*
 - Contain logic that manages the appearance of data (that is the job of the view)
 - Contain logic that manages the persistence of data (that is the job of the model)



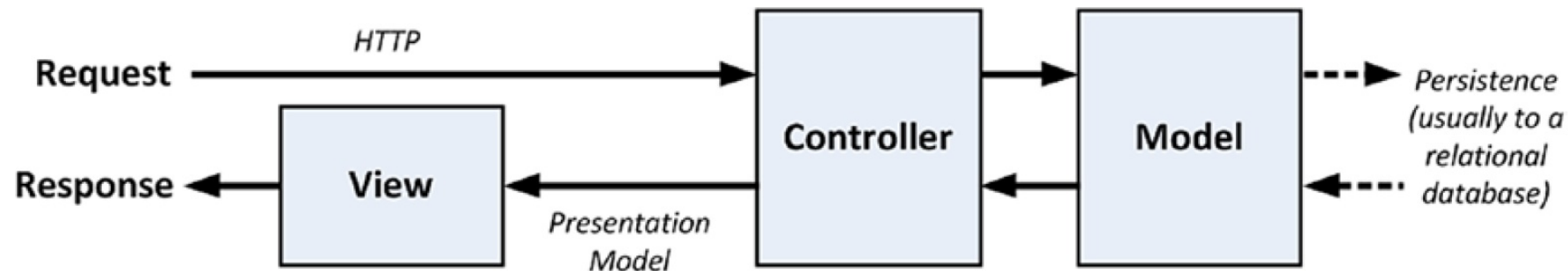
MVC- Views



- Views contain the logic required to display data to the user or to capture data from the user so that it can be processed by a controller action. Views *should*
 - Contain the logic and markup required to present data to the user
- Views *should not*
 - Contain complex logic (this is better placed in a controller)
 - Contain logic that creates, stores, or manipulates the domain model
- Views *can* contain logic, but it should be simple and used sparingly. Putting anything but the simplest method calls or expressions in a view makes the overall application harder to test and maintain.

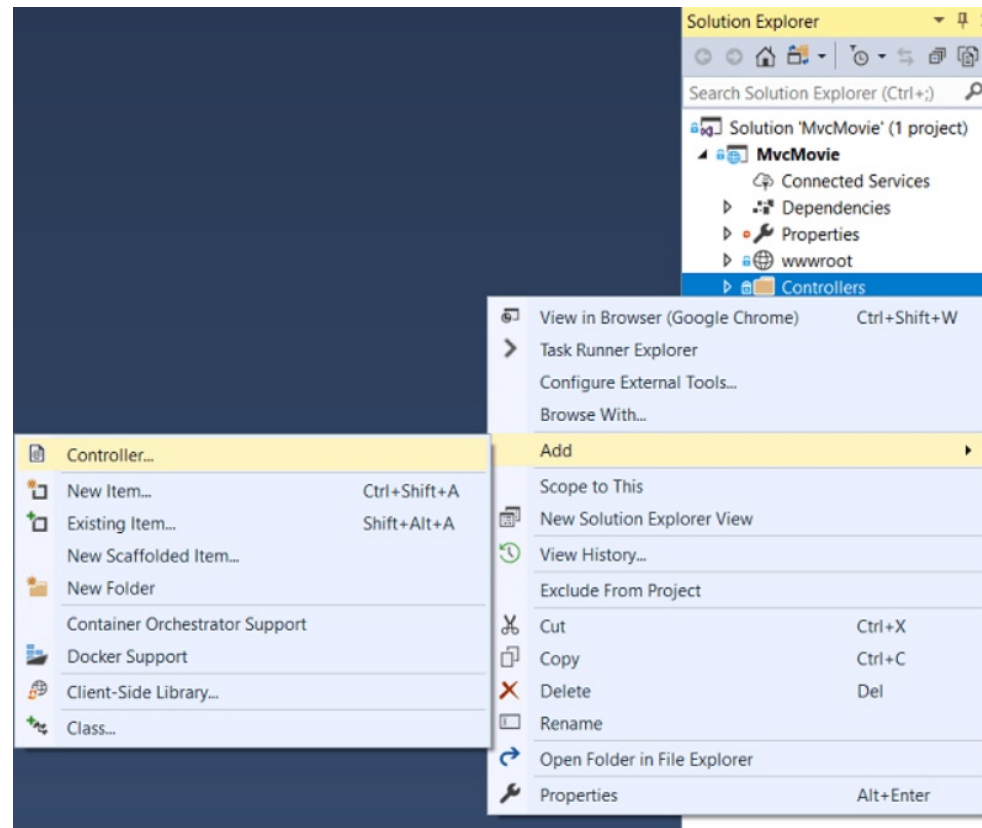
The ASP.NET Implementation of MVC

- In ASP.NET Core MVC, controllers are C# classes, usually derived from the `Microsoft.AspNetCore.Mvc.Controller` class.
- ASP.NET Core MVC uses a view engine, known as Razor, which is the component responsible for processing a view in order to generate a response for the browser.
- ASP.NET Core MVC doesn't apply any constraints on the implementation of your domain model. You can create a model using regular C# objects and implement persistence using any of the databases, object-relational mapping frameworks, or other data tools supported by .NET.

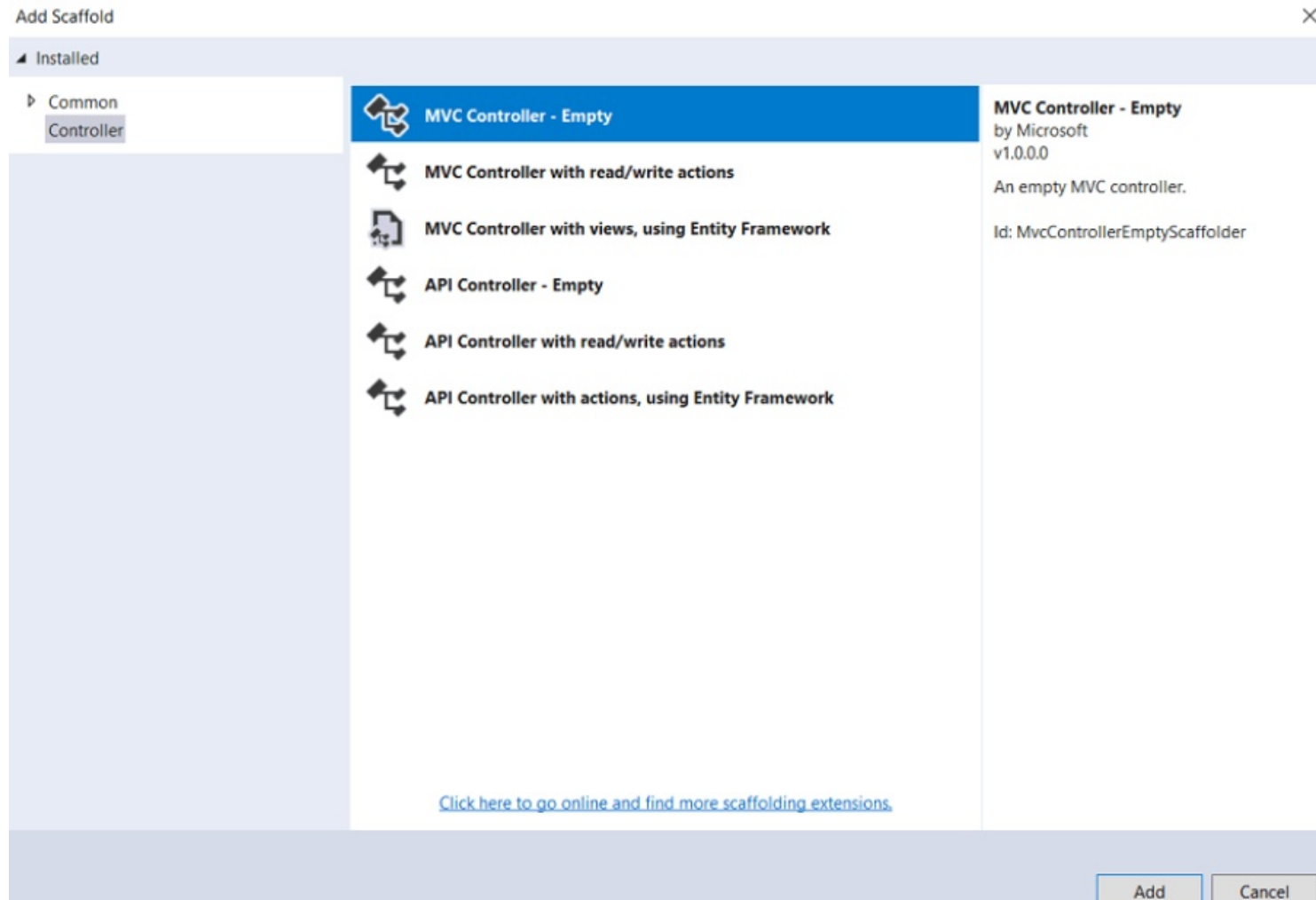


Practice II: Add a controller to an ASP.NET Core MVC app

- Following Practice I: In **Solution Explorer**, right-click **Controllers** > **Add** > **Controller**



- In the **Add Scaffold** dialog box, select **MVC Controller – Empty**
- In the **Add Empty MVC Controller** dialog, enter **HelloWorldController** and select **ADD**



- Replace the contents of *Controllers/HelloWorldController.cs* with the following:

```
using Microsoft.AspNetCore.Mvc;
using System.Text.Encodings.Web;

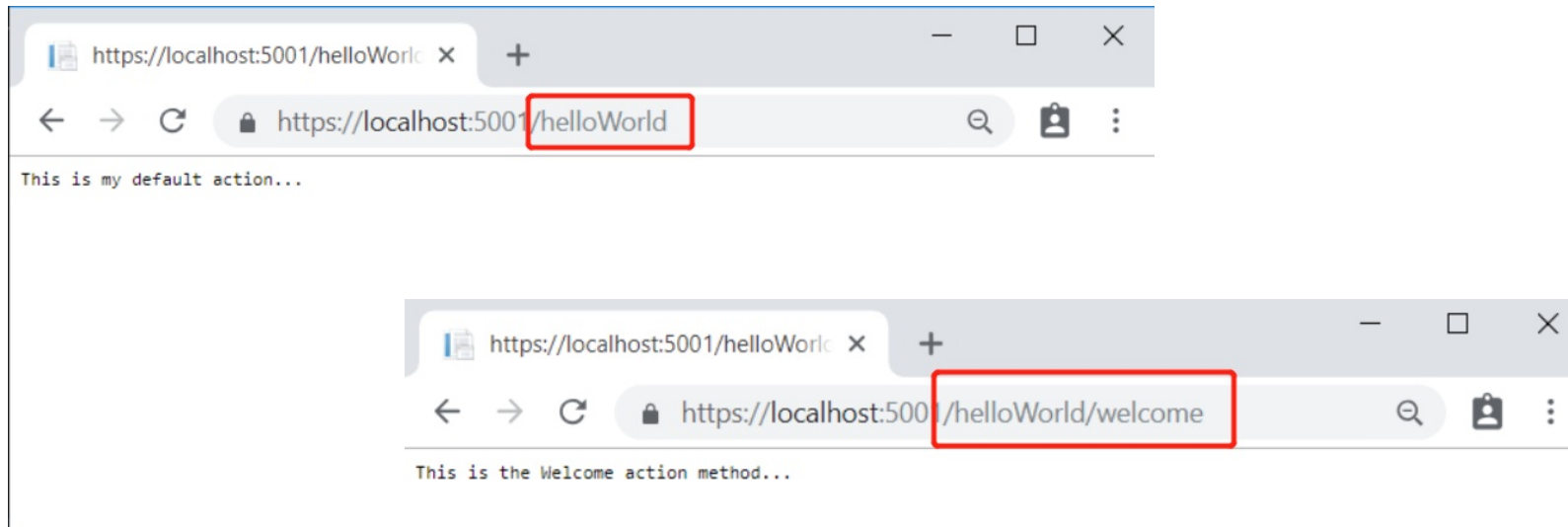
namespace MvcMovie.Controllers
{
    public class HelloWorldController : Controller
    {
        //
        // GET: /HelloWorld/

        public string Index()
        {
            return "This is my default action...";
        }

        //
        // GET: /HelloWorld/Welcome/

        public string Welcome()
        {
            return "This is the Welcome action method...";
        }
    }
}
```

Run the app in non-debug mode and append "HelloWorld" to the path in the address bar. The Index method returns a string.



The routing format is set in the `Configure` method in `Startup.cs` file.

```
C# Copy  
  
app.UseEndpoints(endpoints =>  
{  
    endpoints.MapControllerRoute(  
        name: "default",  
        pattern: "{controller=Home}/{action=Index}/{id?}");  
});
```

When you browse to the app and don't supply any URL segments, it defaults to the "Home" controller and the "Index" method specified in the template line highlighted above.

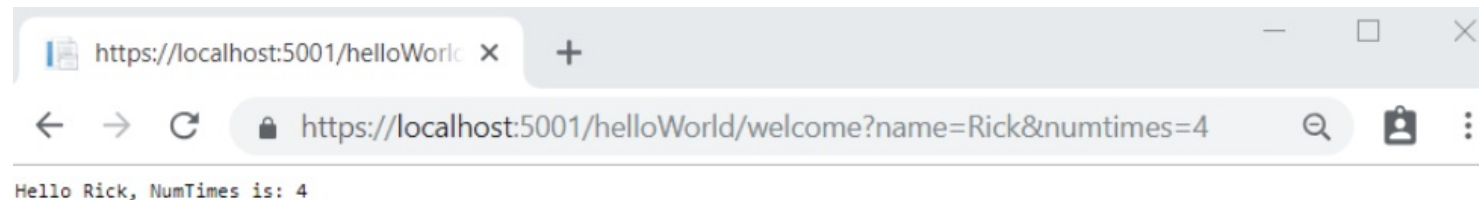
Modify the code to pass some parameter information from the URL to the controller. For example, */HelloWorld/Welcome?name=Rick&numtimes=4*. Change the Welcome method to include two parameters as shown in the following code.

```
// GET: /HelloWorld/Welcome/  
// Requires using System.Text.Encodings.Web;  
public string Welcome(string name, int numTimes = 1)  
{  
    return HtmlEncoder.Default.Encode($"Hello {name}, NumTimes is: {numTimes}");  
}
```

Run the app and browse to:

<https://localhost:{PORT}/HelloWorld/Welcome?name=Rick&numtimes=4>

** Replace {PORT} with your port number.*



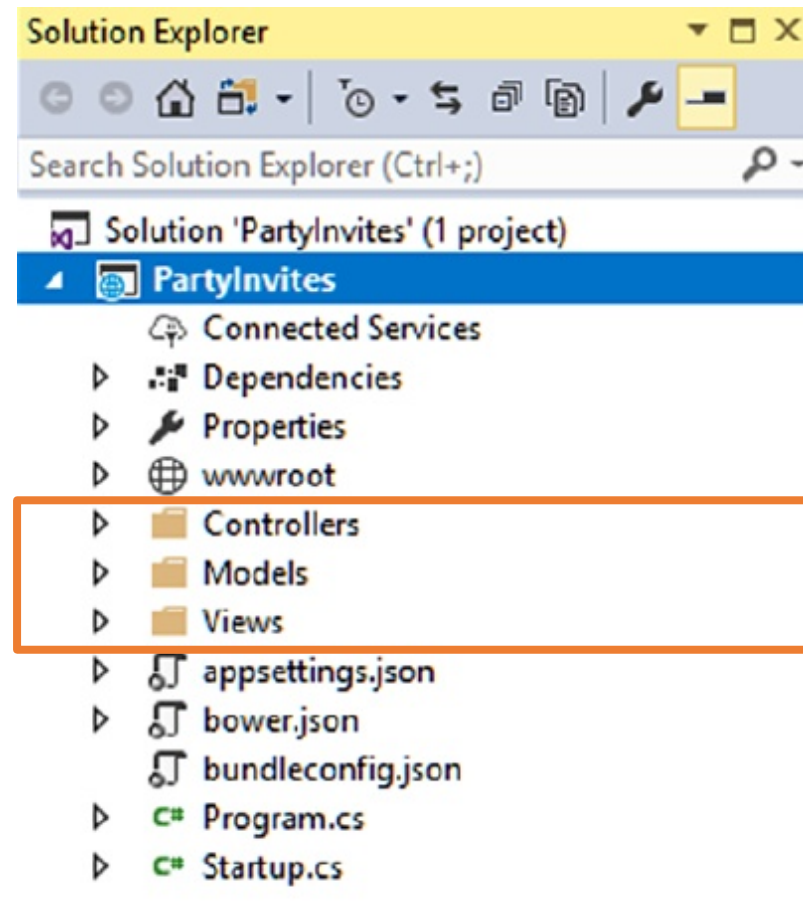
- Try to Replace 'Welcome' with the following code

```
public string Welcome(string name, int ID = 1)
{
    return HtmlEncoder.Default.Encode($"Hello {name}, ID: {ID}");
}
```

Run the app and enter the following URL: <https://localhost:{PORT}/HelloWorld/Welcome/3?name=Rick>

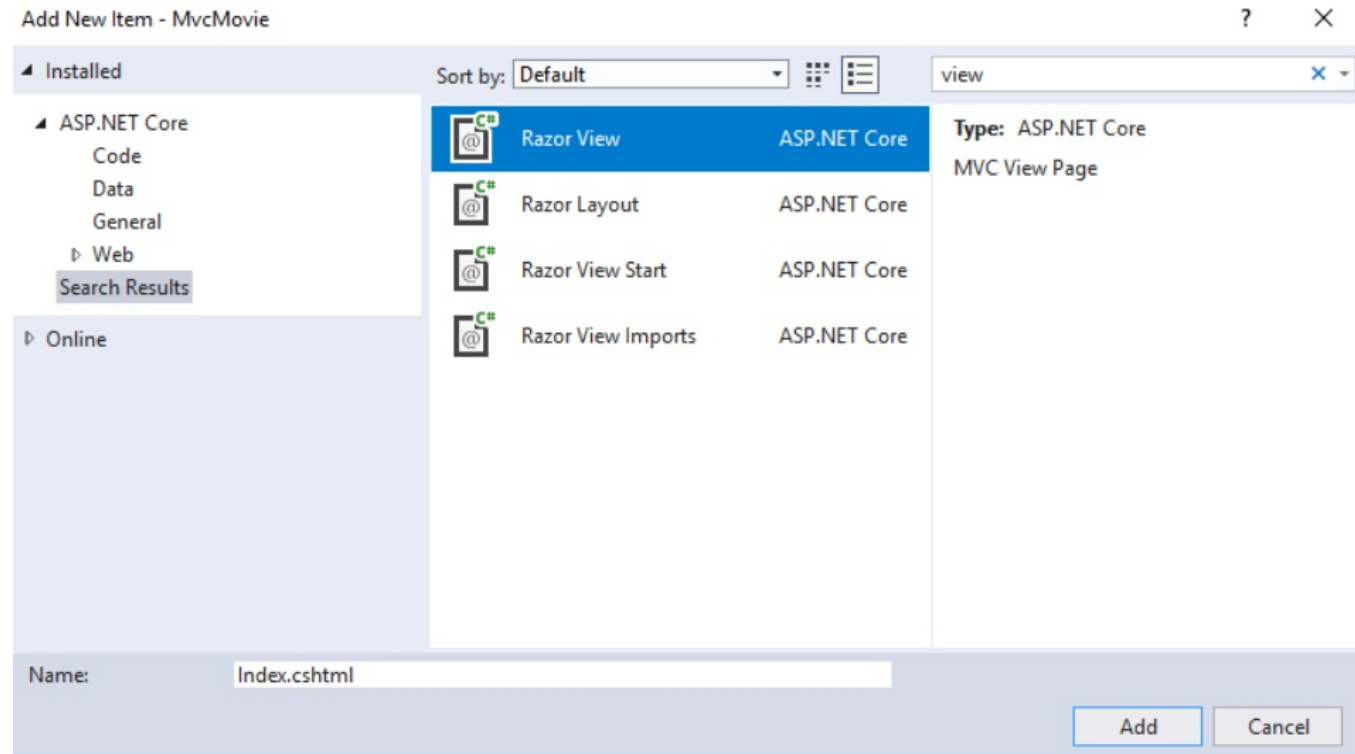
This time the third URL segment matched the route parameter id. The Welcome method contains a parameter id that matched the URL template in the MapControllerRoute method. The trailing ? (in id?) indicates the id parameter is optional.

Structure of a typical MVC Project



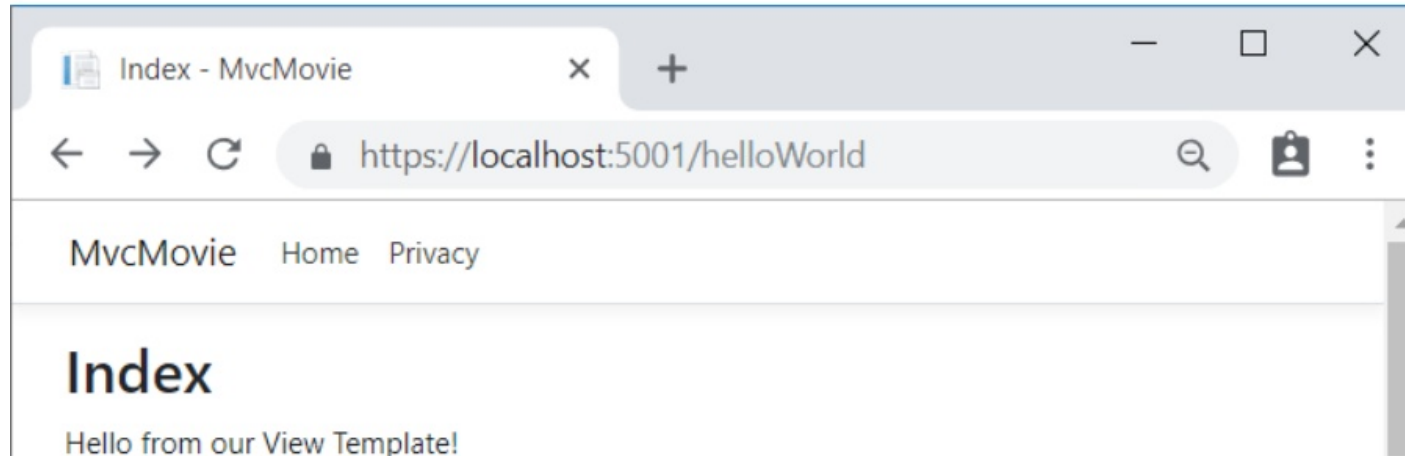
Rendering Web Pages

- The output from the previous example wasn't HTML—it was just the string Hello World. To produce an HTML response to a browser request, we need a *view*, which tells MVC how to generate a response to a request from a browser.
- Right click on the *Views* folder, and then **Add > New Folder** and name the folder *HelloWorld*.
- Right click on the *Views/HelloWorld* folder, and then **Add > New Item**.
- In the **Add New Item - MvcMovie** dialog
 - In the search box in the upper-right, enter *view*
 - Select **Razor View**
 - Keep the **Name** box value, *Index.cshtml*.
 - Select **Add**



Replace the contents of the *Views/HelloWorld/Index.cshtml* Razor view file with the following:

```
@{  
    ViewData["Title"] = "Index";  
}  
<h2>Index</h2>  
<p>Hello from our View Template!</p>
```

- Navigate to `https://localhost:{PORT}/HelloWorld`. The Index method in the HelloWorldController didn't do much; it ran the statement `return View();`, which specified that the method should use a view template file to render a response to the browser. Because a view template file name wasn't specified, MVC defaulted to using the default view file.
- The default view file has the same name as the method (Index), so in the `/Views/HelloWorld/Index.cshtml` is used. The image above shows the string "Hello from our View Template!" hard-coded in the view.

Change the title, footer, and menu link in the layout file

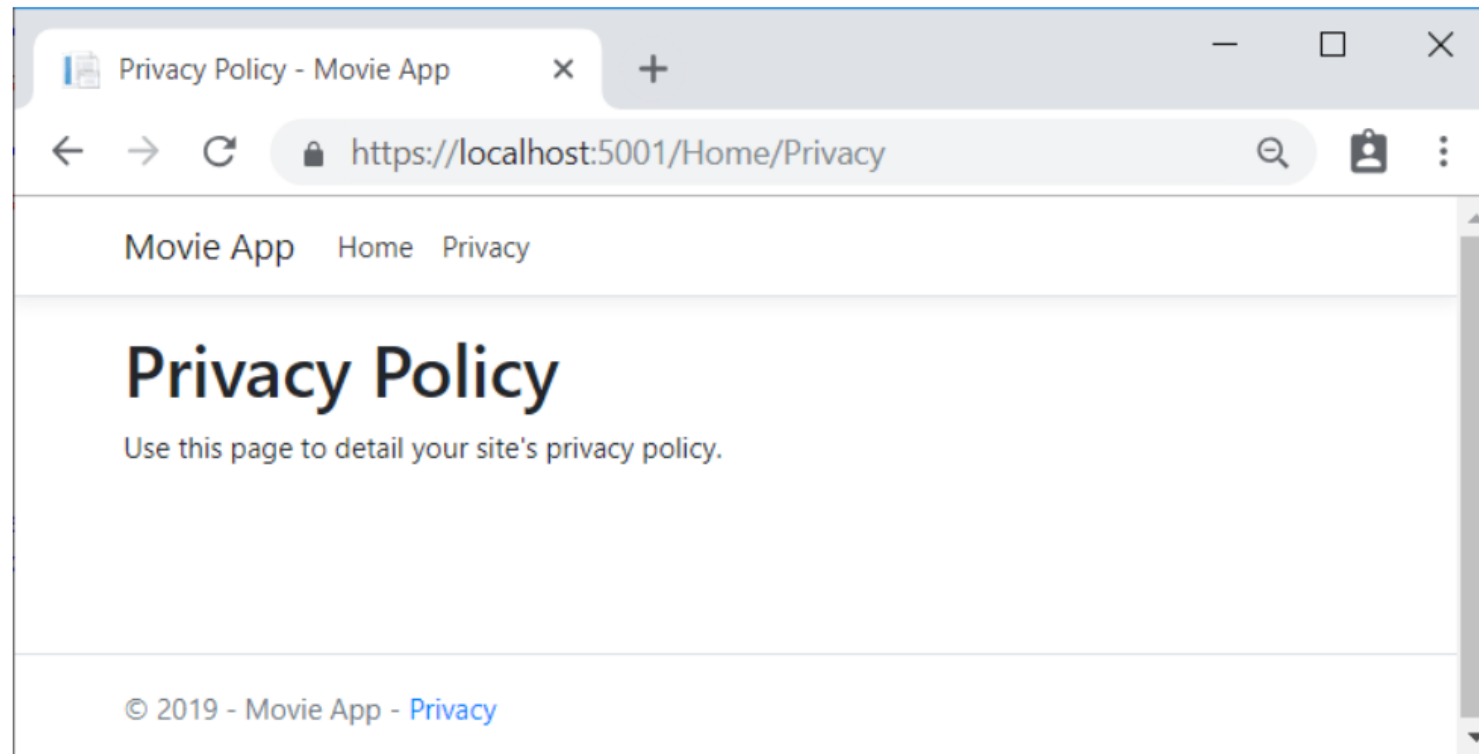
```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="utf-8" />
  <meta name="viewport" content="width=device-width, initial-
scale=1.0" />
  <title>@ViewData["Title"] - Movie App</title>
  <link rel="stylesheet"
href="~/lib/bootstrap/dist/css/bootstrap.css" />
  <link rel="stylesheet" href="~/css/site.css" />
</head>
<body>
  <header>
    <nav class="navbar navbar-expand-sm navbar-toggleable-sm
navbar-light bg-white border-bottom box-shadow mb-3">
      <div class="container">
        <a class="navbar-brand" asp-controller="Movies" asp-
action="Index">Movie App</a>
        <button class="navbar-toggler" type="button" data-
toggle="collapse" data-target=".navbar-collapse" aria-
controls="navbarSupportedContent"
          aria-expanded="false" aria-label="Toggle navigation">
          <span class="navbar-toggler-icon"></span>
        </button>
        <div class="navbar-collapse collapse d-sm-inline-flex flex-sm-
row-reverse">
          <ul class="navbar-nav flex-grow-1">
```

```
<li class="nav-item">
  <a class="nav-link text-dark" asp-area="" asp-
controller="Home" asp-action="Index">Home</a>
</li>
<li class="nav-item">
  <a class="nav-link text-dark" asp-area="" asp-
controller="Home" asp-action="Privacy">Privacy</a>
</li>
</ul>
</div>
</nav>
</header>
<div class="container">
  <main role="main" class="pb-3">
    @RenderBody()
  </main>
</div>

<footer class="border-top footer text-muted">
  <div class="container">
    &copy; 2019 - Movie App - <a asp-area="" asp-
controller="Home" asp-action="Privacy">Privacy</a>
  </div>
</footer>
<script src="~/lib/jquery/dist/jquery.js"></script>
<script src="~/lib/bootstrap/dist/js/bootstrap.bundle.js"></script>
<script src="~/js/site.js" asp-append-version="true"></script>
@RenderSection("Scripts", required: false)
</body>
</html>
```

Replace the content of the *Views/Shared/_Layout.cshtml* file with the following markup. The changes are highlighted:

Save your changes and select the **Privacy** link. Notice how the title on the browser tab displays **Privacy Policy - Movie App** instead of **Privacy Policy - Mvc Movie**:



A Simple Data-Entry Application

- Add a data model class

Right-click the Models folder > Add > Class. Name the file Movie.cs.

Update the *Movie.cs* file with the following code:

```
using System;
using System.ComponentModel.DataAnnotations;
namespace MvcMovie.Models {
    public class Movie {
        public int Id { get; set; }
        public string Title { get; set; }
        [DataType(DataType.Date)]
        public DateTime ReleaseDate { get; set; }
        public string Genre { get; set; }
        public decimal Price { get; set; }
    }
}
```

Lab Exercise: Add a model to an ASP.NET Core MVC app

- Follow the case at the link below:

<https://docs.microsoft.com/en-us/aspnet/core/tutorials/first-mvc-app/adding-model?view=aspnetcore-3.1&tabs=visual-studio>

Software Development Tools & Practices

PUSL2020

Level 2

“Validation”

Dr. Rasika Ranaweera (ranaweera.r@nsbm.ac.lk | +94 11 544 6126)

Faculty of Computing | NSBM Green University

What is form validation?

validation: ensuring that form's values are correct

some types of validation:

- preventing blank values (email address)

- ensuring the type of values

- integer, real number, currency, phone number, Social Security number, postal address, email address, date, credit card number, ...

- ensuring the format and range of values (ZIP code must be a 5-digit integer)

- ensuring that values fit together (user types email twice, and the two must match)

Become a Registered User!

Get access to WeightWatchers.com's buzzing Community:



Start a blog



Participate in a Challenge



Save your favorite recipes



Post to our boards



Important message: Please review and correct the information below.

Create Your Registered User Account Login

First name:

Last name:



Please enter your last name. It's required information.

Your birthdate:



Please select your month, day, and year of birth. It's required information.

Your gender: ☐ Female ☐ Male



Please enter your gender. It's required information.

State:



Please choose a state. It's required information.

Zip code



Please enter a 5-digit ZIP code. It's required information.

E-mail:



Please enter your email address in the following format: abc@example.com. It's required information.

Re-enter e-mail:

Client vs. server-side validation

Validation can be performed:

client-side (before the form is submitted)

can lead to a better user experience, but not secure (why not?)

server-side (in PHP code, after the form is submitted)

needed for truly secure validation, but slower

both

best mix of convenience and security, but requires most effort to program

An example form to be validated

```
<form action="http://foo.com/foo.php" method="get">
  <div>
    City: <input name="city" /> <br />
    State: <input name="state" size="2" maxlength="2" />
  <br />
    ZIP: <input name="zip" size="5" maxlength="5" /> <br />
    <input type="submit" />
  </div>
</form>
```

HTML

Let's validate this form's data on the server...

Basic server-side validation code- PHP

```
<?php
$city = $_REQUEST["city"];
$state = $_REQUEST["state"];
$zip = $_REQUEST["zip"];
if (!$city || strlen($state) != 2 || strlen($zip) != 5) {
?>
    <h2>Error, invalid city/state submitted.</h2>
<?php
}
?>
```

PHP

basic idea: examine parameter values, and if they are bad, show an error message and abort

Basic server-side validation code

validation code can take a lot of time / lines to write

How do you test for integers vs. real numbers vs. strings?

How do you test for a valid credit card number?

How do you test that a person's name has a middle initial?

How do you test whether a given string matches a particular complex format?

Basic Regular Expression

```
/abc/
```

in PHP, regexes are strings that begin and end with /

the simplest regexes simply match a particular substring

the above regular expression matches any string containing "abc":

YES: "abc", "abcdef", "defabc", " .=.abc.=.", ...

NO: "fedcba", "ab c", "PHP", ...

Wildcards

A dot `.` matches any character except a `\n` line break

`"/.oo.y/"` matches `"Doocy"`, `"goofy"`, `"LooNy"`, ...

A trailing `i` at the end of a regex (after the closing `/`) signifies a case-insensitive match

`"/xen/i"` matches `"Xenia"`, `"xenophobic"`, `"Xena the warrior princess"`, `"XEN technologies"` ...

Special characters: |, (), ^, \

| means *OR*

`"/abc|def|g/"` matches "abc", "def", or "g"

There's no *AND* symbol. Why not?

() are for grouping

`"/(Homer|Marge) Simpson/"` matches "Homer Simpson" or "Marge Simpson"

^ matches the beginning of a line; \$ the end

`"/^<!--$/"` matches a line that consists entirely of "<!--"

Special characters: |, (), ^, \

\ starts an escape sequence

many characters must be escaped to match them literally: / \ \$

. [] () ^ * + ?

"/<br \>/" matches lines containing
 tags

Quantifiers: *, +, ?

* means 0 or more occurrences

`/abc*/` matches `"ab"`, `"abc"`, `"abcc"`, `"abccc"`, ...

`/a(bc)*/` matches `"a"`, `"abc"`, `"abcbc"`, `"abcbcbc"`, ...

`/a.*a/` matches `"aa"`, `"aba"`, `"a8qa"`, `"a!?!_a"`, ...

+ means 1 or more occurrences

`/a(bc)+/` matches `"abc"`, `"abcbc"`, `"abcbcbc"`, ...

`/Goo+gle/` matches `"Google"`, `"Goooogle"`, `"Goooooogle"`, ...

? means 0 or 1 occurrences

`/a(bc)?/` matches `"a"` or `"abc"`

More quantifiers: {min,max}

$\{\text{min}, \text{max}\}$ means between min and max occurrences (inclusive)

`/a(bc){2,4}/` matches "abcbcb", "abcbcbcb", or "abcbcbcbcb"

min or max may be omitted to specify any number

{2,} means 2 or more

$\{,6\}$ means up to 6

$\{3\}$ means exactly 3

Character sets: []

[] group characters into a character set; will match any single character from the set

`"/[bcd]art/"` matches strings containing "bart", "cart", and "dart"
equivalent to `"/(b|c|d)art/"` but shorter

inside [], many of the modifier keys act as normal characters

`"/what[!*?]*/"` matches "what", "what!", "what?*!*!", "what??*!?",

Character ranges: [start-end]

inside a character set, specify a range of characters with-

`"/[a-z]/"` matches any lowercase letter

`"/[a-zA-Z0-9]/"` matches any lower- or uppercase letter or digit

an initial `^` inside a character set negates it

`"/[^abcd]/"` matches any character other than a, b, c, or d

Character ranges: [start-end]

inside a character set, - must be escaped to be matched

`"/[+\\-]?[0-9]+/"` matches an optional + or -, followed by at least one digit

Escape sequences

special escape sequence character sets:

`\d` matches any digit (same as `[0-9]`); `\D` any non-digit (`[^0-9]`)

`\w` matches any “word character” (same as `[a-zA-Z_0-9]`); `\W` any non-word

char

`\s` matches any whitespace character (, `\t`, `\n`, etc.); `\S` any non-whitespace

Regular expressions in PHP

regex syntax: strings that begin and end with /, such as `"/[AEIOU]+/"`

<code>preg_match(regex, string)</code>	returns TRUE if string matches regex
<code>preg_replace(regex, replacement, string)</code>	returns a new string with all substrings that match regex replaced by replacement
<code>preg_split(regex, string)</code>	returns an array of strings from given string broken apart using the given regex as the delimiter (similar to explode but more powerful)

Regular expressions example

```
echo preg_match ('/test/', "a test of preg_match");  
echo preg_match ('/tutorial/', "a test of preg_match");  
  
$matchesarray[0] = "http://www.tipsntutorials.com/"  
$matchesarray[1] = "http://"   
$matchesarray[2] = "www.tipsntutorials.com/"  
preg_match ('/(http://)(.*)/', $matchesarray)
```

PHP

Regular expressions example

```
# replace vowels with stars
$str = "stay hungry stay foolish";
$str = preg_replace("/[aeiou]/", "*", $str);
#

# break apart into words
$words = preg_split("/[ ]+/", $str);
#

# capitalize words that had 2+ consecutive vowels
for ($i = 0; $i < count($words); $i++) {
    if (preg_match("/\\{2,}/", $words[$i])) {
        $words[$i] = strtoupper($words[$i]);
    }
}

} #
```

PHP

Regular expressions example

```
# replace vowels with stars
$str = "the quick brown fox";
$str = preg_replace("/[aeiou]/", "*", $str);
# "th* q**ck br*wn f*x"

# break apart into words
$words = preg_split("/[ ]+/", $str);
# ("th*", "q**ck", "br*wn", "f*x")

# capitalize words that had 2+ consecutive vowels
for ($i = 0; $i < count($words); $i++) {
    if (preg_match("/\\{2,}/", $words[$i])) {
        $words[$i] = strtoupper($words[$i]);
    }
}

} # ("th*", "Q**CK", "br*wn", "f*x")
```

PHP

Why validate?

❑ XSS (Cross Site Scripting)

Code	Sample
Application code	<code><input value="userInput"></code>
Malicious string	<code>"><script>...</script><input value="</code>
Resulting code	<code><input value=""><script>...</script><input value=""></code>

❑ SQLI (SQL Injection)

Code	Sample
Application code	<code>SELECT * FROM Users WHERE Username='\$username' AND Password='\$password'</code>
Malicious string	<code>\$username = 1' or '1' = '1</code>
Resulting code	<code>SELECT * FROM Users WHERE Username='1' OR '1' = '1' AND Password='1' OR '1' = '1'</code>

Software Development Tools & Practices

PUSL2020

Level 2

“API Development + Revision”

Dr. Rasika Ranaweera (ranaweera.r@nsbm.ac.lk | +94 11 544 6126)

Faculty of Computing | NSBM Green University

API

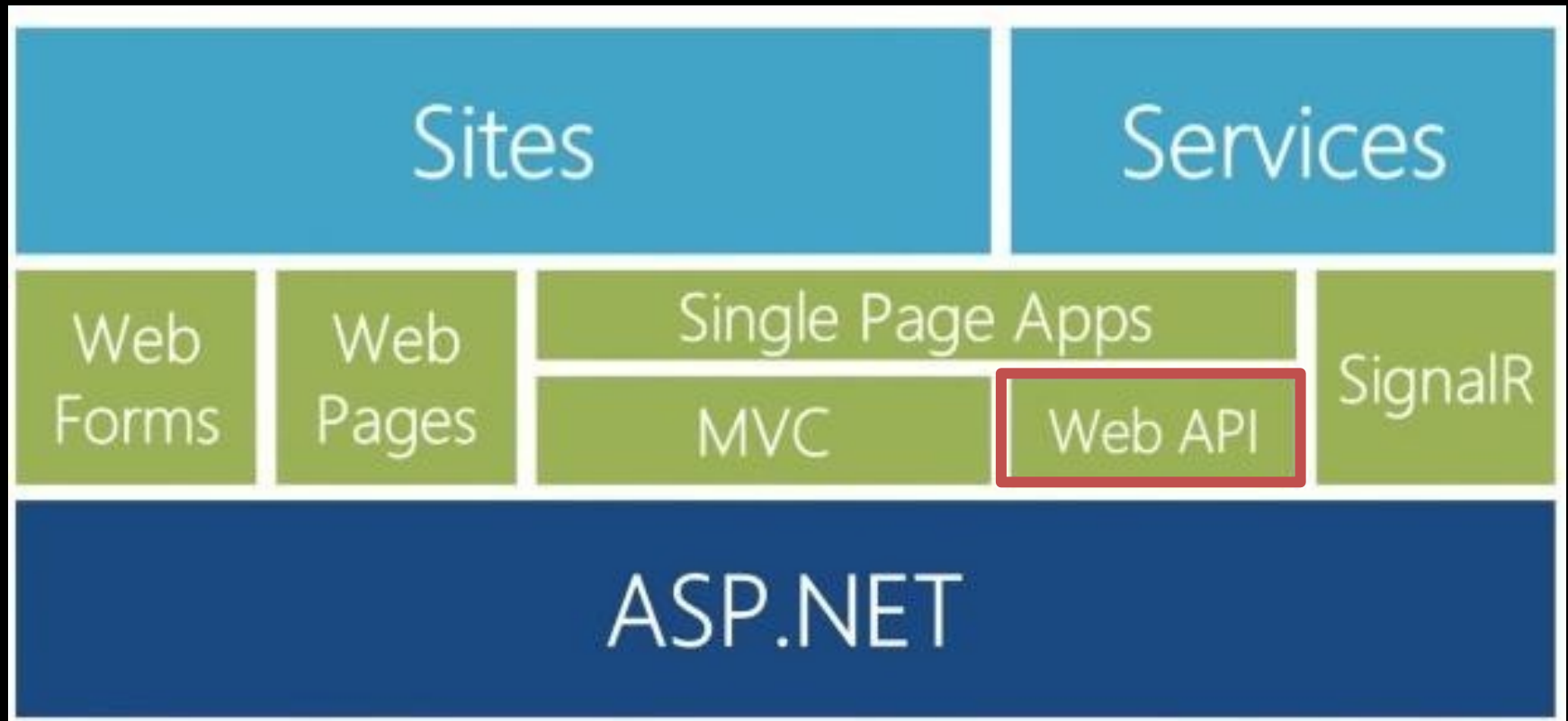
- Application Programming Interface
- Source code interface
 - For library or OS
 - Provides services to a program
- At its base, like a header file
 - But, more complete

```
#include <iostream>
#include <string>

using namespace std;

void prompt(string name[3], int grade[3]);
void init(int index[3]);
void sort(string name[3], int index[3]);
void output(string name[3], int grade[3], int index[3]);
```

ASP.NET



**ASP.NET WEB
API**

A Framework for creating
HTTP Services that can reach
a broad range of clients
including browsers and
mobile devices

What?

- A fully supported and extensible framework for building HTTP based endpoints
- Built on the top of ASP.NET
 - Mostly ASP.NET Routing
- Released with ASP.NET MVC4
 - Not linked to MVC – you can use alone, with MVC4 or you can use with ASP.NET web forms
- Also includes a new HTTP Client

Introduction, Routing, Attribute Routing

Attribute Routing

- Convention-based routing makes it hard to support certain URI patterns that are common in RESTful APIs.
- E.g. Customers have orders, movies have actors, books have authors
- It's natural to create URIs that reflect these relations
 - **/customers/1/orders**
- Attribute routing, is trivial to define a route for this URL.
- Simply by adding an [attribute] to the controller action

```
[Route("customers/{customerId}/orders")]  
public IEnumerable<Order> GetOrdersByCustomer(int customerId) { ... }
```

Override the route

prefix

Use a tilde (~) on the method attribute to override the route prefix:

```
[RoutePrefix("api/books")]
public class BooksController : ApiController
{
    // GET /api/authors/1/books
    [Route("~/api/authors/{authorId:int}/books")]
    public IEnumerable<Book> GetByAuthor(int authorId) { ... }

    // ...
}
```

Route prefix parameters

route prefix can include parameters:

```
[RoutePrefix("customers/{customerId}")]
public class OrdersController : ApiController
{
    // GET customers/1/orders
    [Route("orders")]
    public IEnumerable<Order> Get(int customerId) { ... }
}
```

Route constraints

Route constraints let you restrict how the parameters in the route template are matched. The general syntax is "{parameter:constraint}"

```
[Route("users/{id:int}")]  
public User GetById(int id) { ... }
```

```
[Route("users/{name}")]  
public User GetByName(string name) { ... }
```

Optional URI Parameters and Default Values

Make a URI parameter optional by adding a question mark to the route parameter. If a route parameter is optional, you must define a default value for the method parameter.

```
public class BooksController : ApiController
{
    [Route("api/books/locale/{lcid:int?}")]
    public IEnumerable<Book> GetBooksByLocale(int lcid = 1033) { ... }
}
```

[OR]

```
public class BooksController : ApiController
{
    [Route("api/books/locale/{lcid:int=1033}")]
    public IEnumerable<Book> GetBooksByLocale(int lcid) { ... }
}
```

Supported constraints

Constraint	Description	Example
alpha	Matches uppercase or lowercase Latin alphabet characters (a-z, A-Z)	{x:alpha}
Bool	Matches a Boolean value.	{x:bool}
Datetime	Matches a DateTime value.	{x:datetime}
Decimal	Matches a decimal value.	{x:decimal}
double	Matches a 64-bit floating-point value.	{x:double}
float	Matches a 32-bit floating-point value.	{x:float}
guid	Matches a GUID value.	{x:guid}

Int	Matches a 32-bit integer value.	{x:int}
length	Matches a string with the specified length or within a specified range of lengths.	{x:length(6)} {x:length(1,20)}
Long	Matches a 64-bit integer value.	{x:long}
max	Matches an integer with a maximum value.	{x:max(10)}
maxlength	Matches a string with a maximum length.	{x:maxlength(10)}
min	Matches an integer with a minimum value.	{x:min(10)}
minlength	Matches a string with a minimum length.	{x:minlength(10)}
range	Matches an integer within a range of values.	{x:range(10,50)}
regex	Matches a regular expression.	{x:regex(^\\d{3}-\\d{3}-\\d{4}\$)}

ACTION RESULTS IN WEB API

WEB API Controller Action

- Web API uses different mechanisms to create the **HTTP response**

Return type	How Web API create the response
Void	Return empty 204 (No Content)
HttpResponseMessage	Convert directly to an HTTP response message
IHttpActionResult	Call ExecuteAsync to create an HttpResponseMessage, then convert to an HTTP response message
Other type	Write the serialized return value into the response body; return 200(OK)

VOID

- If the return type is void, Web API simply returns an empty HTTP response with status code **204 (NO CONTENT)**

```
// POST: api/Test  
0 references  
public void Post  
([FromBody]string value)  
{  
}
```

```
HTTP/1.1 204 No Content  
Server: Microsoft-IIS/8.0  
Date: Mon, 27 Jan 2014 02:13:26 GMT
```

HttpResponseMessage

- **HttpResponseMessage** return type, converts the return value directly into an HTTP response message using the properties of the **HttpResponseMessage** object to populate the response.
- The option gives you a lot of control over the response message

```
0 references
public class TestExampleController : ApiController
{
    0 references
    public HttpResponseMessage Get()
    {
        HttpResponseMessage response = Request.CreateResponse
            (HttpStatusCode.OK, "value");
        response.Content = new StringContent("Hello Syed Awase Khirni",
            System.Text.Encoding.Unicode);
        response.Headers.CacheControl = new
            System.Net.Http.Headers.CacheControlHeaderValue()
        {
            MaxAge = TimeSpan.FromMinutes(20)
        };
        return response;
    }
}
```

IHttpActionResult

- Introduced in Web API2
- *IHttpActionResult* Interface defines an HttpResponseMessage factory
- Advantages of using IHttpActionResult
 - Simplifies unit testing webapi controllers
 - Moves common logic for creating HTTP responses into separate classes
 - Makes the intent of the controller action clearer, by hiding the low-level details of constructing the response

Other Return Types

- Web API uses a media formatter to serialize the return value. Web API writes the serialized value into the response body.
- A disadvantage of this approach is that you cannot directly return an error code e.g. 404.
- You can only throw an **HttpResponseException** for error codes.

```
public class ProductsController : ApiController
{
    public IEnumerable<Product> Get()
    {
        return GetAllProductsFromDB();
    }
}
```

Parameter Binding in ASP.NET WEB API

- When a WEB API calls a method on a controller, it must set values for the parameters, a process called binding.

Rules of binding

- Simple types** – include .NET primitive types (int, bool, double, + TimeSpan, DateTime, Guid, decimal and String)
- Complex types** – Web API tries to read the value from the message body, using a media-type formatter

```
HttpResponseMessage Put(int id, Product item) { ... }
```

Simple type

Complex Type

FromUri

- To force WEB API to read a complex type from the URI
- Add the **[FromUri]** attribute to the parameter.

```
0 references
public class GeoPointController : ApiController
{
    0 references
    public HttpResponseMessage Get([FromUri] GeoPoint location)
    {
        HttpResponseMessage response = Request.CreateResponse
            (HttpStatusCode.OK, "value");
        string str = location.Latitude + " " + location.Longitude;
        response.Content = new StringContent(str,
            System.Text.Encoding.Unicode);
        response.Headers.CacheControl = new
            System.Net.Http.Headers.CacheControlHeaderValue()
        {
            MaxAge = TimeSpan.FromMinutes(20)
        };
        return response;
    }
}
```

```
0 references
public class GeoPoint
{
    1 reference
    public double Latitude { get; set; }
    1 reference
    public double Longitude { get; set; }
}
```

<http://localhost/api/geopoint/?Latitude=47.678558&Longitude=122.130989>

JSON (JavaScript Object Notation)

- A lightweight text based data-interchange format
- Completely language independent
- Based on a subset of the JavaScript Programming Language
- Easy to understand, manipulate and generate

NOT

- Overly Complex
- A “document” format
- A markup language
- A programming language

JSON Data Types

- Integer, real, scientific
- Strings (wrapped in double quotes)
- No octal or hex
- No NaN or Infinity – Use **null** instead
- Booleans: true or false
- Null: A value that specifies nothing or no value
- Objects: Unordered key/value pairs wrapped in { }
- Arrays: Ordered key/value pairs wrapped in []

JSON Example

```
var employeeData = {  
    "employee_id": 1236937,  
    "name": "Jeff Fox",  
    "hire_date": "1/1/2013",  
    "location": "Norwalk, CT",  
    "consultant": false,  
    "random_nums": [ 24,65,12,94 ]  
};
```

XML (Extensible Markup Language)

- Structure of document
- XML document is viewed as tree
- Elements divide the document into its constituent parts
- Elements are organized into a hierarchy

```
<?xml version="1.0"?>
```

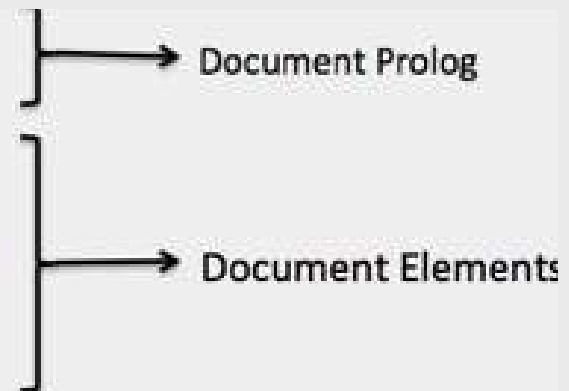
```
<contact-info>
```

```
  <name>Tanmay Patil</name>
```

```
  <company>TutorialsPoint</company>
```

```
  <phone>(011) 123-4567</phone>
```

```
</contact-info>
```



An Example XML Document

```
<!-- \Prolog"    -->
<?xml version="1.0"?>
<!DOCTYPE Greeting SYSTEM "wel.dtd">
<!-- End of Prolog -->
<!-- Content-->
<Greeting>
<from>Instructor</from>
<greetings>Welcome to XML</greetings>
<to>CSIB 313 Class</to>
<myGreetings>
<greetings>Welcome to XML</greetings>
<greetings>Welcome to XML</greetings>
</myGreetings>
</Greeting>
```

JSON vs. XML

Similarities

- Plain text formats
- “Self-describing” (human readable)
- Hierarchical (Values can contain lists of objects or values)

Difference

- JSON is lighter and faster than XML
- JSON uses typed objects. All XML values are type-less strings and must be parsed at runtime.
- JSON has less syntax, no semantics
- JSON properties are immediately accessible to JS code

Thank You!